Generalizing the Quarkus Dev-Agent

https://share.gemini.google/rfWG33bbLkAI

Die folgenden Antworten wurden mit dem Projekt eines Creators nach benutzerdefinierten Anweisungen erstellt. 25. Oktober 2026 um 14:48
Veröffentlicht am 25. Oktober 2026 um 18:05

> Dev-Agent generalization: what needs to be done to covert the existing concept to a generalized dev-agent that is usable for all my Quarkus projects?

To convert the existing **OmniDepot Dev-Agent** (`tools/dev-agent`) into a **generalized Quarkus Dev-Agent** usable across all your Quarkus projects, you need to transition from an embedded, repo-specific tool into an independent, universally installable CLI engine.

The original design already established the core architectural principle of **Engine vs. Knowledge Separation**—where the CLI acts as a generic execution engine and `.agents/` provides project-specific declarative knowledge. However, the current implementation contains domain couplings, hardcoded OmniDepot rules, and embedded repository assumptions that must be decoupled.

1. Repository & Packaging Decoupling (Standalone CLI)

Extract from Monorepo to Standalone Project: Currently, the agent is coupled to OmniDepot as a sub-module under `tools/dev-agent` behind a `-Pdev-tools` Maven profile. Extract it into a dedicated repository (e.g., `quarkus-dev-agent` or `dev-agent-cli`) with its own independent build, lifecycle, and release pipeline.

Global Installation & Distribution: Package the GraalVM native binary as a globally executable CLI (installed to `~/.local/bin/dev-agent` , or distributed via Homebrew, SDKMAN!, or JBang). When invoked in any project directory (`user.dir`), the agent dynamically resolves the target project's context.

2. Generalizing AI Personas and Dynamic Prompts

In the OmniDepot blueprint, the system prompts hardcode project-specific identities and ADRs:

Remove Hardcoded Identities & Domain Rules: The current `@RegisterAiService` declarations hardcode prompts such as *"You are the Lead Software Engineer for OmniDepot"* and explicitly cite OmniDepot ADRs (ADR-001, ADR-006 Zero-GC streaming, ADR-015 CAS).

Dynamic Quarkus Personas ...a to an expert **Quarkus Senior**

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

active project's `.agents/rules/` and ADR directories, rather than compiling them into system messages.

Externalized System Prompt Overrides: Allow projects to provide custom system instructions in `.agents/prompts/` or `.agents/dev-agent.yaml` to override default personas without recompilation.

3. Generalizing State Persistence & Multi-Tenancy

Relocate and Namespace the Local Database: The embedded H2 database is currently hardcoded to `~/.omnidepot/dev-agent`.

- Relocate the default database path to `~/.config/dev-agent/data/agent.mv.db` (or `~/.quarkus-dev-agent/state.mv.db`).
- Introduce **Project Multi-Tenancy**: Tag all `TaskRunEntity`, audit traces, and metric records with a unique `projectId` or repository coordinate (e.g., derived from Git remote URL or directory path hash). This ensures tasks, runs, and metrics from different Quarkus repositories do not collide in the shared local database.

Dynamic Markdown Projection: Ensure `StateSyncService` writes projections strictly to the target project's `./.agents/runs/<task-id>.md` based on the active execution directory.

4. Build Engine & Project Structure Generalization

The current agent relies heavily on Maven multi-module assumptions tailored to OmniDepot's layout:

Auto-Detect Project Layout (Single-Module vs. Multi-Module):

- In OmniDepot, the agent resolves `-pl <module>` coordinates via `SubmoduleReactorResolver`.
- Generalize `MavenExecutionTools` to detect whether a Quarkus project is a standalone single-module application or part of a multi-module reactor. If single-module, omit `-pl` and `-am` flags dynamically.

Build Tool Abstraction (Maven Wrapper vs. System Maven vs. Gradle):

- Detect whether the target project uses `./mvnw`, `mvn`, `./gradlew`, or `gradle`.
- Adapt target isolation: Use `-Dproject.build.directory=target-agent` for Maven, or `-PbuildDir=build-agent` for Gradle projects to prevent lock contention with IDEs.

Quarkus Dev Mode & Container Handling:

- Ensure the test execution validator detects and supports Quarkus continuous testing configurations and skips Testcontainers/Docker boots during micro-TDD loops unless explicitly required.

5. Universal Quarkus Rules & Pluggable Archetypes (`dev-agent init`)

The `dev-agent init` command currently extracts OmniDepot-specific templates (CAS limits, Zero-GC Netty rules) from `harness-templates/`.

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

CDI Scoping guidelines (`@ApplicationScoped` vs `@RequestScoped` , avoiding circular dependencies).

Build-Time vs. Runtime rules (proper usage of `@RegisterForReflection` , avoidance of unsupported dynamic reflection).

Reactive vs. Blocking routes (avoiding blocking Vert.x event loops, using `@Blocking` or Mutiny appropriately).

Quarkus Panache & Persistence best practices.

Archetype/Profile Flavors on `init` :

Expand `InitCommand` to accept profile templates:

```bash
dev-agent init --type=rest-panache
dev-agent init --type=reactive-messaging
dev-agent init --type=modulith
```

This initializes customized rules in `.agents/rules/` matching the project's specific architectural style while keeping the core engine generic.

6. Dynamic ADR & Invariant Ingestion (Local RAG)

Flexible ADR Discovery: Instead of looking specifically for OmniDepot's `docs/tech/architecture/adrs.md` , the semantic indexing service should dynamically discover architectural documents across standard project locations:

`docs/adr/*.md` , `doc/architecture/decisions/` , `.agents/rules/*.md` , or `ADR.md` .

Universal ArchUnit Governance: Provide generalized ArchUnit rules in the base toolkit to enforce standard Quarkus package boundaries, immutability, and CDI best practices across all your projects.

7. Configurable Remote Platforms & Quality Gates

Parameterize GitHub & SonarCloud Coordinates:

In OmniDepot, remote GitHub API clients and SonarCloud project keys are partially hardcoded or tightly coupled.

Read repository coordinates (`owner/repo`), branch defaults, and Sonar project keys dynamically from Git remotes (`git remote get-url origin`) or `.agents/dev-agent.yaml` .

Optional Quality Gates:

Allow projects that do not use SonarCloud or CodeRabbit to cleanly toggle those phases off in `.agents/dev-agent.yaml` :

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
agent:
  ci:
    provider: github-actions
  sonar:
    enabled: false
  coderabbit:
    enabled: false
```

The orchestrator can then transition directly from green CI checks to ready-for-review status without failing on missing SonarCloud configurations.

Summary Checklist for Generalization

Area	OmniDepot-Specific (Current)	Generalized Quarkus Dev-Agent
Location & Packaging	Submodule in `tools/dev-agent` under `-Pdev-tools`	Standalone repository; installed globally to `~/.local/bin/dev-agent`
Prompts & Personas	Hardcoded OmniDepot domain, ADR-001..028	Universal Quarkus expert persona; ADRs injected dynamically via RAG
Local Database	Fixed at `~/.omnidepot/dev-agent`	Global `~/.config/dev-agent/` with project-namespaced multi-tenancy
Templates (`init`)	OmniDepot Zero-GC and CAS templates	Universal Quarkus archetypes (Reactive, REST/Panache, Modulith)
Build Reactor	Fixed Maven multi-module (`-pl` , `-am`)	Auto-detects single vs. multi-module; Maven wrapper vs. Gradle
ADR Ingestion	Fixed paths to OmniDepot ADR documents	Discovers arbitrary `docs/adr/` and `.agents/rules/` dynamically
Remote Services	OmniDepot GitHub repo & SonarCloud key	Inferred from Git remote or configured in `.agents/dev-agent.yaml`

Can I include front-end capabilities as well?

Yes, absolutely. Expanding the generalized Dev-Agent into a **full-stack agent** (supporting front-end engineering alongside Quarkus backends) is a natural fit—especially since Quarkus architectures often package single-page applications (Angular, React, Vue) directly into `META-INF/resources/` via plugins like `frontend-maven-plugin` or extensions like **Quarkus Quinoa**.

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

1. Dedicated Front-End Tooling Suite (`FrontendExecutionTools`)

Just as you have `MavenExecutionTools` for running targeted Java tests, you introduce an `@ApplicationScoped` CDI bean for front-end execution:

```
FrontendExecutionTools (@Tool)
  • runTargetedFrontendTest(testPattern)
  • runTypeCheck() (tsc --noEmit)
  • runLint(filePath) (eslint / ng lint)
  • runStorybookTests() (test-storybook)
  • generateTypeScriptApiClient() (OpenAPI Codegen)
```

Package Manager Auto-Detection: Auto-detect whether the project uses `pnpm` , `npm` , `yarn` , or `bun` by checking for lockfiles (`pnpm-lock.yaml` , `package-lock.json` , etc.) in the project or UI submodule.

Targeted UI Test Execution: Instead of running the entire web test suite, execute single test files during micro-TDD loops (e.g., `npx jest src/app/.../spec.ts` , `vitest run <file>` , or `ng test --include=...`).

Log Sanitization for JS/TS: Extend `LogSanitizerService` to strip noise from npm/Vite/Webpack/TypeScript outputs, extracting only the exact line errors, missing types, and failing assertions for the LLM prompt.

2. Contract-Driven Synchronization (Quarkus OpenAPI → TypeScript)

One of the biggest pain points in full-stack development is frontend models drifting from backend DTOs. The Dev-Agent can automate this end-to-end:

1. **Quarkus OpenAPI Schema Generation**: Quarkus exposes or builds the OpenAPI specification via SmallRye OpenAPI (at `/q/openapi` or via `mvn quarkus:generate-code`).
2. **Automated Client Generation Tool**:

   Add a tool `generateApiClient()` that triggers tools like `openapi-generator-cli` or `orval` .

   When implementing a new backend endpoint, the agent automatically regenerates strongly typed TypeScript interfaces and HTTP services in the front-end directory, eliminating manual model duplication.

3. Front-End Architectural Rules & Archetypes (`.agents/rules/`)

Under the Engine vs. Knowledge separation model, your core Java CLI remains generic, while front-end guidelines are declared in `.agents/rules/` :

Framework Invariants (e.g., for Angular):

Gemini Canvas ausprobieren

Modern reactivity rules: Prefer Angular Signals over raw RxJS subjects for component

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Memory leak prevention (destroyRef / `takeUntilDestroyed`).

Design System & UI Tokens:

Enforcing design tokens (such as Material Design 3 tokens: Reactive Cyan, Carbon Gray surfaces, dark-first styling).

WCAG AA accessibility standards (contrast ratios, aria attributes, keyboard navigation).

Component Isolation with Storybook:

Rules requiring every reusable widget in component libraries to have an accompanying `.stories.ts` file.

The agent can write and verify Storybook stories in isolation without requiring the Quarkus server to be running.

4. Build Engine & Lifecycle Integration

Quarkus projects typically integrate front-ends in one of two ways:

Embedded SPA Packaging: The front-end build outputs directly into `src/main/resources/META-INF/resources/` (via `frontend-maven-plugin` or Quarkus Quinoa), served natively as static assets by Quarkus Vert.x HTTP.

Monorepo / Workspace Layout: Multi-project setups (such as Nx workspaces or Angular multi-projects) with separate build pipelines.

The agent's workspace resolver should:

Detect whether a task touches the Java backend (`src/main/java`), the frontend (`src/main/webapp/`, `projects/`, `frontend/`), or both.

Settle targeted execution commands accordingly (run Maven test compile for Java, run `tsc --noEmit` or Jest for TypeScript).

5. Multi-Language AST & IDE Diagnostics

The progressive enhancement layer (via IntelliJ MCP on `http://localhost:63342`) already supports IDE symbol lookups and file diagnostics.

Because IntelliJ IDEA / WebStorm natively analyzes TypeScript, HTML, and CSS, the agent's MCP diagnostics automatically surface frontend compiler and linter errors without needing a separate TypeScript Language Server integration.

6. Embedded Local Diff & Component Review UI (`--gui`)

When working on front-end components, approving terminal diffs via ANSI strings is difficult:

You can leverage the agent's optional embedded Vert.x review server (`dev-agent feature --gui` on `http://localhost:9999`).

The local web dashboard can display side-by-side syntax-highlighted code diffs (via `diff2html`) and even embed an `<iframe>` pointing to the local Storybook preview server (`http://localhost:6006`) so you can visually inspect newly generated components before approving the patch.

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
                    User Invocation:
  $ dev-agent feature "Add Storage Quota Progress Meter"
                            │
                            ▼
1. Phase 1 (Plan): Agent identifies Backend DTO + REST Endpoint
                   + Angular Component + Storybook Story.
                            │
                            ▼
2. Phase 2 (Backend TDD): Quarkus Panache Entity & REST Endpoint
                          verified via targeted Maven test.
                            │
                            ▼
3. Phase 3 (API Codegen): Agent runs OpenAPI generator to update
                          TypeScript client interfaces.
                            │
                            ▼
4. Phase 4 (Frontend TDD): Agent creates Component & Storybook story;
                           verifies with `test-storybook` & `tsc`.
                            │
                            ▼
5. Phase 5 (Review Gate): User inspects full-stack diff & Storybook
                          preview on http://localhost:9999.
                            │
                            ▼
6. Phase 6 (Commit & PR): Conventional commit & Draft PR created.
```

By adding `FrontendExecutionTools` and dynamic front-end rule scaffolding to your CLI, the generalized agent transitions smoothly into a full-stack developer companion across all your Quarkus repositories.

If this is the final goal, should I also start in a separate repository?

Yes, absolutely. If the final goal is a generalized, full-stack agent usable across all your Quarkus and frontend projects, **you should start in a separate, dedicated repository from Day 1.**

While embedding the agent into OmniDepot under `tools/dev-agent` made sense when it was conceived strictly as an internal pair-programmer tied to OmniDepot's 28+ ADRs, shifting to a generalized multi-project agent fundamentally changes its role: **it is no longer a sub-feature of a single product, but developer infrastructure.**

Starting directly in a separate repository (e.g., `quarkus-dev-agent` or `dev-agent-cli`) provides decisive architectural and practical advantages:

1. It Enforces the "Engine vs. Knowledge" Boundary from Commit 1

Gemini Canvas ausprobieren

The greatest risk of building a generic agent inside an existing project is **accidental gravity**:

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

bake in domain assumptions (such as CAS chunking or Netty zero-GC constraints).

Starting in a separate repository forces you to honor the core rule: The agent is a zero-knowledge execution engine, and target projects provide their own declarative brains (`.agents/`).

If the engine needs to know something about a project, you will be forced to implement it properly via dynamic discovery (CLI flags, Git remote detection, project inspection, or `.agents/rules/`).

2. OmniDepot Becomes "Customer Zero"

Developing the CLI in a separate repository enables a clean, realistic testing methodology:

True Dogfooding: You test the agent by running it against OmniDepot as an external consumer (`cd ~/workspace/omnidepot && dev-agent feature ...`).

Immediate Multi-Project Testing: You can also point the agent at an empty Quarkus starter project (`code.quarkus.io`), a standalone Angular app, or a simple single-module service.

If the agent works seamlessly on both a lean microservice and OmniDepot's complex multi-module layout, you have proven its generalization immediately—without needing a risky "extraction phase" later.

3. Decoupled Build Lifecycles & GraalVM Pipelines

Heavy Native Builds: Compiling GraalVM native binaries with `quarkus-langchain4j` , Hibernate Panache, and H2 takes considerable memory and time (often 2–5 minutes). Keeping that in OmniDepot's Maven reactor—even behind `-Pdev-tools` —adds clutter and configuration overhead to the product codebase.

Independent Versioning & Releases: As an infrastructure tool, the Dev-Agent follows its own semantic versioning (SemVer) and release cadence independent of your applications. You can upgrade LangChain4j, LLM models, or terminal UX without touching the commit history of your business projects.

4. Avoiding the "Git Extraction Debt"

Projects that start embedded in monorepos with the intention of being "extracted later" almost always accumulate technical debt:

Maven dependencies get entangled with root BOMs and dependency management blocks .

Splitting the Git history later requires complex and error-prone `git subtree` or `git-filter-repo` migrations to preserve author attribution and issues.

Test suites written with monorepo file fixtures need to be rewritten.

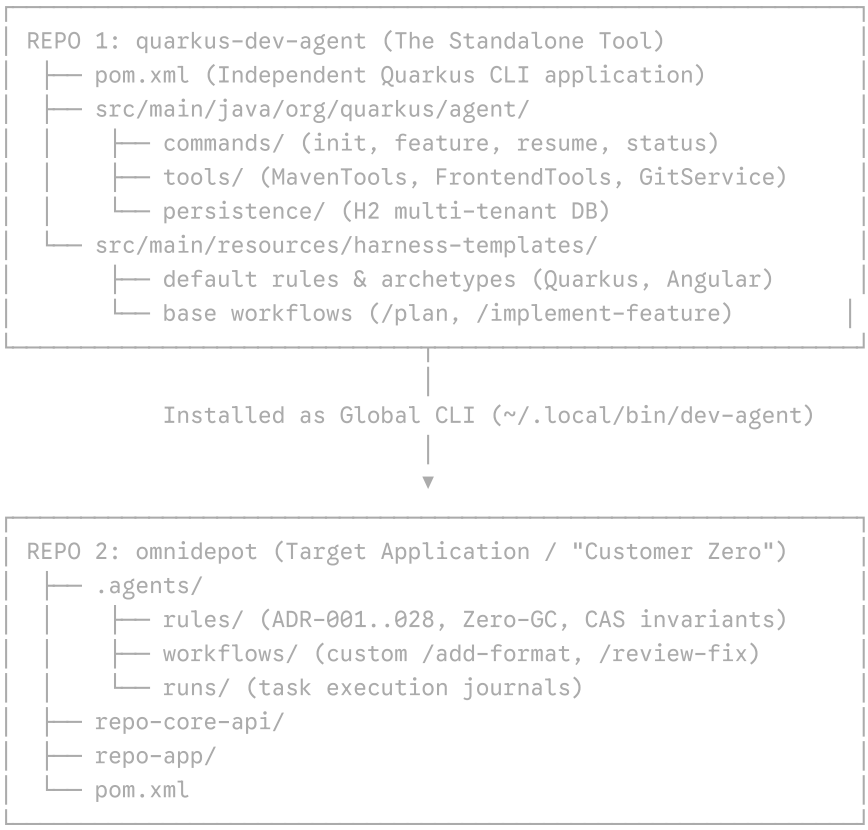
Starting clean means your test suite will naturally use hermetic `@TempDir` mock projects and dynamic fixtures from the very beginning.

Gemini Canvas ausprobieren

What the Two Repositories Look Like

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps



How to Maintain Fast Development Velocity Across Repos

You don't lose velocity by separating repositories if you use a lightweight local workflow:

1. **Local Linking for Testing:**
   During active agent development, use a local bash alias or symlink pointing to your development build rather than rebuilding native binaries every time:

```bash
# In ~/.bashrc or ~/.zshrc
alias dev-agent-dev="java -jar /path/to/quarkus-dev-agent/target/quarkus-app/qua
```

2. **Instant Feedback:**
   You can run `mvn quarkus:dev` inside `quarkus-dev-agent` for instant live-reload while modifying prompts or tools, or test CLI invocations directly in any directory with Quarkus test runners (`@QuarkusMainTest`).

3. **OmniDepot Repository Stays Clean:**
   OmniDepot's repository will only contain the declarative `.agents/` folder (rules, workflows, and configurations), keeping the product repository focused strictly on its business domain.

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Start in a separate repository. Building the CLI as an external tool from day one forces clean abstractions, keeps domain coc
OmniDepot, it is already ready

Which paths have to be configurable or auto-detected in this new product vision

In the generalized, full-stack product vision, the Dev-Agent must transition from hardcoded assumptions to a clean division of **auto-detected paths** (discovered at runtime via conventions) and **configurable paths** (overridable via Picocli flags, environment variables, or the configuration cascade).

The paths requiring configuration or auto-detection are categorized below:

1. Workspace & Project Discovery Paths

Previously, the agent assumed an OmniDepot monorepo layout with Maven reactor modules. In a generalized vision:

Project Root (`REPO_ROOT` / `workingRoot`):

Auto-Detection: Discovered by traversing upward from `System.getProperty("user.dir")` until finding the nearest `.git` directory, root `pom.xml` , or `build.gradle` .

Configurable: Can be explicitly overridden via Picocli `--dir=<path>` or `--workspace=<path>` .

Build Tool Executable:

Auto-Detection: Probes for `./mvnw` → `mvn` → `./gradlew` → `gradle` within the detected root.

Configurable: `agent.build.tool-executable` in `.agents/dev-agent.yaml` or via environment variable `BUILD_TOOL_EXECUTABLE` .

Submodule Reactor & Project Layout:

Auto-Detection: Replaces the OmniDepot-specific `SubmoduleReactorResolver` . It inspects whether the root POM contains `<modules>` (or Gradle `settings.gradle` includes subprojects). If multi-module, it derives `-pl <relative-module-path> -am` dynamically; if single-module, it omits `-pl` entirely.

Target Build Isolation Directory:

Auto-Detection / Default: Defaults to `target-agent/` (Maven) or `build-agent/` (Gradle) so the agent's builds never lock IDE background compilation.

Configurable: `agent.build.target-directory` (e.g., `-Dproject.build.directory=...`).

2. Front-End & Static Asset Paths (Full-Stack Support)

Quarkus serves Single Page Applications (SPA) as static assets, but frontend directories vary between projects:

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Auto-Detection: Scans the workspace for common frontend layouts: `frontend/`, `src/main/webapp/`, `ui/`, `web/`, or dedicated workspace sub-packages (e.g., in multi-module or Nx setups).

Configurable: `agent.frontend.root-dir` in `.agents/dev-agent.yaml` (e.g., `frontend` or `ui`).

Package Manager & Lockfiles:

Auto-Detection: Probes the frontend root for `pnpm-lock.yaml` (`pnpm`) → `package-lock.json` (`npm`) → `yarn.lock` (`yarn`) → `bun.lockb` (`bun`).

Configurable: `agent.frontend.package-manager` (e.g., `pnpm run test`, `npm test`).

Quarkus Static Asset Target (`META-INF/resources`):

Auto-Detection / Default: Defaults to `src/main/resources/META-INF/resources/` (standard Quarkus static web serving path via Quinoa or `frontend-maven-plugin`).

Configurable: `agent.frontend.output-dir`.

OpenAPI Specification Path:

Auto-Detection: Checks for generated OpenAPI specs at `target/openapi.yaml`, `target/classes/META-INF/openapi.yaml`, or `/q/openapi`.

Configurable: `agent.frontend.openapi-spec-path` (used by the TypeScript client code generator).

3. Declarative AI Harness Paths (`.agents/`)

The `.agents/` directory is the single source of truth for repository knowledge:

Harness Base Path:

Default: Resolved relative to the active workspace as `./.agents/`.

Configurable: Injected via SmallRye Config `@ConfigMapping` (`@WithDefault(".agents") Path harnessDir();`) or the Picocli `init --dir=<path>` flag.

Workflows Directory:

Default: `${harnessDir}/workflows/`.

Behavior: Auto-detects all `*.md` files (such as `plan.md`, `implement-feature.md`, `review-fix.md`) to dynamically populate slash commands without recompilation.

Rules & Invariants Directory:

Default: `${harnessDir}/rules/`.

Behavior: Scanned and loaded into prompt contexts or indexed for local RAG.

Skills Directory:

Default: `${harnessDir}/skills/` (e.g., `skills/*/SKILL.md`).

Gemini Canvas ausprobieren

Execution Journals & Resumption:

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Configurable: `agent.runs-dir`.

4. Architectural Decisions & Documentation Paths (Local RAG)

The original implementation hardcoded OmniDepot's ADR file:

Architecture Decision Records (ADRs):

Hardcoded in OmniDepot: `docs/tech/architecture/adrs.md`.

Auto-Detection in Generalized Vision: Probes standard ADR conventions:

`docs/adr/` or `doc/adr/`

`docs/architecture/decisions/`

`docs/tech/architecture/adrs.md`

Root `ADR.md`

Configurable: `agent.knowledge.adr-paths` (list of glob paths or markdown files to index into the LangChain4j embedding store).

5. Multi-Tier Configuration & User-Scope Paths

To prevent secrets or user preferences from contaminating target repositories, configuration is tiered:

User-Scope Configuration (Global):

Default Path: `${user.home}/.config/dev-agent/application.yaml` (Linux/macOS) or `%APPDATA%\dev-agent\application.yaml` (Windows).

Purpose: Stores personal PATs, API keys (`GEMINI_API_KEY`, `OPENROUTER_API_KEY`), and WSL2 host bridge addresses.

Project-Scope Configuration (Repository):

Default Path: `${REPO_ROOT}/.agents/dev-agent.yaml` and `${REPO_ROOT}/.agents/config.json`.

Purpose: Declares project-wide SLAs, active AI model mappings, and test patterns.

User Prompts & Skills Fallback:

Default Path: `${user.home}/.config/dev-agent/prompts/` and `${user.home}/.config/dev-agent/skills/`.

Behavior: Used as a fallback when a project does not supply its own `.agents/prompts/`.

6. Persistence & Embedded Database Paths

Local H2 Database File:

Old OmniDepot Path: Hardcoded to `jdbc:h2:file:~/.omnidepot/dev-agent`.

Generalized Vision: Must be decoupled to the agent's application directory:

Gemini Canvas ausprobieren

Default: `${user.home}/.config/dev-agent/data/agent.mv.db`.

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Multi-Tenancy: All tables (`TaskRunEntity` , audit traces) store a `project_id` or repository root hash to isolate tasks between different Quarkus projects.

7. Sandboxing, Git Worktrees & Execution Jails

Git Worktree Isolation:

Default Path: `${REPO_ROOT}/.git/agent-worktrees/<task-id>/` .

Configurable: `agent.worktree-dir` .

Workspace Jail Boundary:

Auto-Detection: `WorkspaceJailValidator` roots itself to the active `workingRoot` (either the project root or the isolated worktree).

Protected System Paths: The validator strictly blocks traversal (`../`) and forbids writes to `.git/` , the CLI installation directory (`~/.local/bin/dev-agent`), and user configuration directories.

Summary Checklist of Path Responsibilities

Category	Path Component	Resolution Strategy	Default / Fallba
Workspace	Project Root (`REPO_ROOT`)	Auto-detected	Upward search f `.git` or `pom.`
Workspace	Build Executable (`mvnw` / `gradlew`)	Auto-detected	`./mvnw` → `mv` `gradle`
Workspace	Build Output Directory	Configurable / Convention	`target-agent` `build-agent/`
Frontend	UI Source Directory	Auto-detected / Configurable	`frontend/` , `u` `src/main/web`
Frontend	Web Distribution Output	Auto-detected / Configurable	`src/main/res` `INF/resource`
Harness	AI Harness Root	Configurable via CLI / Config	`${REPO_ROOT}`
Harness	Workflows & Rules	Auto-detected	`.agents/work` `.agents/rule`
Knowledge	ADRs & Invariants	Auto-detected / Configurable	`docs/adr/` , `doc/architec` `docs/tech/ar`
State	Embedded H2 Database	Configurable / Global	`${user.home}` `agent/data/a`
User Scope	Personal Config & Secrets	Fixed OS Convention	`${user.home}` `agent/applic`
Isolation	Detached Git Worktrees	Convention / Configurable	`.git/agent-w` `id>/`

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

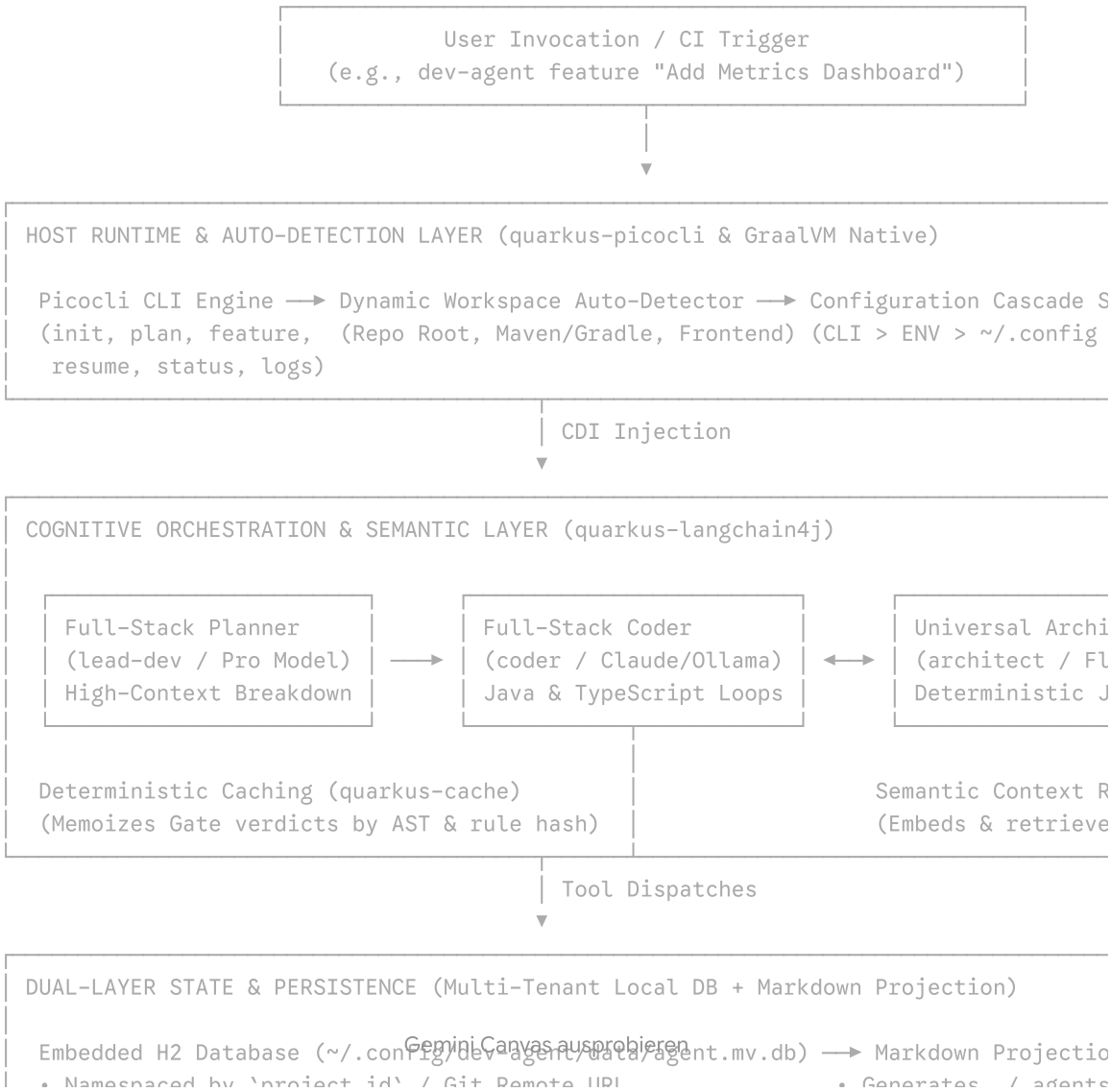
Add the updated architecture definition for the generalized agent

Technical Architecture Specification: Generalized Full-Stack Dev-Agent (`quarkus-dev-agent`)

1. Executive Summary & Paradigm Shift

The **Generalized Quarkus Dev-Agent** is an autonomous pair-programmer and full-stack software-engineering CLI built on **Quarkus 3.x**, **Picocli**, and **Quarkus-LangChain4j**.

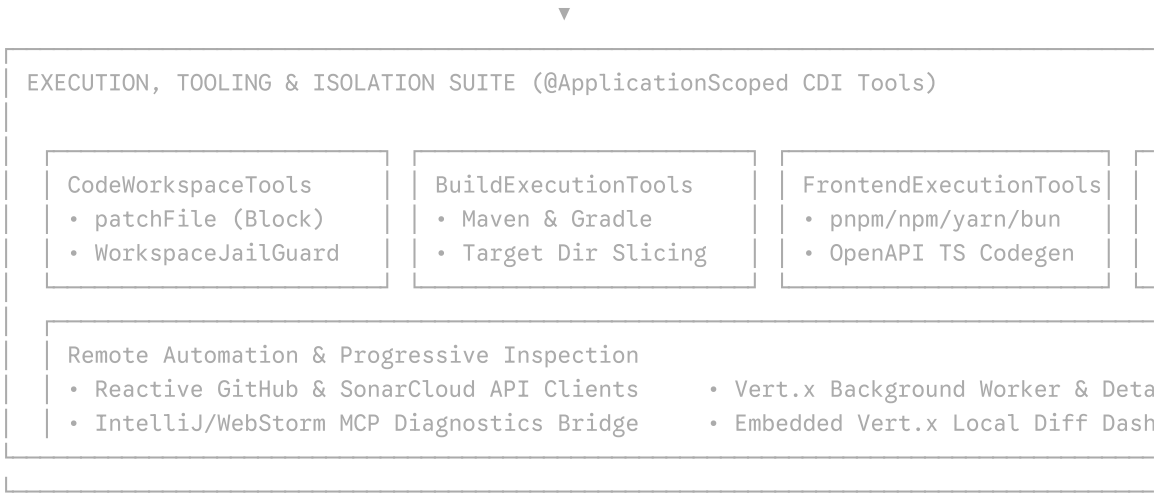
Unlike single-repository pair tools, this system is architected as an **independent developer infrastructure tool** hosted in its own repository and distributed as a globally installed GraalVM Native Image binary (`~/.local/bin/dev-agent`).



Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps



Core Architectural Invariants

1. **Engine vs. Knowledge Separation**: The CLI engine contains **zero hardcoded project domain rules or repository paths**. All architectural constraints, ADRs, and workflows are ingested dynamically from the target project's `.agents/` and documentation directories.
2. **Full-Stack Execution Parity**: Backend Java/Quarkus code and frontend TypeScript/framework components (Angular, React, Vue) are first-class targets with dedicated execution, linting, type-checking, and test tools.
3. **Three-Way Workflow Compatibility**: Output journals and scaffolded harnesses remain 100% interoperable with manual developers, generic agentic CLIs (Antigravity `agy`, Claude Code, Aider), and the native `dev-agent` binary.
4. **Isolated Workstation Multi-Tenancy**: The local state engine maintains complete isolation between different projects via workspace namespacing in an embedded H2 database. Isolated Git worktrees and build directories prevent collision with developer IDEs.

2. Host & Runtime Layer

The host layer is packaged as an independent CLI built on `quarkus-picocli`, configured for Ahead-of-Time (AOT) GraalVM compilation (<50 ms cold boot).

Dynamic Workspace & Build Auto-Detection

At startup, `WorkspaceContextResolver` inspects `System.getProperty("user.dir")` and initializes runtime coordinates:

Workspace Root (`REPO_ROOT`): Discovered by traversing parent directories to locate the root `.git` directory, root `pom.xml`, or `build.gradle`.

Build Engine Detection:

Probes execution binaries in order: `./mvnw` → `mvn` → `./gradlew` → `gradle`.

Inspects project structure: Resolves whether the application is a single-module project or a multi-module reactor. For multi-module Maven projects, it derives `-pl <relative_module_path> -am` dynamically; for single-module projects `-pl`

Gemini Canvas ausblenden

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Appends `-Dproject.build.directory=target-agent` (Maven) or `-PbuildDir=build-agent` (Gradle) during automated verification to prevent lock contention with background IDE compilation and `quarkus:dev`.

Frontend Subsystem Auto-Detection:

Detects frontend root directories by searching for `frontend/`, `src/main/webapp/`, `ui/`, `web/`, or workspace packages containing `package.json`.

Inspects package managers via lockfile presence: `pnpm-lock.yaml` (`pnpm`) → `package-lock.json` (`npm`) → `yarn.lock` (`yarn`) → `bun.lockb` (`bun`).

CLI Commands & Ergonomics

`init [--type=<archetype>]`: Scaffolds the `.agents/` harness (rules, workflows, configs) into the target project using embedded templates. Archetypes include `rest-panache`, `reactive-messaging`, and `fullstack-angular`.

`plan "<story>"`: Generates an ADR-aligned DAG implementation plan covering backend and frontend layers.

`feature "<story>" [--gui] [--detach]`: Executes the full-stack TDD implementation lifecycle.

`resume [<task-id>]`: Hydrates and resumes interrupted sessions from the database and disk journals.

`status`: Displays running background tasks, active worktrees, and recent run metrics.

`logs --follow <task-id>`: Streams logs from detached background workers or remote CI pollers.

3. Cognitive Orchestration & Semantic Layer

Powered by `quarkus-langchain4j`, the cognitive layer decouples LLM provider bindings from orchestration logic.

Java

```java
// Role 1: Full-Stack Feature Planner
@RegisterAiService(modelName = "planner")
public interface FullStackPlannerAiService {
    @SystemMessage("""
        You are a Principal Full-Stack Software Architect specializing in Quarkus and
        Analyze repository interfaces, DTOs, schemas, and UI components.
        Generate a phased implementation DAG adhering to the project's architectural
        Project Guidelines:
        {rulesContext}
        """)
    String draftPlan(@UserMessage String userStory, @V("rulesContext") String rulesCo
}
```

Gemini Canvas ausprobieren

```
// Role 2: Universal Architect Gatekeeper
```

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
        Evaluate the provided plan against the project's active ADRs and architectura
        Relevant Architectural Decisions:
        {retrievedAdrs}
        Return ONLY valid JSON matching the GateVerdict schema.
        """)
    GateVerdict evaluatePlan(@UserMessage String plan, @V("retrievedAdrs") String ret

    @RegisterForReflection
    record GateVerdict(
        boolean approved,
        String summary,
        List<String> violations,
        List<String> recommendations
    ) {}
}

// Role 3: Full-Stack Code Synthesizer
@RegisterAiService(
    modelName = "coder",
    tools = { CodeWorkspaceTools.class, BuildExecutionTools.class, FrontendExecutionT
)
public interface FullStackCodeSynthesizerAiService {
    @SystemMessage("""
        You are the Full-Stack Implementation Engineer.
        1. Write failing tests first (TDD) for backend endpoints or frontend componen
        2. Apply precision modifications using 'patchFile'. Never overwrite entire fi
        3. When backend contracts change, regenerate TypeScript interfaces via 'gener
        4. Validate changes using targeted test tools.
        """)
    String executeImplementationStep(@UserMessage String instruction);
}
```

Semantic Invariant Ingestion (Local RAG)

Instead of serializing and injecting all repository ADRs into system prompts (which causes context bloating and attention drift):

Automatic Discovery: Ingestion probes standard paths: `docs/adr/`, `doc/architecture/decisions/`, `docs/tech/architecture/adrs.md`, `.agents/rules/`, and root `ADR.md`.

LangChain4j Vector Store: On startup, modified documents are chunked and embedded into an in-memory or Lucene-backed `EmbeddingStore` using a lightweight embedding model.

Selective Retrieval: When a task touches specific modules (e.g., an S3 persistence adapter or an Angular reactive service), the top 3–5 matching ADRs are retrieved and injected into `{retrievedAdrs}`.

Deterministic Cache Layer (`quarkus-cache`)

Architect audits execute with `temperature: 0.0`. Results are wrapped in `@CacheResult(cacheName = "architect-verdicts")` keyed by `SHA-256(diffContent + rulesHash)`. Minor formatting or comment edits bypass remote

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

4. Full-Stack Tooling & Safety Engine

All agent capabilities are implemented as thread-safe, `@ApplicationScoped` CDI beans exposing `@Tool` methods:

```
CodeWorkspaceTools (@Tool)
  • readFile(path, startLine, endLine)
  • patchFile(path, searchBlock, replaceBlock)
  • createFile(path, content)
  • findFiles(globPattern), searchCode(query)
```

```
BuildExecutionTools (@Tool)
  • runTargetedTest(testClassPattern)
  • runCompileCheck()
  • LogSanitizerService (ANSI/noise filter, token saver)
```

```
FrontendExecutionTools (@Tool)
  • runFrontendTest(testPattern)
  • runTypeCheck() (tsc --noEmit)
  • runLinter(filePath) (eslint / ng lint)
  • runStorybookTests() (test-storybook)
  • generateTypeScriptClient() (OpenAPI Codegen)
```

```
GitWorktreeService (@ApplicationScoped)
  • allocateWorktree(taskId, branchName)
  • createScratchpadSnapshot()
  • rollbackToSnapshot()
  • releaseWorktree(taskId)
```

Contract-Driven Synchronization (Quarkus OpenAPI → TypeScript)

To prevent drift between backend APIs and web clients:

1. Quarkus generates OpenAPI definitions via SmallRye OpenAPI (written to `target-agent/openapi.yaml` during compile checks).
2. `FrontendExecutionTools.generateTypeScriptClient()` triggers client generation (e.g., `openapi-generator-cli` or `orval`).
3. Strongly-typed TypeScript interfaces and API services are automatically updated in the frontend source tree before frontend TDD loops commence.

Progressive IDE Enhancement (MCP)

The agent queries the local IDE Model Context Protocol endpoint (`http://localhost:63342`) on boot:

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

When running headless or in CI, it falls back to native filesystem searches, `mvn test-compile`, and `tsc --noEmit`.

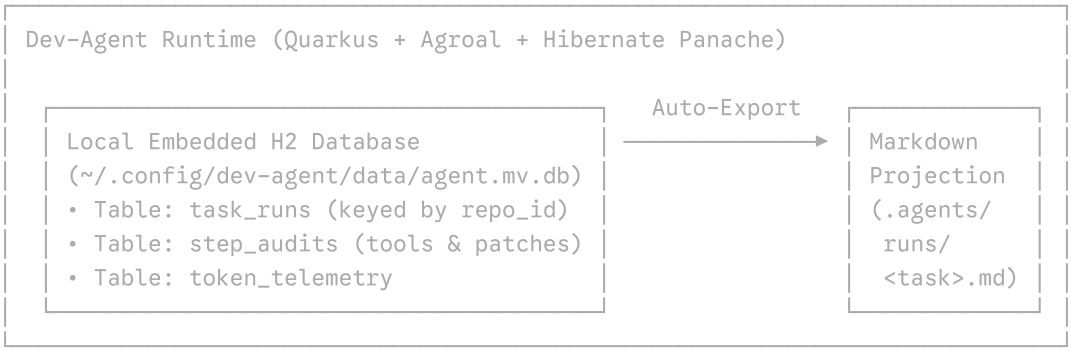
Execution Jails & Circuit Breakers

`WorkspaceJailValidator`: Restricts all file writes and reads within the canonical boundaries of the detected `REPO_ROOT` or allocated worktree. Forbids traversal into `.git/` metadata, the CLI installation directory (`~/.local/bin`), or system configuration files.

Oscillatory Doom-Loop Breaker: Hashes compiler stacktraces and patch hunks across turns. If identical error hashes occur across 2 consecutive self-healing iterations, the agent halts, executes `git stash` rollback, and exits with code 1.

5. Multi-Tenant State Engine & Persistence

The agent separates persistent internal state from external human-readable journals.



Embedded Relational Database (SSOT)

Location: Centralized at `${user.home}/.config/dev-agent/data/agent.mv.db`.

Isolation & Multi-Tenancy: Every database table includes a `project_id` column derived from the Git remote URL (or an MD5 hash of the repository path). Multiple Quarkus applications can run tasks concurrently without row collisions.

Concurrency Configuration: Connection URLs enforce `AUTO_SERVER=TRUE;DB_CLOSE_DELAY=-1;MODE=PostgreSQL`, allowing CLI operations (such as `dev-agent status`) to query state while a background worker is writing.

Markdown Projection Engine

After every atomic transaction in H2, `StateSyncService` regenerates `./.agents/runs/<task-id>.md` in the target repository. This ensures tools like `agy`, Claude Code, or human developers inspecting git can review progress without interacting with the binary database.

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
[1. Picocli CLI Flags]       (--offline, --max-retries=5, --yes, --gui)
        |
        ▼
[2. Environment Variables]   (GEMINI_API_KEY, AGENT_MAX_SELF_HEALING_RETRIES)
        |
        ▼
[3. User-Scope Config]       (~/.config/dev-agent/application.yaml)
        |
        ▼
[4. Project-Scope Config]    (.agents/dev-agent.yaml & .agents/config.json)
        |
        ▼
[5. Classpath Defaults]      (src/main/resources/application.yaml)
```

Java

```java
@ConfigMapping(prefix = "agent")
public interface GeneralizedAgentSettings {
    @WithDefault(".agents")
    Path harnessDir();

    @WithDefault("3")
    int maxSelfHealingRetries();

    @WithDefault("true")
    boolean interactiveDiffApproval();

    BuildSettings build();
    FrontendSettings frontend();
    CiSettings ci();
    SonarSettings sonar();

    interface BuildSettings {
        @WithDefault("target-agent")
        String targetDir();
        Optional<String> toolExecutable();
    }

    interface FrontendSettings {
        @WithDefault("frontend")
        String rootDir();
        Optional<String> packageManager();
        @WithDefault("src/main/resources/META-INF/resources")
        String outputDir();
    }

    interface CiSettings {
        @WithDefault("true")
        boolean enabled();
        @WithDefault("15m")
        String timeout();
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
        boolean enabled();
        Optional<String> projectKey();
    }
}
```

7. End-to-End Full-Stack Autonomous Lifecycle

```
[dev-agent feature "<spec>"]
            |
            ▼
 1. Phase 1: Workspace Auto-Detection & Project Context Hydration
            |
            ▼
 2. Phase 2: Full-Stack Planning & Semantic Gatekeeper Audit
            |
            ▼
 3. Phase 3: Backend TDD Loop (Quarkus REST/Panache + targeted Maven/Gradle test)
            |
            ▼
 4. Phase 4: API Contract Sync (OpenAPI Export ⟶ TypeScript Code Generation)
            |
            ▼
 5. Phase 5: Frontend TDD Loop (Components, Storybook, Signals + Vitest/Jest/tsc)
            |
            ▼
 6. Phase 6: Local Integration Verification & Visual Review Gate (--gui / :9999)
            |
            ▼
 7. Phase 7: Remote CI/CD & Automated Remediation Loop (GitHub Draft PR ⟶ Ready)
```

1. **Phase 1 (Workspace Discovery)**: Auto-detects project root, build tool, frontend directory, and initializes state in H2.
2. **Phase 2 (Planning & Gate)**: The Planner constructs a DAG spanning entities, APIs, and UI components. The Gatekeeper validates the plan against semantically retrieved ADRs.
3. **Phase 3 (Backend TDD Loop)**: Synthesizes failing Quarkus tests first, applies code modifications via `patchFile`, and verifies via sanitized build logs using the isolated build target (`target-agent/`).
4. **Phase 4 (Contract Sync)**: Automatically compiles backend models to OpenAPI and runs client codegen, ensuring TypeScript models match backend DTOs.
5. **Phase 5 (Frontend TDD Loop)**: Develops UI components, models, and Storybook stories. Verifies functionality via `FrontendExecutionTools` (`tsc --noEmit`, test runners).
6. **Phase 6 (Review Gate)**: Developer reviews unified changes in the terminal (JAnsi) or opens the embedded localhost dashboard (`--gui` on `http://localhost:9999`) for side-by-side diff previews and Storybook component inspection.
7. **Phase 7 (Remote Automation)**: Pushes to a feature branch, creates a GitHub Draft PR, monitors GitHub Actions CI, fixes SonarCloud issues if configured, and transitions the PR

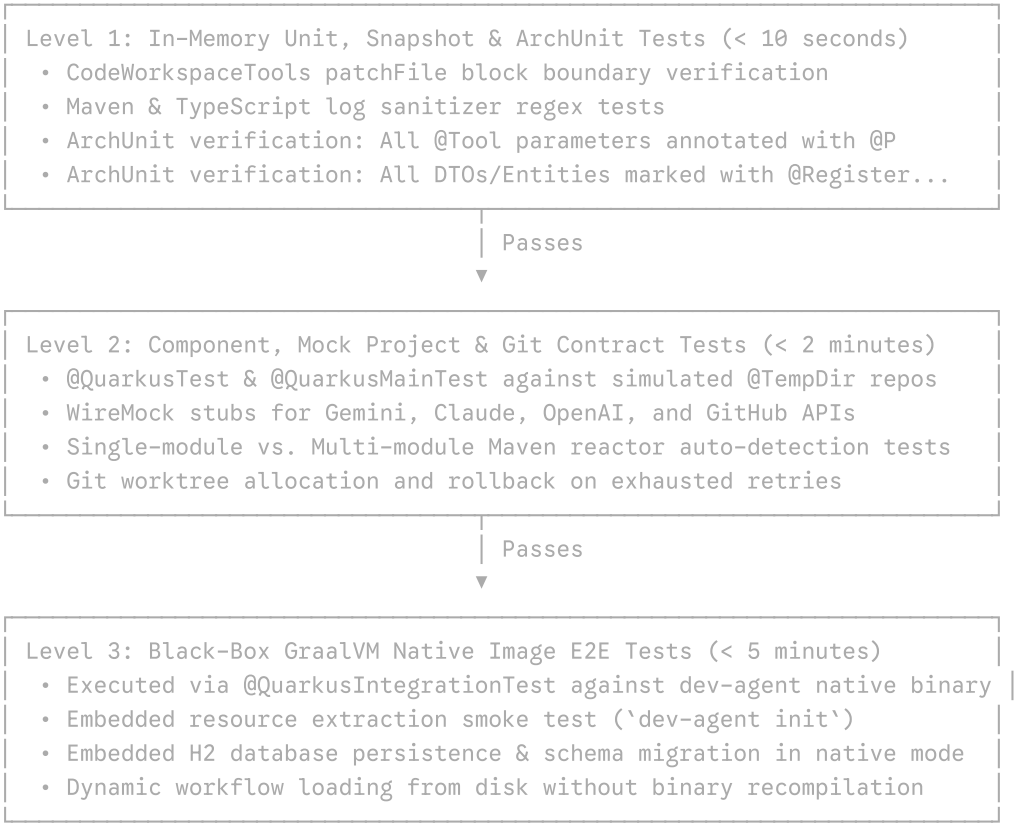
Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

8. Testing & Verification Governance

The generalized agent enforces a three-tier hermetic test strategy:



Mandatory ArchUnit Governance Rule

To guarantee GraalVM native binary reliability and prevent reflection runtime crashes:

Java

```java
@AnalyzeClasses(packages = "org.quarkus.agent")
public class AgentArchitectureTest {

    @ArchTest
    public static final ArchRule toolsMustBeProperlyAnnotated =
        methods()
            .that().areAnnotatedWith(Tool.class)
            .should().beDeclaredInClassesThat().areAnnotatedWith(ApplicationScoped.cl

    @ArchTest
    public static final ArchRule recordsAndEntitiesMustRegisterReflection =
        classes()
            .that().resideInAnyPackage("..model..", "..entity..", "..remote..")
            .and().areNotInterfaces()
```

GeminiCanvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

9. Comparison: OmniDepot Embedded vs. Generalized Architecture

Dimension	OmniDepot Embedded Agent (`tools/dev-agent`)	Generalized Dev-Agent (`quarkus-dev-agent`)
Repository Layout	Embedded submodule inside monorepo under `-Pdev-tools`	Standalone repository; independent SemVer release lifecycle
Target Scope	OmniDepot backend services & 28+ specific ADRs	Any Quarkus project (single/multi-module) + Frontend SPAs
Front-End Support	Secondary / Manual runbooks	First-class `FrontendExecutionTool` (npm/pnpm, TS, Storybook, OpenAPI)
AI System Prompts	Hardcoded "OmniDepot Lead Engineer" persona & rules	Universal Quarkus Architect persona; project rules injecte dynamically
Knowledge Ingestion	Fixed path: `docs/tech/architecture/adrs.md`	Dynamic discovery (`docs/adr/` , `.agents/rules/`) via loca semantic RAG
State Persistence	Embedded H2 fixed at `~/.omnidepot/dev-agent`	Multi-tenant H2 at `~/.config/dev-agent/data/` , namespaced by project
Build Slicing	Fixed Maven multi-module flags (`-pl` , `-am`)	Auto-detects Maven wrapper vs Gradle, single vs multi-module layout
Remote Integration	Hardcoded OmniDepot GitHub repo & Sonar key	Inferred from Git remote or configured in `.agents/dev-agent.yam`

Add the updated implementation plan

Master Implementation Plan: Generalized Full-Stack Dev-Agent (`quarkus-dev-agent`)

This plan adapts the evolutionary **Walking Skeleton** methodology to the updated product vision: a **standalone developer infrastructure CLI** hosted in its own repository, providing native **full-stack support** (Quarkus backend + TypeScript/SPA frontend), dynamic workspace path auto-detection, multi-tenant embedded state, and automated remote review loops.

Gemini Canvas ausprobieren

```
CYCLE 1: Standalone Full-Stack Walking Skeleton (Days 1–5)
```

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
CYCLE 2: Multi-Tenant State Engine & Memory Hardening (Week 2)
 • Embedded H2 SSOT (~/.config/dev-agent/data/) namespaced by `project_id`
 • Markdown Projection: Project-local `./.agents/runs/<task-id>.md`
 • 3-Tier Hierarchical Memory (Pinned Invariants + Pinned Task + Sliding Buffer)
 • Deterministic Secret Redaction Filter & Crash Recovery (`dev-agent resume`)

CYCLE 3: Dynamic ADR Semantic RAG & Decision Caching (Week 3)
 • Dynamic ADR discovery (`docs/adr/`, `.agents/rules/`, etc.)
 • In-process ONNX EmbeddingStore (`all-minilm-l6-v2`); Top 3-5 ADR retrieval
 • `quarkus-cache` memoization for Temp=0 Gatekeeper evaluations
 • Normalized Loop Detection & Oscillatory Circuit Breaker

CYCLE 4: Non-Blocking Workspaces, Build Slicing & Detached Workers (Week 4)
 • Git Worktree Isolation (`.git/agent-worktrees/<task-id>/`)
 • Build Output Slicing (`target-agent/`, `build-agent/`)
 • Detached Vert.x Background Worker (`--detach`) + Desktop OS Notifications
 • CLI Process Supervision (`dev-agent status`, `logs --follow`, `abort`)

CYCLE 5: Contract-Driven Full-Stack Sync & Remote Quality Gates (Week 5)
 • Contract Tool: Quarkus SmallRye OpenAPI ⟶ TypeScript Client Code Generation
 • Configurable GitHub REST & GraphQL Clients (Draft PR, Check-Runs, Promotion)
 • Configurable SonarCloud Quality Gate Poller & Auto-Remediation Pipeline
 • CodeRabbit 3-Way Triage State Machine & Human Override Listener
 • Test Tier Boundary Guardrail (Blocking slow `@Testcontainers` in inner loops)

CYCLE 6: GraalVM Native Hardening, Visual Review UI & Release (Week 6)
 • Sub-50ms GraalVM Native Image (`~/.local/bin/dev-agent`) + ArchUnit Rules
 • Dynamic Model Routing (`ModelRoutingService` with offline Ollama failover)
 • Financial Dollar & Token Hard Ceilings in H2
 • Embedded Vert.x Local Diff Dashboard & Storybook Preview (`--gui` on `:9999`)
 • Multi-Project E2E Dogfooding (OmniDepot as Customer Zero + Standalone SPA)
```

Cycle 1: Standalone Full-Stack Walking Skeleton (Days 1–5)

Primary Objective: Deliver a working, globally installable CLI in a new repository (`quarkus-dev-agent`) that auto-detects project structure, executes an end-to-end full-stack TDD loop, and commits changes locally.

Step 1.1: Standalone Repository & Build Setup

1. Initialize a new, independent Git repository: `quarkus-dev-agent` .
2. Configure dependencies in `pom.xml` :
   - `io.quarkus:quarkus-picocli` (CLI host runtime)
   - `io.quarkus:quarkus-config-yaml` (Configuration cascade)
   - `io.quarkiverse.langchain4j:quarkus-langchain4j-google-gemini` & `quarkus-langchain4j-openai`
   - `org.fusesource.jansi:jansi:2.4.1` (Terminal colors and formatting)
   - `com.tngtech.archunit:archunit-junit5:1.3.0` (Tool and reflection

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

`rules/quarkus-base-invariants.md` (CDI scoping, build-time reflection rules)

`rules/frontend-invariants.md` (Modern reactivity, TypeScript strictness, WCAG standards)

`workflows/plan.md` , `workflows/implement-feature.md`

`dev-agent.yaml`

Step 1.2: Dynamic Workspace Context Resolver

Implement `WorkspaceContextResolver` to inspect `System.getProperty("user.dir")` dynamically at runtime:

Project Root Discovery: Walk up the directory hierarchy to find the nearest `.git` directory, root `pom.xml` , or `build.gradle` .

Build Tool Detection: Probe for `./mvnw` → `mvn` → `./gradlew` → `gradle` .

Reactor Layout Detection: Parse root build descriptors to determine if the project is single-module or multi-module. For multi-module Maven setups, derive the target module relative path and append `-pl <module> -am` dynamically; for single-module projects, omit reactor flags.

Frontend Detection: Check for directories (`frontend/` , `src/main/webapp/` , `ui/` , `web/`) containing a `package.json` . Inspect lockfiles (`pnpm-lock.yaml` , `package-lock.json` , `yarn.lock` , `bun.lockb`) to select the correct package manager.

Step 1.3: Core File & Sandbox Tooling

1. `CodeWorkspaceTools` (`@ApplicationScoped` , `@Tool`):

   `readFile(path, startLine, endLine)` : Line-ranged reading with boundary clamping.

   `patchFile(path, searchBlock, replaceBlock)` : Precision multi-line search-and-replace.

   `createFile(path, content)` : Creates new files along with intermediate parent directories (mandatory for generating new test classes in TDD).

   `findFiles(globPattern)` : Recursively locates workspace files.

2. `WorkspaceJailValidator` :

   Resolves canonical paths for all file operations.

   Blocks path traversal (`../`), protects `.git/` , and forbids writes to the CLI binary or user home configuration.

Step 1.4: Dual Log Sanitizer Engine (`LogSanitizerService`)

Build token-compacting sanitizers that strip terminal escape codes and download noise:

Backend Sanitizer: Strips Maven/Gradle plugin download spam; extracts only `[ERROR]` compilation lines and JUnit/Surefire failure assertions.

Gemini Canvas ausprobieren

Frontend Sanitizer: Cleans Vite/Webpack/tsc noise; extracts TypeScript compilation

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Step 1.5: Build & Frontend Execution Tools

1. `BuildExecutionTools` : Executes targeted backend tests via the detected build tool (e.g., `./mvnw test -Dtest=...` or `./gradlew test --tests ...`) using `-Dproject.build.directory=target-agent` .
2. `FrontendExecutionTools` : Executes targeted frontend tests (e.g., `npx jest src/.../spec.ts`), lint checks, and TypeScript verification (`tsc --noEmit`).

Step 1.6: Transactional Git Safety Net

Implement `GitCliService` :

Invokes `git stash create` or checks out an ephemeral branch before modifying files.

If test self-healing loops fail after the retry limit, executes `git reset --hard` / `git checkout -- .` , completely cleaning dirty workspace state before exiting with code 1.

Step 1.7: Cognitive Triad Wiring (LangChain4j)

Declare the 3 AI service roles with dynamic prompt variables:

`FullStackPlannerAiService` (Gemini 1.5 Pro): Formulates a phased implementation DAG covering backend and frontend layers.

`UniversalArchitectGatekeeperAiService` (Gemini Flash or local Ollama): Audits plans against active project rules; outputs strict `GateVerdict` JSON.

`FullStackCodeSynthesizerAiService` (Claude 3.5 Sonnet or local Ollama): Executes targeted TDD patches.

Cycle 1 Definition of Done (DoD)

[ ] `quarkus-dev-agent` builds independently as a standalone Quarkus application.

[ ] `dev-agent init` extracts rules, workflows, and templates into an empty `@TempDir` .

[ ] Invoking `dev-agent feature "Add Ping Endpoint"` inside Customer Zero (OmniDepot) auto-detects Maven + multi-module layout, creates a test file, patches production code, passes `./mvnw test -pl repo-app -Dtest=...` , and creates a local Git commit.

Cycle 2: Multi-Tenant State Engine & Memory Hardening (Week 2)

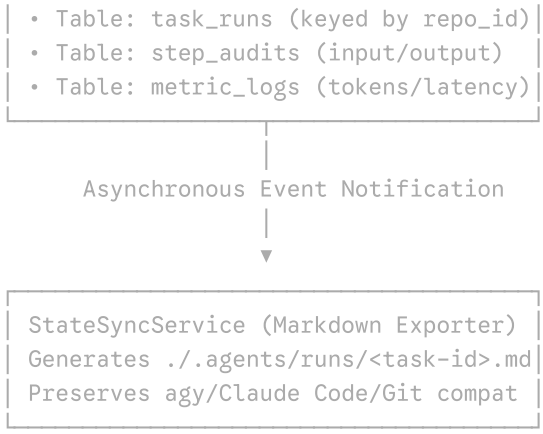
Primary Objective: Replace fragile plaintext Markdown state parsing with an ACID-compliant embedded H2 database as the Single Source of Truth (SSOT), while preserving 3-way compatibility by continuously exporting `./.agents/runs/<task-id>.md` projections.

```
State Transition / Tool Execution
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps



Step 2.1: Embedded H2 Persistence & Multi-Tenancy

1. Add `quarkus-jdbc-h2` and `quarkus-hibernate-orm-panache`.
2. Configure database location globally in `application.yaml`: `jdbc:h2:file:~/.config/dev-agent/data/agent;AUTO_SERVER=TRUE;DB_CLOSE_DELAY=-1`.
3. Design entities with project multi-tenancy:

   `TaskRunEntity`: `projectId` (hash of Git remote or absolute project root), `taskId`, `status` (`PLANNING`, `PATCHING`, `VERIFYING`, `CI_POLLING`), `branchName`, `currentPhase`, `pid`, `updatedAt`.

   `StepAuditEntity`: `taskRunId`, `toolName`, `sanitizedInput`, `sanitizedOutput`, `durationMs`, `exitCode`.

   `MetricLogEntity`: `taskRunId`, `phase`, `promptTokens`, `completionTokens`, `modelName`, `costUsd`.

Step 2.2: Markdown Projection Engine (`StateSyncService`)

After every atomic H2 database write, `StateSyncService` renders the task's current state to the active project's `./.agents/runs/<task-id>.md` using a structured Qute template.

Guarantees that external tools (`agy`, Claude Code, human reviewers) can inspect live progress directly in the target repository without needing access to the binary H2 database.

Step 2.3: 3-Tier Hierarchical Memory Model

Replace naive sliding-window chat memories with a structured `HierarchicalMemoryStore`:

Tier 1 (Pinned System Invariants): Active repository rules and governance instructions (never evicted).

Tier 2 (Pinned Task Spec & Plan): The original user story, acceptance criteria, and approved implementation DAG (never evicted).

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Step 2.4: Deterministic Secret Redaction Filter

Implement `SecretRedactionFilter` to scrub sensitive strings prior to writing to the H2 database or exporting Markdown runbooks:

- Scrubs GitHub PATs (`ghp_...`), OpenAI/Anthropic/Gemini API keys (`AIzaSy...` , `sk-ant-...`), AWS credentials, and Bearer authorization headers.

Step 2.5: Resilient Crash Recovery (`dev-agent resume`)

Implement `CrashRecoveryDetector` :

- Scans H2 for incomplete tasks (`status NOT IN ('COMPLETED', 'FAILED')`) for the active `projectId` .
- Checks whether previous processes are still running using `ProcessHandle.of(pid).isPresent()` .

Implement `ResumeCommand` :

- Hydrates `HierarchicalMemoryStore` from H2 state and resumes the workflow at the interrupted phase without re-running completed steps.

Cycle 2 Definition of Done (DoD)

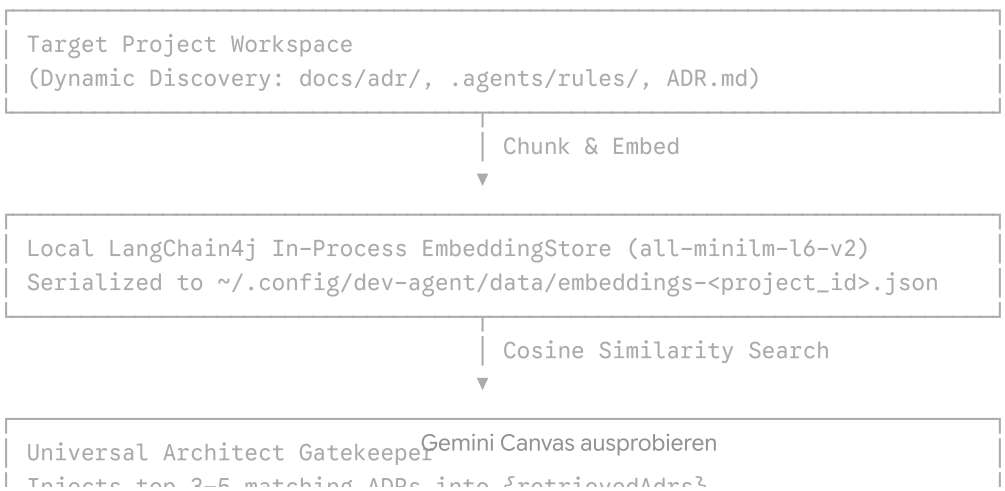
[ ] H2 database commits state transactions with unique `project_id` namespaces.

[ ] Markdown journals in `./.agents/runs/<task-id>.md` update in sync with database transitions.

[ ] Terminating a running task via `kill -9` and executing `dev-agent resume` continues from the exact failure point without re-drafting the plan.

Cycle 3: Dynamic ADR Semantic RAG & Decision Caching (Week 3)

Primary Objective: Stop injecting all ADRs into prompts. Implement dynamic discovery and local semantic search (RAG) to inject only the 3–5 relevant ADRs, while caching deterministic gate evaluations to save tokens.



Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Step 3.1: ADR & Invariant Discovery Engine

Implement `AdrDiscoveryService`:

Dynamically probes standard documentation paths in the target repository: `docs/adr/`, `doc/architecture/decisions/`, `docs/tech/architecture/adrs.md`, `.agents/rules/*.md`, and root `ADR.md`.

Parses document headings (`# ADR-(\\d+): (.*)` or Markdown frontmatter) into distinct `TextSegment` objects, preserving decision context and invariant rules as atomic units.

Step 3.2: Local In-Process Embedding Store

1. Add `dev.langchain4j:langchain4j-embeddings-all-minilm-l6-v2`.
2. Configure an in-memory or Lucene-backed `EmbeddingStore` that persists indexed vectors to `~/.config/dev-agent/data/embeddings-<project_id>.json`.
3. Compute a SHA-256 hash of all discovered documentation files on boot; regenerate embeddings only when documentation changes are detected (<800 ms index refresh).

Step 3.3: Dynamic Semantic Rule Retriever

Wire `SemanticRuleRetriever` into `FullStackPlannerAiService` and `UniversalArchitectGatekeeperAiService`.

Queries the vector store using the task description and touched module paths (e.g., matching "REST validation" retrieves bean validation and HTTP error response ADRs only).

Injects the top 3–5 matches into `{retrievedAdrs}`, cutting prompt overhead by 70–80%.

Step 3.4: Deterministic Decision Caching (`quarkus-cache`)

1. Add `io.quarkus:quarkus-cache` (Caffeine).
2. Annotate `UniversalArchitectGatekeeperAiService.evaluatePlan()` with `@CacheResult(cacheName = "architect-verdicts")`.
3. Derive the cache key via a composite hash: `SHA-256(diffContent + retrievedAdrsHash)`. Repeated checks at `temperature: 0.0` return cached verdicts in 0 ms without consuming LLM tokens.

Step 3.5: Normalized Loop Detection Circuit Breaker

Implement `OscillatoryLoopBreaker`:

Normalizes compiler and linter error traces by stripping absolute paths and line numbers.

Tracks error signatures and patch hunks in a sliding ring buffer.

If identical failure hashes recur across 2 consecutive self-healing iterations, trips the circuit breaker, rolls back workspace modifications via `GitCliService`, and exits cleanly.

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

[ ] Gatekeeper prompt tokens drop by at least 60% compared to monolithic rule injection.

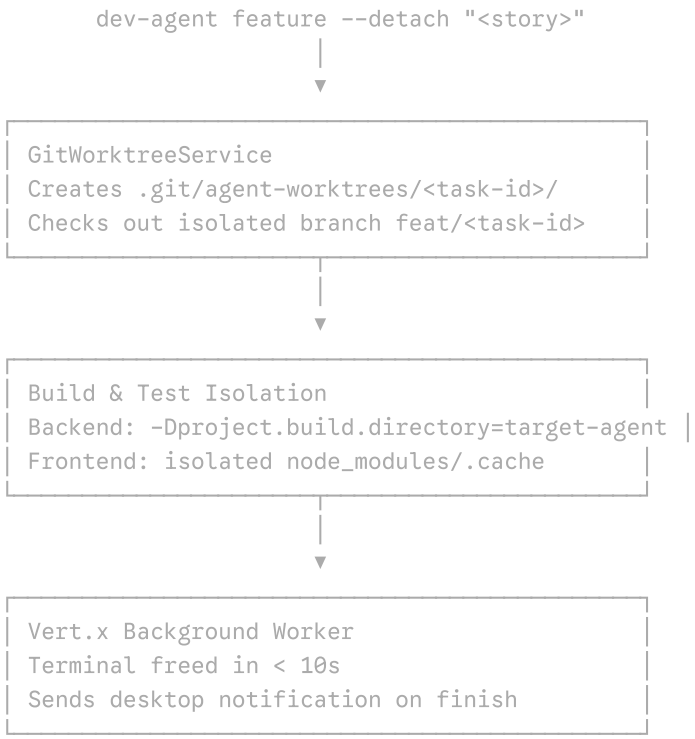
[ ] Architect Gatekeeper correctly catches domain violations using only semantically retrieved ADRs.

[ ] Re-evaluating an identical code patch returns a cached verdict in 0 ms.

[ ] An intentional cyclical compilation failure triggers the loop breaker and rolls back changes within 3 turns.

Cycle 4: Non-Blocking Workspaces, Build Slicing & Detached Workers (Week 4)

Primary Objective: Prevent background tasks from locking the developer's terminal or colliding with their IDE by isolating execution in dedicated Git worktrees and segregated build output directories.



Step 4.1: Git Worktree Lifecycle Engine

Implement `GitWorktreeService`:

`allocateWorktree(taskId, branchName)`: Executes `git worktree add .git/agent-worktrees/<task-id> -b feature/<task-id>`.

All file reading, patching, and test executions execute inside this isolated path.

The developer's primary working tree (`user.dir`) remains untouched and free from `.git/index.lock` collisions while work continues in their IDE.

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Guarantees that the agent's background compilation does not collide with active `mvn quarkus:dev` processes or IDE background compilation.

Step 4.3: Detached Process Daemonization (Vert.x Worker)

1. Support the `--detach` flag on `feature` commands.
2. When passed, Phase 1 (Planning) runs interactively; upon approval, the orchestrator delegates execution to a background worker managed via Eclipse Vert.x and detaches the CLI.
3. The main CLI returns within seconds, outputting the task ID and instructions for monitoring.

Step 4.4: Cross-Platform Desktop Notifications

Implement `DesktopNotificationService`:

Emits system notifications when background jobs complete or encounter a blocking failure:

Linux: `notify-send`

macOS: `osascript -e 'display notification ...'`

Windows/WSL: PowerShell `New-BurntToastNotification` or native toast bridges

Step 4.5: Supervision CLI Commands

`dev-agent status`: Queries H2 and displays an ANSI table of active/completed background tasks across all registered repositories.

`dev-agent logs --follow <task-id>`: Tails the active worktree log file (`.git/agent-worktrees/<task-id>/agent.log`) or streams records from `StepAuditEntity`.

`dev-agent abort <task-id>`: Terminates the background process, releases the Git worktree, and updates task status to `ABORTED` in H2.

Cycle 4 Definition of Done (DoD)

[ ] Executing `dev-agent feature --detach ...` returns the host terminal prompt in under 10 seconds.

[ ] The developer can edit files on their current Git branch in an IDE while the agent compiles and tests on its feature branch in parallel.

[ ] An OS desktop notification arrives upon task completion.

[ ] Worktrees are cleanly removed upon task completion or via `dev-agent abort`.

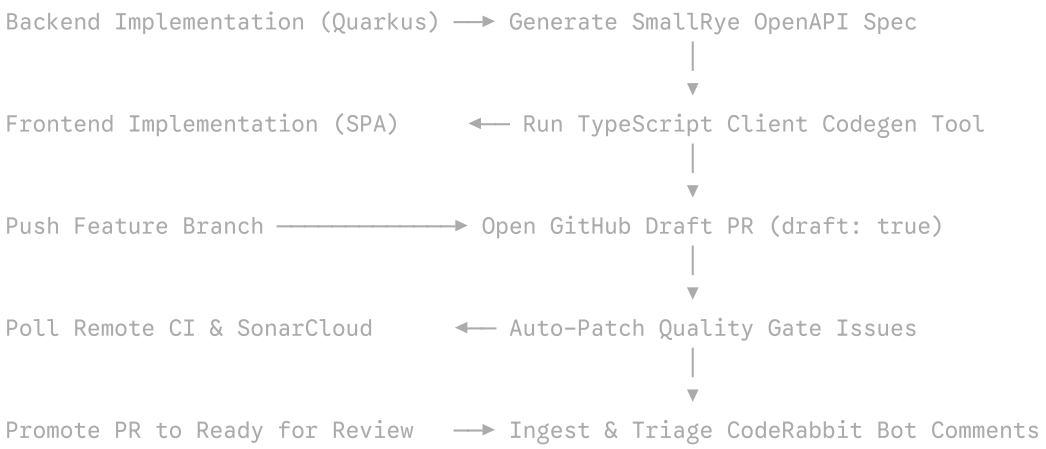
Cycle 5: Contract-Driven Full-Stack Sync & Remote Quality Gates (Week 5)

Primary Objective: Unify frontend and backend generation via automated contract synchronization, push Draft PRs, and autonomously resolve remote CI failures, SonarCloud issues, and CodeRabbit review threads.

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps



Step 5.1: Contract-Driven Code Generation Tool

Add `generateTypeScriptClient()` to `FrontendExecutionTools`:

1. Triggers Quarkus to emit the latest OpenAPI schema (via `target-agent/openapi.yaml`).
2. Runs the configured TypeScript generator (e.g., `openapi-generator-cli` or `orval`) targeting the project's frontend directory.
3. Updates frontend API models and HTTP services automatically prior to executing the frontend TDD cycle.

Step 5.2: Declarative Remote REST & GraphQL Clients

Implement declarative HTTP clients using `quarkus-rest-client-reactive-jackson`:

`GitHubApiClient`:

Creates feature branch pull requests in **Draft** state (`draft: true`).

Polls commit check-runs (`/repos/{owner}/{repo}/commits/{sha}/check-runs`) with exponential backoff.

Executes GraphQL mutations to promote PRs (`markPullRequestReadyForReview`) and resolve conversation threads.

`SonarCloudClient`:

Queries Quality Gate status (`/api/qualitygates/project_status`).

Retrieves line-level issues (`/api/issues/search`) with rule keys and file coordinates.

Step 5.3: Automated Remediation Engine

CI Remediation Loop: If remote CI checks fail, downloads and sanitizes the check-run logs, submits the failure to the Coder model, applies targeted patches, pushes a fix commit, and restarts the check poller.

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

PR Promotion: Once CI is green and SonarCloud reports `OK`, calls the GraphQL mutation to promote the PR to "Ready for Review".

Step 5.4: CodeRabbit Triage State Machine

Implement `CodeRabbitTriageService`:

Fetches review comments posted by `coderabbitai[bot]`.

Classifies each finding into one of three action states:

1. `ACCEPT_AND_FIX`: Applies the suggestion, runs tests, commits, and resolves the thread.
2. `REBUT_WITH_ADR`: Posts an architectural rebuttal citing the project's specific ADR (e.g., explaining why a pattern was chosen for performance or scoping).
3. `DISMISS`: Rejects hallucinations or stylistic nitpicks with clear rationale.

Bot Ping-Pong Breaker: Caps automated review discussions at 2 turns to prevent endless bot-to-bot arguments.

Human Override Listener: Re-checks threads for developer comments; if a developer requests a dismissed change, flips its status back to mandatory.

Step 5.5: Test Tier Slicing Guardrails

Enforce inner-loop test execution boundaries: Code synthesis tools reject test runs that boot slow integration test fixtures (e.g., `@Testcontainers`, full application boots) during the rapid TDD loop, restricting inner micro-loops strictly to fast unit and contract tests (<3 seconds).

Cycle 5 Definition of Done (DoD)

[ ] OpenAPI changes in Quarkus automatically generate typed TypeScript clients in the frontend module.

[ ] The agent pushes a feature branch, creates a Draft PR, and polls remote CI checks.

[ ] Simulated SonarCloud findings trigger automated local patches and fix commits.

[ ] The PR transitions automatically from Draft to "Ready for Review" via GraphQL.

[ ] CodeRabbit comments receive automated patches or technical ADR-backed responses.

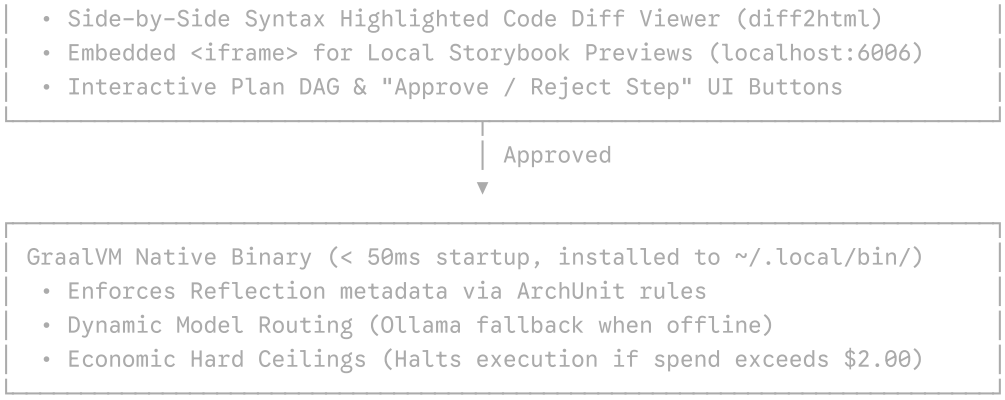
Cycle 6: GraalVM Native Hardening, Visual Review UI & Release (Week 6)

Primary Objective: Package the CLI as a production-grade GraalVM native binary (<50 ms boot), add offline local LLM failover, implement financial budget ceilings, and provide an optional local browser UI for reviewing code diffs and Storybook components.

Gemini Canvas ausprobieren
dev-agent feature --gui

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps



Step 6.1: GraalVM Native Image Optimization & ArchUnit Governance

1. Configure `quarkus.native.resources.includes=harness-templates/**` so default templates remain accessible in compiled native mode.
2. Add ArchUnit tests enforcing GraalVM native compatibility rules:

   Verify all `@Tool` methods reside in `@ApplicationScoped` classes and all parameters carry `@P` annotations.

   Verify all entities, DTOs, and gatekeeper records carry `@RegisterForReflection`.
3. Compile native executable: `mvn clean package -Dnative`.

Step 6.2: Dynamic Model Routing & Local Ollama Failover

Implement `ModelRoutingService`:

Reads the active model provider from the configuration cascade.

If `--offline` is specified or remote API keys are missing, gracefully routes AI services to a local Ollama instance (`http://localhost:11434` , model: `qwen2.5-coder:7b`).

Step 6.3: Financial Hard Ceilings & Token Accumulators

Track per-task and lifetime token consumption and cost estimates in H2.

If a task exceeds the configured ceiling (e.g., $2.00 or 250,000 tokens), trips the economic circuit breaker, safely rolls back uncommitted changes, and exits with code 1.

Step 6.4: Embedded Local Web Review Engine (`--gui`)

1. Support the `--gui` flag on `feature` and `plan` commands.
2. Starts a lightweight embedded Vert.x HTTP endpoint on `http://localhost:9999`:

   Renders side-by-side syntax-highlighted code diffs via `diff2html`.

   Embeds an `<iframe>` pointing to the local Storybook preview server (`http://localhost:6006`) to allow visual inspection of frontend components before approval.

Gemini Canvas ausprobieren

Displays "Approve" and "Request Changes" buttons that resume or redirect the CLI

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Step 6.5: Lifetime Telemetry & Repository Metrics

Implement `dev-agent metrics`:

Queries H2 across all registered projects and outputs analytics:

Average tokens and cost per feature.

Top 5 most frequently triggered ADR violations.

Compilation self-healing success rates and tool invocation latencies.

Step 6.6: Multi-Project Dogfooding & Packaging

1. Install the native binary to `~/.local/bin/dev-agent`.
2. Run full validation passes against:

Customer Zero (OmniDepot): Validates multi-module Maven layout, Netty/Zero-GC invariants, and H2/Liquibase patterns.

Standalone Full-Stack App: Validates single-module Quarkus backend + Angular/React frontend, OpenAPI TypeScript client generation, and Storybook testing.

Cycle 6 Definition of Done (DoD)

[ ] Native executable boots in <50 ms.

[ ] `--offline` mode runs local TDD workflows against local Ollama models.

[ ] Financial circuit breakers halt runaway loops when spend limits are reached.

[ ] Running `dev-agent feature --gui` serves an interactive browser diff dashboard on port 9999.

[ ] All Level 1, Level 2, and Level 3 Native Integration tests pass cleanly across both single-module and multi-module repositories.

Master Schedule & Progression Matrix

Cycle	Primary Milestone	Key Capabilities Added	Target SLA
Cycle 1	**Standalone Walking Skeleton**	Standalone repo, workspace/frontend auto-detection, file patcher, log sanitizer, Git rollback	**Days 1–5**
Cycle 2	**Multi-Tenant State Engine**	Embedded H2 SSOT, multi-tenancy, Markdown projection sync, 3-tier memory, crash resume	**Week 2**
Cycle 3	**Dynamic ADR Semantic RAG**	Dynamic ADR discovery, in-process embeddings, top-3 ADR retrieval, decision caching, loop breaker	**Week 3**

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Cycle	Primary Milestone	Key Capabilities Added	Target SLA
		desktop alerts, CLI supervision	
Cycle 5	**Contract Sync & Remote Gates**	OpenAPI → TypeScript codegen, Draft PR lifecycle, CI & Sonar self-healing, CodeRabbit triage	**Week 5**
Cycle 6	**Native Hardening & Release**	GraalVM native binary (<50 ms), Ollama failover, budget ceilings, local	**Week 6**

Now add the updated details for phase 1

Phase 1: Standalone Full-Stack Walking Skeleton (Detailed Specification)

1. Architectural Scope & Boundary Definition

The primary objective of **Phase 1** is to establish an operational, end-to-end full-stack software-engineering loop within a **standalone, dedicated repository** (`quarkus-dev-agent`).

The resulting CLI must be capable of running against any Quarkus project directory (`user.dir`), auto-detecting the project's backend reactor and frontend layout, formulating an implementation plan, executing a targeted red-green TDD loop across Java and TypeScript, and committing or rolling back changes cleanly.

```
PHASE 1 ARCHITECTURAL BOUNDARY
┌──────────────────────────────────────────────────────────────────────┐
│ IN SCOPE (Phase 1 Deliverables)                                      │
│  • Independent Standalone Maven Project (`quarkus-dev-agent`)        │
│  • Picocli CLI Entrypoint (`init`, `feature`) with JAnsi ANSI Terminal UX │
│  • Dynamic Workspace Resolver (Project Root, Maven/Gradle, Frontend layout) │
│  • Workspace Sandboxing (`WorkspaceJailValidator` with canonical resolution)│
│  • Core Workspace Tools (`createFile`, `readFile`, `patchFile`, `findFiles`)│
│  • Dual Diagnostic Log Sanitizer (Java/Maven/Gradle + TypeScript/npm/Vite) │
│  • Backend & Frontend Execution Tools (`BuildExecutionTools`, `Frontend...`) │
│  • Transactional Git Safety Net (Stash/Branch Snapshot + Hard Rollback)  │
│  • LangChain4j Triple-Role AI Wiring (Generic Planner, Gatekeeper, Coder) │
│  • Ephemeral In-Memory State & Summary Markdown Export (.agents/runs/<id>.md)│
│  • Level 1 & Level 2 Hermetic Test Suites (ArchUnit, Snapshots, Mock LLMs) │
├──────────────────────────────────────────────────────────────────────┤
│ EXPLICITLY DEFERRED TO LATER PHASES                                  │
│  ✕ Multi-Tenant Local H2 Database & Panache State Machine ──→ Phase 2  │
│  ✕ 3-Tier Hierarchical Memory & Secret Scrubbing Pipeline ──→ Phase 2  │
│  ✕ LangChain4j Local Semantic ADR Vector Store (RAG)      ──→ Phase 3  │
│  ✕ Deterministic Decision Caching (Semantic Hash)         ──→ Phase 3  │
│  ✕ Detached Vert.x Worker & Git Worktree Isolation        ──→ Phase 4  │
```

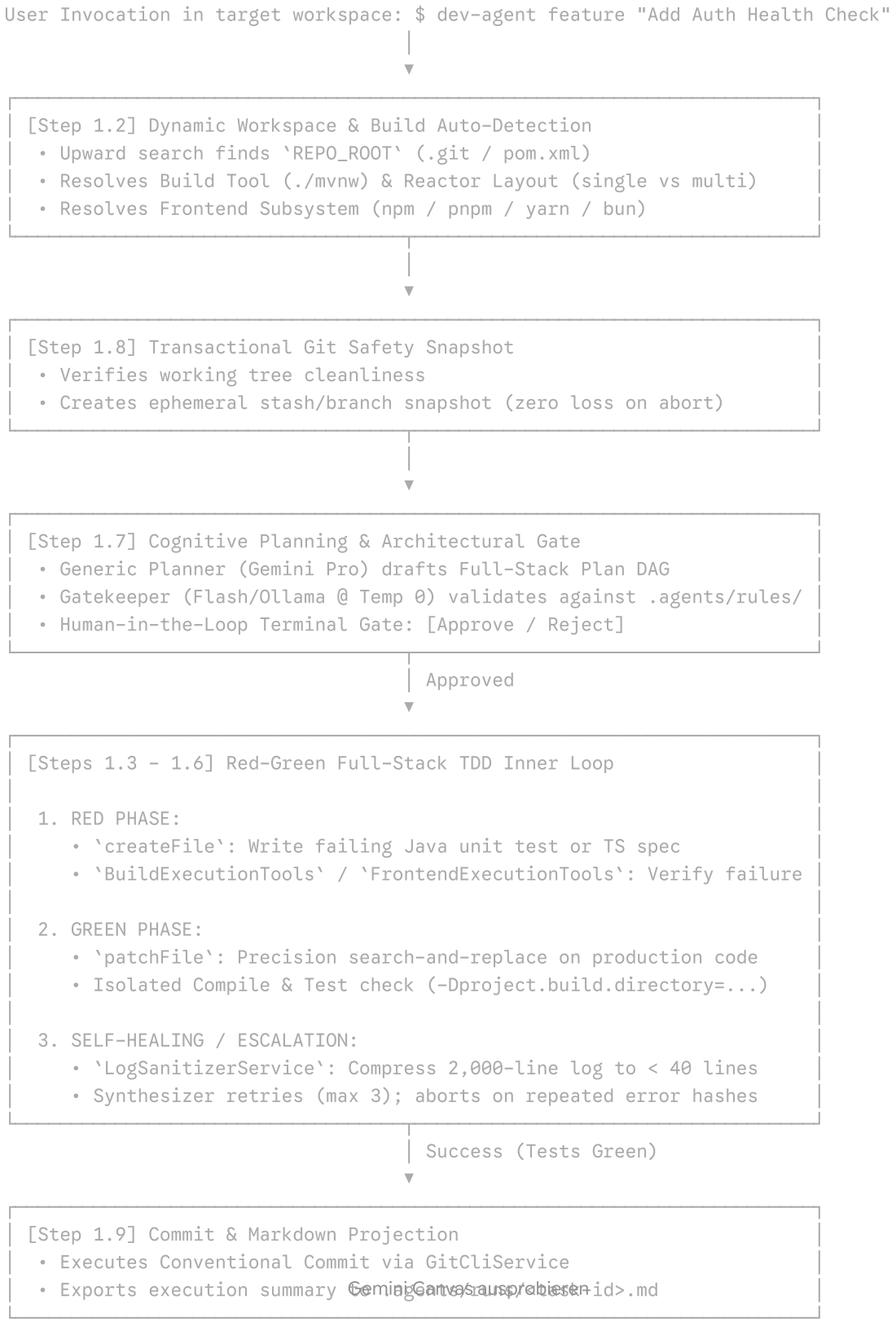
Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
| ✕ GraalVM Native Image Compilation & Web Diff UI (:9999) ──► Phase 6 |
```

2. Execution Sequence Diagram



GeminiCanvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

3. Detailed Step-by-Step Implementation Breakdown

Step 1.1: Standalone Repository Scaffolding & Packaging

1. **Initialize Standalone Project:**
   Create a dedicated Maven repository: `quarkus-dev-agent`.

```xml
<groupId>org.quarkus.agent</groupId>
<artifactId>quarkus-dev-agent</artifactId>
<version>0.1.0-SNAPSHOT</version>
<packaging>jar</packaging>
```

2. **Core Dependencies (`pom.xml`):**
   - `io.quarkus:quarkus-picocli` (CLI host framework and lifecycle)
   - `io.quarkus:quarkus-config-yaml` (Multi-tiered YAML configuration)
   - `io.quarkiverse.langchain4j:quarkus-langchain4j-google-gemini` (Planner & Gatekeeper)
   - `io.quarkiverse.langchain4j:quarkus-langchain4j-openai` (Coder model via OpenAI/OpenRouter or Ollama)
   - `org.fusesource.jansi:jansi:2.4.1` (Terminal ANSI styling, tables, and colors)
   - `com.tngtech.archunit:archunit-junit5:1.3.0` (Tool architecture and reflection governance)
3. **Embedded Scaffolding Templates:**
   Package default declarative templates in `src/main/resources/harness-templates/` for the `init` command:
   - `rules/quarkus-base-invariants.md`: Rules for CDI scoping (`@ApplicationScoped` vs `@RequestScoped`), build-time reflection guidelines, and Vert.x non-blocking loop rules.
   - `rules/frontend-invariants.md`: Rules for TypeScript strictness, Angular Signals / React hooks reactivity, accessibility (WCAG AA), and component isolation.
   - `workflows/plan.md` & `workflows/implement-feature.md`: Base agent workflows.
   - `dev-agent.yaml`: Default project configuration schema.

Step 1.2: Dynamic Workspace & Subsystem Auto-Detection

Implement `WorkspaceContextResolver` as an `@ApplicationScoped` CDI bean to replace OmniDepot's hardcoded paths:

- **Workspace Root Discovery:**
  Traverse upward starting from `Path.of(System.getProperty("user.dir"))` to locate the canonical `REPO_ROOT` (the first ancestor containing `.git`, a root `pom.xml`, or `build.gradle`).

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

1. `./mvnw` (Maven Wrapper)
2. `mvn` (System Maven)
3. `./gradlew` (Gradle Wrapper)
4. `gradle` (System Gradle)

Reactor Structure Resolver:
Inspect the root build descriptor.

- If Maven: Parse `<modules>` in `pom.xml`. If multi-module, resolve the relative path to the targeted module and construct reactor coordinates: `-pl <relative-path> -am`. If single-module, omit `-pl`.
- Target Build Isolation: Mandate `-Dproject.build.directory=target-agent` (Maven) or `-PbuildDir=build-agent` (Gradle) on all test runs to avoid locking developer IDE targets.

Frontend Subsystem Resolver:
Scan `REPO_ROOT` for frontend roots: `frontend/`, `ui/`, `web/`, or `src/main/webapp/` containing `package.json`.

- Detect the active package manager via lockfiles: `pnpm-lock.yaml` (`pnpm`) → `package-lock.json` (`npm`) → `yarn.lock` (`yarn`) → `bun.lockb` (`bun`).

Step 1.3: Workspace Sandboxing & Path Jail

Implement `WorkspaceJailValidator` to prevent arbitrary file corruption:

Canonical Boundary Validation:
Resolve all incoming file paths against `REPO_ROOT.toRealPath()`.

Java

```java
public Path validateAndResolve(String requestedPath) {
    Path resolved = repoRoot.resolve(requestedPath).normalize();
    if (!resolved.startsWith(repoRoot)) {
        throw new SecurityException("Path traversal jailbreak attempt: " + reque
    }
    return resolved;
}
```

Protected System Paths:
Explicitly reject write and delete operations targeting:

- `.git/**` (Git metadata and hooks)
- `~/.config/dev-agent/**` (Global user configurations and database files)
- The CLI binary executable itself (`~/.local/bin/dev-agent`)

Gemini Canvas ausprobieren

Step 1.4: Core Workspace Manipulation Tools

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

`readFile(path, startLine, endLine)` :
Reads line-delimited content. Enforces boundary clamping (if `endLine` exceeds file length, clamp to EOF; do not throw exceptions).

`createFile(path, content)` :
Creates a new file on disk along with intermediate missing parent directories (critical for creating new test classes in the TDD Red phase). Throws a descriptive error if the file already exists (directing the model to use `patchFile` instead).

`patchFile(path, searchBlock, replaceBlock)` :
Applies precision block replacement.

- Matches exact lines first.
- If exact match fails, applies a whitespace/indentation-normalized fallback match.
- If `searchBlock` matches multiple locations ambiguously, aborts and returns: `"Ambiguous search block: matches X times. Provide more surrounding context lines."`

`findFiles(globPattern)` :
Recursively discovers project files matching standard globs (e.g., `**/src/test/**/*Test.java` , `**/*.spec.ts`).

Step 1.5: Dual Diagnostic Log Sanitizer (`LogSanitizerService`)

Raw compiler and test output regularly exceeds 20,000 tokens per build. Implement a deterministic filtering engine that strips noise and compresses diagnostic output down to under 500 tokens:

```
Raw Process Output (Maven/Gradle/npm/tsc)
                    |
                    ▼
 1. ANSI Escapes Stripper: Remove all \u001B\[[0-9;]*[a-zA-Z] sequences.
                    |
                    ▼
 2. Download & Hook Noise Filter: Drop download progress bars, plugin resolution,
    and task execution headers.
                    |
                    ▼
 3. Backend Error Parser:
    • Match Java compiler errors: ^\[ERROR\]\s+(/.*\.java):\[(\d+),(\d+)\]\s+(.*)$
    • Match Surefire test failures: AssertionFailedError: expected <X> but was <Y>
    • Trim internal JVM stackframes (retain only application package prefixes).
                    |
                    ▼
 4. Frontend Error Parser:
    • Match TypeScript errors: (.*\.ts)\((\d+),(\d+)\):\s+error\s+(TS\d+):\s+(.*)
    • Match Vitest/Jest/Karma failures: FAIL <spec-file> and AssertionError.
    • Trim node_modules/* frames.
```

Gemini Canvas ausprobieren

Step 1.6: Build & Frontend Execution Engines

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

`runBackendCompile()` : Executes `./mvnw test-compile -DskipTests -Dproject.build.directory=target-agent`.

`runTargetedBackendTest(testClassPattern)` : Executes `./mvnw test -Dtest=<pattern> -Dproject.build.directory=target-agent` .

Both methods pipe process streams asynchronously to avoid OS pipe deadlocks and return sanitized strings via `LogSanitizerService` .

2. `FrontendExecutionTools` (`@ApplicationScoped` , `@Tool`):

   `runFrontendTypeCheck()` : Executes `npx tsc --noEmit` inside the detected frontend root.

   `runTargetedFrontendTest(specPath)` : Executes targeted test runners (e.g., `npm test -- <specPath>` or `pnpm test <specPath>`).

   Returns sanitized errors, missing exports, and assertion failures.

Step 1.7: Cognitive Triad Wiring & TDD Orchestrator

Configure the three LangChain4j AI roles with dynamic prompt context (zero hardcoded repository domains):

Java

```
// 1. Planner Service
@RegisterAiService(modelName = "planner")
public interface FullStackPlannerAiService {
    @SystemMessage("""
        You are a Principal Software Engineer specializing in Quarkus and modern Fron
        Draft a phased implementation plan (Plan DAG) containing:
        1. Red Phase: Test files to create (Java / TypeScript).
        2. Green Phase: Production files to modify using minimal targeted patches.
        Project Architecture Guidelines:
        {rulesContext}
        """)
    String draftPlan(@UserMessage String userStory, @V("rulesContext") String rulesCo
}

// 2. Architect Gatekeeper
@RegisterAiService(modelName = "architect-gate")
public interface UniversalArchitectGatekeeperAiService {
    @SystemMessage("""
        You are the Architectural Invariant Gatekeeper.
        Audit the provided plan against project rules.
        Rules:
        {rulesContext}
        Return ONLY valid JSON matching the GateVerdict record schema.
        """)
    GateVerdict evaluatePlan(@UserMessage String plan, @V("rulesContext") String rule
```

Gemini Canvas ausprobieren

```
    @RegisterForReflection
```

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
        List<String> recommendations
    ) {}
}

// 3. Code Synthesizer
@RegisterAiService(
    modelName = "coder",
    tools = { CodeWorkspaceTools.class, BuildExecutionTools.class, FrontendExecutionT
)
public interface FullStackCodeSynthesizerAiService {
    @SystemMessage("""
        You are the Implementation Engineer.
        - In the RED phase: Create failing tests first via 'createFile'.
        - In the GREEN phase: Modify production code using 'patchFile'.
        - Never overwrite entire files.
        - If tests fail, analyze the sanitized error log and apply targeted fixes.
        """)
    String executeStep(@UserMessage String stepInstructions);
}
```

Orchestration Lifecycle (`FeatureWorkflowOrchestrator.java`):

1. *Rule Ingestion*: Read all `*.md` files from `${REPO_ROOT}/.agents/rules/` into `{rulesContext}` .
2. *Plan Formulation*: Call `planner.draftPlan(story, rulesContext)` .
3. *Architect Gate*: Call `architectGate.evaluatePlan(plan, rulesContext)` . If rejected, feed recommendations back for up to 1 refinement turn.
4. *Human-in-the-Loop Approval*: Present plan in terminal via JAnsi; prompt user `[A]pprove / [R]eject` (bypassed if `--yes` flag is set).
5. *Red TDD Phase*: Coder invokes `createFile` to author unit tests. Execute test tools; assert that tests fail with an assertion error (proving the test is valid and red).
6. *Green TDD Phase*: Coder patches production classes via `patchFile` . Re-run test tools until green.
7. *Self-Healing Loop*: If tests fail:
    - Feed sanitized log back to coder.
    - Track error hashes: if identical error signatures occur across 2 consecutive iterations, trigger the oscillatory circuit breaker and halt.
    - Maximum retry limit: 3 iterations.

Step 1.8: Transactional Git Safety Net (`GitCliService`)

Implement `GitCliService` to guard the workstation:

Workspace Cleanliness Check: Prior to running modifications, run `git status --porcelain` . Warn the user if dirty files exist.

Snapshot Allocation: Run `git stash create` or checkout an ephemeral branch (`agent/scratchpad-<task-id>`) before touching files.

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Conventional Commit: On success, stage modified files and generate a commit formatted as:
`feat(<scope>): <user-story-summary>`.

Step 1.9: CLI Entrypoint, Interactive UX & Markdown Summary

Implement Picocli commands using `quarkus-picocli`:

`DevAgentRootCommand`: Defines global options (`--dir=<workspace>`, `--yes`, `--verbose`).

`InitCommand`:

Scaffolds `./.agents/` in the target project.

Extracts embedded rules, archetypes, and workflow templates (`plan.md`, `implement-feature.md`).

`FeatureCommand`: Executes the end-to-end full-stack workflow.

Terminal Ergonomics (JAnsi):

Color-coded progress badges: `[PLANNING]`, `[GATE: APPROVED]`, `[TDD: RED]`, `[TDD: GREEN]`, `[COMMIT]`.

Phase 1 Execution Summary:
Write an execution Markdown journal to `./.agents/runs/<task-id>.md` recording the prompt, files modified, test status, and commit hash upon completion.

Step 1.10: Verification & Test Suite

Enforce strict test gates before concluding Phase 1:

1. **Level 1 In-Memory Unit & Governance Tests (`src/test/`)**:

   `CodeWorkspaceToolsTest`: Verify `createFile` fails safely if target exists; verify `patchFile` detects duplicate blocks and fails safely without uncaught exceptions; verify `readFile` clamping.

   `WorkspaceJailValidatorTest`: Verify `../../etc/passwd` and `.git/config` throws `SecurityException`.

   `LogSanitizerServiceTest`: Verify raw Maven and TypeScript compiler logs are reduced by $> 85\%$ against snapshot fixtures.

   `DevAgentArchUnitTest`: Verify all `@Tool` methods belong to `@ApplicationScoped` classes with documented `@P` parameters; verify all records carry `@RegisterForReflection`.

2. **Level 2 CLI Contract Tests (`@QuarkusMainTest`)**:

   Execute `dev-agent init` against an ephemeral `@TempDir` Git repo; assert exit code 0 and template creation.

   Stub AI responses with WireMock: Simulate a red-green TDD feature loop; assert exit code 0 and verified Git commit.

   Simulate a deliberate compiler failure that exhausts retries; assert exit code 1 and

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Markdown

```
[ ] Scaffolding: `quarkus-dev-agent` builds as an independent, standalone Quarkus app
[ ] Sandboxing: `WorkspaceJailValidator` blocks traversal attacks and protects `.git/
[ ] Workspace Auto-Detection: Correctly detects `REPO_ROOT`, build tools (`mvnw`/`gra
[ ] Reactor Slicing: Single-module builds omit `-pl`; multi-module reactors construct
[ ] Tool Robustness: `createFile`, `readFile`, and `patchFile` operate safely with bo
[ ] Dual Sanitization: Compresses Maven/Gradle and TypeScript log outputs by > 85%, e
[ ] Git Safety: Aborted runs or exhausted retries execute an automatic `git reset --h
[ ] AI Cognitive Flow: Generates a plan, passes gatekeeper audit, authors failing tes
[ ] Markdown Summary: Successfully exports `./.agents/runs/<task-id>.md` upon task co
[ ] Test Gate: 100% of Level 1 (Unit, ArchUnit)
```

And now update the plan for phase 2

Phase 2: Multi-Tenant State Engine & Memory Hardening (Detailed Specification)

1. Architectural Scope & Boundary Definition

The primary objective of **Phase 2** is to transition the standalone Dev-Agent from ephemeral in-memory state into an **ACID-compliant embedded relational database** acting as the **Single Source of Truth (SSOT)**.

In a generalized, multi-project environment, the agent cannot bind its state to a single repository or hardcoded path (such as `~/.omnidepot/`). Instead, the database operates as a **centrally located, multi-tenant store** (`~/.config/dev-agent/data/agent.mv.db`) capable of isolating task runs across distinct Quarkus and frontend workspaces via dynamic project namespacing.

Simultaneously, Phase 2 preserves **three-way workflow compatibility** (developer, native CLI, generic tools like `agy` and Claude Code) by automatically projecting execution states to disk as human-readable Markdown journals (`./.agents/runs/<task-id>.md`) in the active project. To eliminate context amnesia during self-healing loops, the conversational context is structured into a **3-Tier Hierarchical Memory Model**, while a deterministic **Secret Redaction Pipeline** scrubs sensitive credentials prior to disk or database persistence.

PHASE 2 ARCHITECTURAL BOUNDARY

```
┌──────────────────────────────────────────────────────────────────────┐
│ IN SCOPE (Phase 2 Deliverables)                                      │
│  • Global Embedded H2 Database (~/.config/dev-agent/data/agent.mv.db)│
│  • Project Multi-Tenancy (Namespacing by Git remote or Repo Root Hash)│
│  • Panache Active-Record Entities (TaskRun, StepAudit, MetricLog)    │
│  • Event-Driven Dual-Representation Engine (H2 SSOT ──► Markdown Projection)│
│  • Dynamic Project-Local Markdown Export (./.agents/runs/<task-id>.md)│
│  • 3-Tier Hierarchical Memory Store (Pinned Rules + Task Spec + Scratchpad)│
│  • Deterministic Secret Redaction Filter (PATs, API keys, Bearer tokens)│
│  • Dead-Process Crash Recovery & Resumption Engine (dev-agent resume)│
│  • Level 1 Unit & Level 2 Contract Test Suites for State & Resumption│
└──────────────────────────────────────────────────────────────────────┘
```

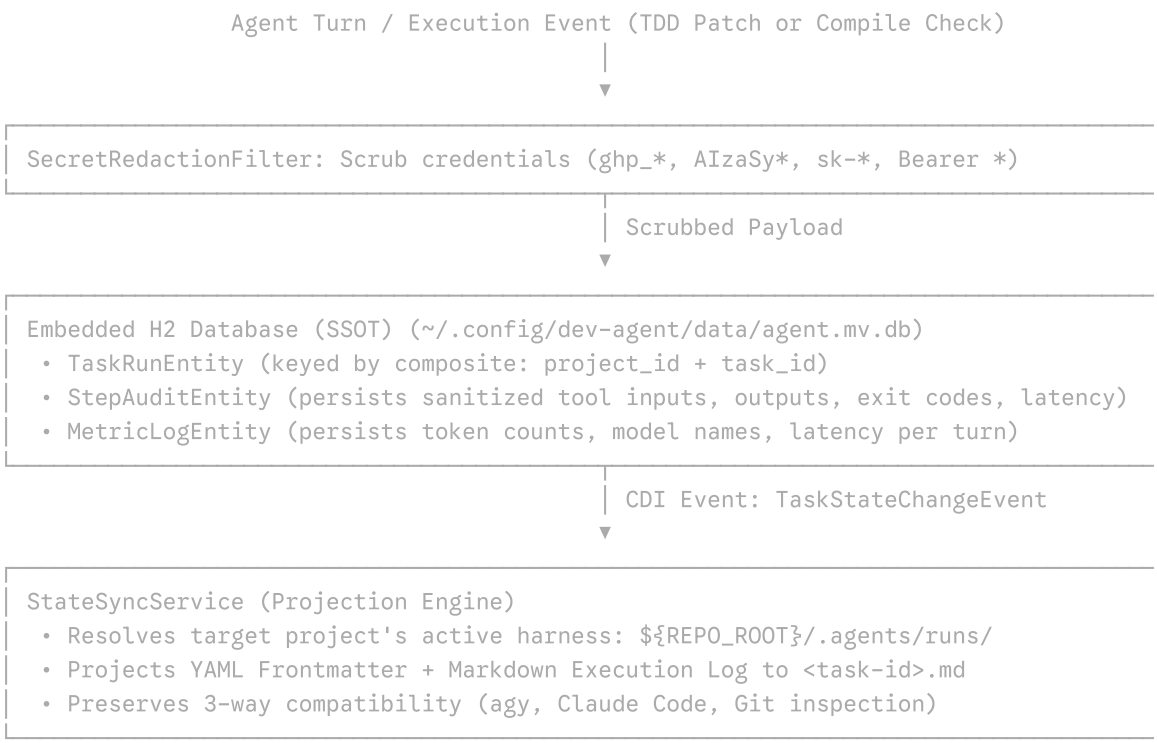
Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
| ✕ Oscillatory Loop Breaker Engine                          ⟶ Phase 3 |
| ✕ Isolated Git Worktrees & Build Slicing (target-agent)⟶ Phase 4 |
| ✕ Vert.x Detached Worker & Desktop OS Alerts               ⟶ Phase 4 |
| ✕ Contract-Driven TypeScript Codegen & Remote CI Gates ⟶ Phase 5 |
| ✕ GraalVM Native Image Compilation & Web Diff UI       ⟶ Phase 6 |
```

2. State & Event Flow Diagram



3. Detailed Step-by-Step Implementation Breakdown

Step 2.1: Relational Schema Design & Global Multi-Tenancy

1. **Quarkus Extensions & Global Datasource:**
   Add `io.quarkus:quarkus-jdbc-h2` and `io.quarkus:quarkus-hibernate-orm-panache`.
   Configure the global datasource in `application.yaml` with server mode enabled to support concurrent read CLI access while tasks execute:

YAML

```yaml
quarkus:
  datasource:
    db-kind: h2
    jdbc:
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
database:
  generation: update
log:
  sql: false
```

2. **Project Coordinate Resolver (`ProjectIdResolver`):**
   To isolate records when running tasks across multiple Quarkus and frontend workspaces, implement `ProjectIdResolver` :

   Checks `git config --get remote.origin.url` .

   If no remote is configured (e.g., local prototype or offline test repo), falls back to `SHA-256(REPO_ROOT.toRealPath())` .

   Ensures every database query is scoped strictly to the calling repository.

3. **Panache Active-Record Entities:**

   `TaskRunEntity` : Represents the lifecycle of a user story or feature run.

Java

```java
@Entity
@Table(name = "task_runs", indexes = {
    @Index(name = "idx_project_task", columnList = "project_id, task_id", ui
    @Index(name = "idx_status", columnList = "status")
})
public class TaskRunEntity extends PanacheEntityBase {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    public Long id;

    @Column(name = "project_id", nullable = false)
    public String projectId;

    @Column(name = "task_id", nullable = false)
    public String taskId;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    public TaskStatus status; // PLANNING, RED_TEST, GREEN_PATCH, VERIFYING

    @Column(name = "current_phase", nullable = false)
    public String currentPhase;

    @Column(name = "branch_name")
    public String branchName;

    @Column(name = "base_commit")
    public String baseCommit;

    @Column(name = "checkpoint_stash")
    public String checkpointStash;

    Gemini Canvas ausprobieren
    @Column(name = "approved_plan_json", columnDefinition = "TEXT")
```

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
    public Long pid;

    @Column(name = "retry_count")
    public int retryCount;

    @Column(name = "created_at", nullable = false)
    public Instant createdAt;

    @Column(name = "updated_at", nullable = false)
    public Instant updatedAt;

    @OneToMany(mappedBy = "taskRun", cascade = CascadeType.ALL, orphanRemova
    public List<StepAuditEntity> steps = new ArrayList<>();

    public static TaskRunEntity findByProjectAndTask(String projectId, Stri
        return find("projectId = ?1 and taskId = ?2", projectId, taskId).fi
    }
}
```

`StepAuditEntity` : Complete audit log of every tool execution (file read, patch, Maven test, TypeScript check).

Java

```
@Entity
@Table(name = "step_audits")
public class StepAuditEntity extends PanacheEntityBase {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    public Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "task_run_id", nullable = false)
    public TaskRunEntity taskRun;

    @Column(name = "tool_name", nullable = false)
    public String toolName;

    @Column(name = "sanitized_input", columnDefinition = "TEXT")
    public String sanitizedInput;

    @Column(name = "output_summary", columnDefinition = "TEXT")
    public String outputSummary;

    @Column(name = "duration_ms")
    public long durationMs;

    @Column(name = "exit_code")
    public int exitCode;

    @Column(name = "created_at", nullable = false)
    public Instant createdAt;
}
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

`MetricLogEntity` : Granular telemetry on token expenditure, financial estimates, and latency.

Java

```java
@Entity
@Table(name = "metric_logs")
public class MetricLogEntity extends PanacheEntityBase {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    public Long id;

    @Column(name = "task_run_id", nullable = false)
    public Long taskRunId;

    @Column(name = "phase_name", nullable = false)
    public String phaseName;

    @Column(name = "model_name", nullable = false)
    public String modelName;

    @Column(name = "prompt_tokens")
    public int promptTokens;

    @Column(name = "completion_tokens")
    public int completionTokens;

    @Column(name = "cost_usd")
    public BigDecimal costUsd;

    @Column(name = "created_at", nullable = false)
    public Instant createdAt;
}
```

Step 2.2: Dual-Representation Markdown Projection Engine (`StateSyncService`)

To ensure developers and third-party tools (`agy` , Claude Code) can track progress directly inside the repository without touching H2, the agent projects updates to disk.

1. **Decoupled State Change Events:**
   Define `TaskStateChangeEvent` to decouple domain execution from disk serialization:

Java

```java
public record TaskStateChangeEvent(
    String projectId,
    String taskId,
    TaskStatus newStatus,
    String phaseName,
    String stepSummary,
    Instant timestamp
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

2. The Synchronization Service (`StateSyncService.java`):
Listens for CDI events, commits H2 state, and projects a YAML-frontmatter Markdown journal into `${REPO_ROOT}/.agents/runs/<task-id>.md` :

Java

```java
@ApplicationScoped
public class StateSyncService {

    @Inject
    WorkspaceContextResolver workspaceResolver;

    @Inject
    SecretRedactionFilter secretFilter;

    @Transactional
    public void onStateTransition(@Observes TaskStateChangeEvent event) {
        TaskRunEntity entity = TaskRunEntity.findByProjectAndTask(event.projectI
        if (entity == null) {
            return;
        }

        entity.status = event.newStatus();
        entity.currentPhase = event.phaseName();
        entity.updatedAt = event.timestamp();
        entity.persist();

        projectToMarkdown(entity);
    }

    private void projectToMarkdown(TaskRunEntity entity) {
        Path targetRunsDir = workspaceResolver.getHarnessRoot().resolve("runs");

        String markdown = """
            ---
            task_id: "%s"
            project_id: "%s"
            status: "%s"
            current_phase: "%s"
            branch: "%s"
            base_commit: "%s"
            checkpoint_stash: "%s"
            pid: %d
            retry_count: %d
            updated_at: "%s"
            ---

            # Task Execution Journal: %s

            ## User Story
            %s

            ## Approved Implementation Plan
            ```json
            %s
            ```
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
                """.formatted(
                    entity.taskId,
                    entity.projectId,
                    entity.status,
                    entity.currentPhase,
                    entity.branchName != null ? entity.branchName : "unassigned",
                    entity.baseCommit != null ? entity.baseCommit : "HEAD",
                    entity.checkpointStash != null ? entity.checkpointStash : "none"
                    entity.pid != null ? entity.pid : 0L,
                    entity.retryCount,
                    DateTimeFormatter.ISO_INSTANT.format(entity.updatedAt),
                    entity.taskId,
                    entity.userStory != null ? entity.userStory : "None specified",
                    entity.approvedPlanJson != null ? entity.approvedPlanJson : "{}"
                    renderStepsHistory(entity)
                );

            try {
                Files.createDirectories(targetRunsDir);
                Path journalFile = targetRunsDir.resolve(entity.taskId + ".md");
                Files.writeString(
                    journalFile,
                    secretFilter.scrub(markdown),
                    StandardOpenOption.CREATE,
                    StandardOpenOption.TRUNCATE_EXISTING,
                    StandardOpenOption.SYNC
                );
            } catch (IOException e) {
                System.err.println("WARN: Failed to project Markdown state journal:
            }
        }

        private String renderStepsHistory(TaskRunEntity entity) {
            StringBuilder sb = new StringBuilder();
            for (StepAuditEntity step : entity.steps) {
                sb.append(String.format("- [%s] **%s** (%dms): %s\n",
                    step.exitCode == 0 ? "x" : " ",
                    step.toolName,
                    step.durationMs,
                    step.outputSummary
                ));
            }
            return sb.toString();
        }
    }
```

Step 2.3: 3-Tier Hierarchical Memory Architecture

Standard LangChain4j sliding-window chat memories (e.g., `MessageWindowChatMemory(5)`) drop early turns during self-healing loops, causing the model to lose the user story and approved plan. Phase 2 implements a structured **3-Tier Hierarchical Memory Store**:

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
TIER 1: Pinned Invariants (Never Evicted)
 • Quarkus & Frontend Architecture Rules from .agents/rules/*.md
 • CDI Scoping, Non-blocking Vert.x, TypeScript strictness
──────────────────────────────────────────────────────────────
TIER 2: Pinned Task Specification & DAG Plan (Never Evicted)
 • Raw User Story & Acceptance Criteria
 • Architect Gatekeeper Approved Implementation DAG
 • Active Submodule & touched file coordinates
──────────────────────────────────────────────────────────────
TIER 3: Sliding Scratchpad Window (Bounded to Last 4 Micro-Turns)
 • Turn N-1: Applied `patchFile` search/replace hunks
 • Turn N: Sanitized compiler error or failing test assertion
 • Oldest scratchpad turns evicted automatically as new turns are added
```

1. `HierarchicalMemoryStore.java`:
   Implements `dev.langchain4j.memory.ChatMemory` backed by in-memory collections and hydrated via H2:

Java

```java
@ApplicationScoped
public class HierarchicalMemoryStore implements ChatMemory {

    private SystemMessage pinnedInvariants;
    private UserMessage pinnedTaskSpec;
    private final List<ChatMessage> slidingScratchpad = new LinkedList<>();
    private final int maxScratchpadMessages = 4;

    public void pinInvariants(String invariants) {
        this.pinnedInvariants = new SystemMessage(invariants);
    }

    public void pinTaskSpecification(String story, String planJson) {
        this.pinnedTaskSpec = new UserMessage(String.format(
            "### TASK SPECIFICATION\n%s\n\n### APPROVED IMPLEMENTATION PLAN\n%s"
            story, planJson
        ));
    }

    @Override
    public void add(ChatMessage message) {
        if (message instanceof SystemMessage) {
            this.pinnedInvariants = (SystemMessage) message;
            return;
        }
        slidingScratchpad.add(message);
        while (slidingScratchpad.size() > maxScratchpadMessages) {
            slidingScratchpad.remove(0); // Evict oldest scratchpad message
        }
    }

    @Override
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
        }
        if (pinnedTaskSpec != null) {
            assembled.add(pinnedTaskSpec);
        }
        assembled.addAll(slidingScratchpad);
        return Collections.unmodifiableList(assembled);
    }

    @Override
    public void clear() {
        slidingScratchpad.clear();
    }
}
```

Step 2.4: Deterministic Secret Redaction Filter (`SecretRedactionFilter`)

Before storing tool inputs/outputs in H2 or writing to disk journals, all strings must pass through `SecretRedactionFilter` to prevent token leakage:

Target Patterns:

- GitHub Personal Access Tokens (`ghp_[a-zA-Z0-9]{36}` , `github_pat_[a-zA-Z0-9_]{82}`)
- Google Gemini API Keys (`AIzaSy[a-zA-Z0-9_-]{33}`)
- OpenAI / Anthropic Keys (`sk-[a-zA-Z0-9]{48}` , `sk-ant-[a-zA-Z0-9_-]{95}`)
- npm tokens (`npm_[a-zA-Z0-9]{36}`)
- HTTP Header Credentials (`Authorization: Bearer [a-zA-Z0-9._-]+` , `Basic [a-zA-Z0-9=+/]+`)

Redaction Logic:
Replaces all detected sensitive credential patterns with `***REDACTED_CREDENTIAL***` .

Java

```
@ApplicationScoped
public class SecretRedactionFilter {
    private static final List<Pattern> SECRET_PATTERNS = List.of(
        Pattern.compile("ghp_[a-zA-Z0-9]{36}"),
        Pattern.compile("github_pat_[a-zA-Z0-9_]{82}"),
        Pattern.compile("AIzaSy[a-zA-Z0-9_-]{33}"),
        Pattern.compile("sk-[a-zA-Z0-9]{48}"),
        Pattern.compile("sk-ant-[a-zA-Z0-9_-]{95}"),
        Pattern.compile("npm_[a-zA-Z0-9]{36}"),
        Pattern.compile("(?i)bearer\\s+[a-zA-Z0-9_\\-\\.=]+"),
        Pattern.compile("(?i)basic\\s+[a-zA-Z0-9_\\-+/=]+")
    );

    public String scrub(String input) {
        if (input == null || input.isBlank()) return input;
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
        return result;
    }
}
```

Step 2.5: Crash Recovery & Resumption Engine (`dev-agent resume`)

Allows interrupted or crashed workflows (e.g., shell termination, IDE restart, power outage) to resume seamlessly without re-drafting plans or generating duplicate Git branches:

1. **Dead Process Detection (`CrashRecoveryDetector.java`):**
   Queries H2 for incomplete tasks within the current repository:

Java

```java
@ApplicationScoped
public class CrashRecoveryDetector {

    @Inject
    ProjectIdResolver projectResolver;

    public Optional<TaskRunEntity> findResumableTask() {
        String currentProjectId = projectResolver.resolveProjectId();

        // Find the most recent incomplete task for this project
        TaskRunEntity latest = TaskRunEntity.find(
            "projectId = ?1 and status not in (?2, ?3) order by updatedAt desc",
            currentProjectId, TaskStatus.COMPLETED, TaskStatus.FAILED
        ).firstResult();

        if (latest == null) {
            return Optional.empty();
        }

        // Check if the registered PID is still active on the host OS
        if (latest.pid != null && ProcessHandle.of(latest.pid).isPresent()) {
            // Task is actively running in another shell
            return Optional.empty();
        }

        return Optional.of(latest);
    }
}
```

2. **The `ResumeCommand` CLI Implementation:**
   Add `ResumeCommand` to Picocli subcommands:

   Resolves the target task (auto-detects the last crashed task or accepts an explicit `<task-id>` argument).

   Verifies that the active Git branch matches `TaskRunEntity.branchName` .

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Injects project rules into Tier 1 (Pinned Invariants).

Restores the user story and approved plan into Tier 2 (Pinned Spec).

Reconstructs the last turns into Tier 3 (Sliding Scratchpad).

Resumes execution directly at `currentPhase` (e.g., resuming directly at the Green patch verification step, skipping planning and gatekeeper re-checks).

Step 2.6: Telemetry & Metric Persistence Pipeline

Integrate `MetricLogEntity` with LLM responses and tool runs:

On every LangChain4j response, record `promptTokens`, `completionTokens`, model name, and execution duration.

Compute running financial cost estimates using configured per-token rates (e.g., Gemini Pro vs Claude Sonnet).

Output telemetry markers to the CLI in verbose mode (`[TOKENS: 1,240 | SPEND: $0.0024 | DURATION: 1,420ms]`).

Step 2.7: Verification, Test Suite & Quality Gates

Phase 2 must pass rigorous unit and integration gates before sign-off:

1. **Level 1 Unit & Secret Scrubbing Tests (`src/test/`):**

   `SecretRedactionFilterTest`:

   Assert GitHub tokens, Gemini keys, OpenAI keys, and Authorization headers are masked to `***REDACTED_CREDENTIAL***`.

   Assert general source code and normal text strings are untouched.

   `HierarchicalMemoryStoreTest`:

   Add 10 successive chat messages to the memory store.

   Assert total messages returned never exceed 6 ($1\ \text{System} + 1\ \text{Task Spec} + 4\ \text{Scratchpad}$).

   Assert that Tier 1 (Invariants) and Tier 2 (Task Spec & Plan) are strictly preserved at indices 0 and 1.

2. **Level 2 Relational & Resumption Tests (`@QuarkusTest`, `@QuarkusMainTest`):**

   `StatePersistenceIntegrationTest`:

   Insert a `TaskRunEntity` across two simulated projects (`project-alpha` and `project-beta`).

   Assert queries with `projectId` namespace return isolated records without cross-talk.

   Verify that `StateSyncService` projects `.agents/runs/<task-id>.md` to disk matching H2 state exactly.

   `CrashRecoveryWorkflowResumptionTest` (`@QuarkusMainTest`):

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Assert the agent skips planning, loads the approved plan from H2, and executes the patch verification step immediately, returning exit code 0.

4. Phase 2 Definition of Done (DoD) Checklist

Markdown

```
[ ] Global Datasource: Embedded H2 database initializes cleanly at ~/.config/dev-agen
[ ] Multi-Tenancy: TaskRunEntity, StepAuditEntity, and MetricLogEntity isolate data v
[ ] Dual-Representation: StateSyncService writes to H2 first, then projects ./.agents
[ ] 3-Way Compatibility: External agents (agy, Claude Code) and human developers can
[ ] 3-Tier Memory: Pinned Invariants and Task Spec DAG are never evicted during itera
[ ] Secret Redaction: SecretRedactionFilter masks GitHub PATs, Gemini keys, OpenAI to
[ ] Dead Process Detection: CrashRecoveryDetector accurately
[ ] Resume Command: 'dev-agent resume' hydrates context fro
[ ] Telemetry Logging: MetricLogEntity accurately records pr
[ ] Automated Test Suite: All Level 1 unit/redaction tests and Level 2 persistence/re
```

Continue with phase 3

Phase 3: Dynamic ADR Semantic RAG, Decision Caching & Oscillatory Loop Breakers (Detailed Specification)

1. Architectural Scope & Boundary Definition

The primary objective of **Phase 3** is to eliminate prompt bloat, token waste, cognitive attention degradation ("lost-in-the-middle" drift), and infinite self-healing loops across your Quarkus and frontend codebases.

In Phases 1 and 2, the agent either injected all discovered project rules into `@V("rulesContext")` or relied on basic file parsing. In complex Quarkus and full-stack projects, architecture rules and sealed ADRs can span dozens of documents (e.g., reactive Mutiny event-loop rules, CDI scoping, zero-GC buffer streaming, OpenAPI sync, TypeScript strictness, and WCAG AA guidelines). Injecting every rule into every prompt inflates token consumption by 10,000 to 20,000 tokens per turn, dilutes model reasoning, and drives up API costs.

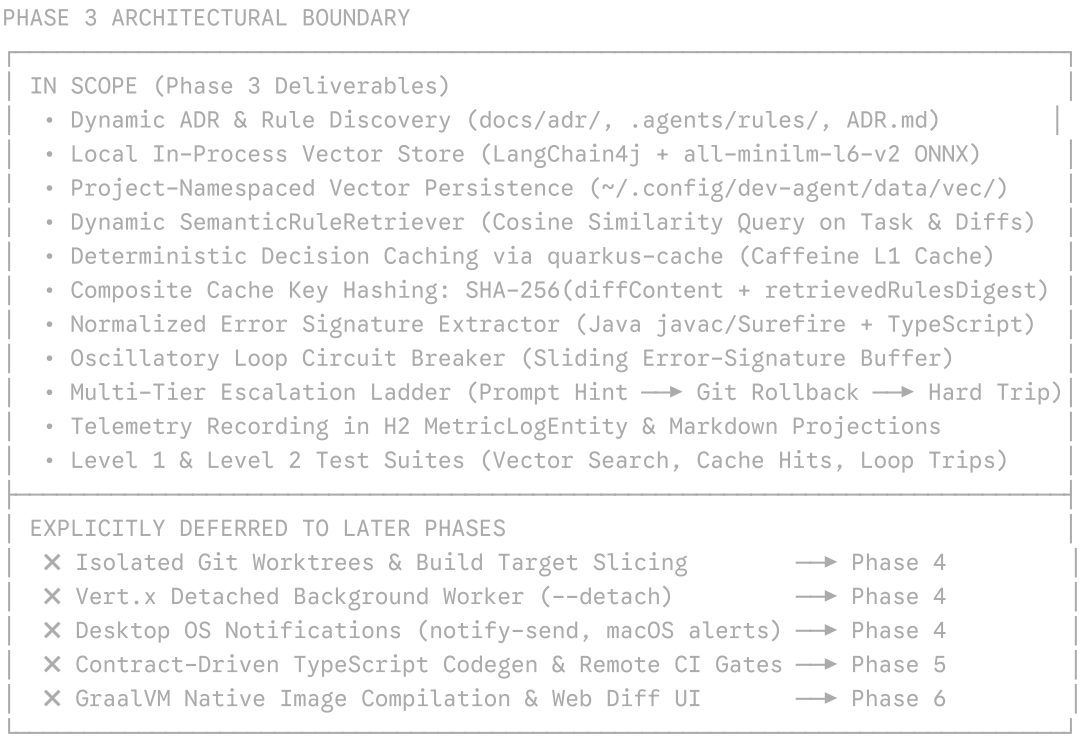
Phase 3 resolves this by implementing:

1. **Dynamic ADR & Rules Ingestion**: Auto-discovers and segments architectural decisions from any standard directory structure (`docs/adr/` , `.agents/rules/` , etc.).
2. **Local In-Process Semantic RAG**: Runs an in-memory/embedded vector store using an ONNX-based embedding model (`all-minilm-l6-v2`), retrieving only the **top 3–5 relevant ADRs** per task with zero external API calls.
3. **Deterministic Decision Caching (`quarkus-cache`)**: Memoizes `temperature: 0.0` Architect Gatekeeper evaluations using composite hashes of the diff and retrieved rule digests, returning cached verdicts in 0 ms.
4. **Normalized Error Signatures & Oscillatory Loop Breaker**: Strips line numbers to detect when the LLM alternates between recurring compiler/test errors (e.g., $A \rightarrow B \rightarrow A \rightarrow B$), executing an automated rollback before depleting retry budgets.

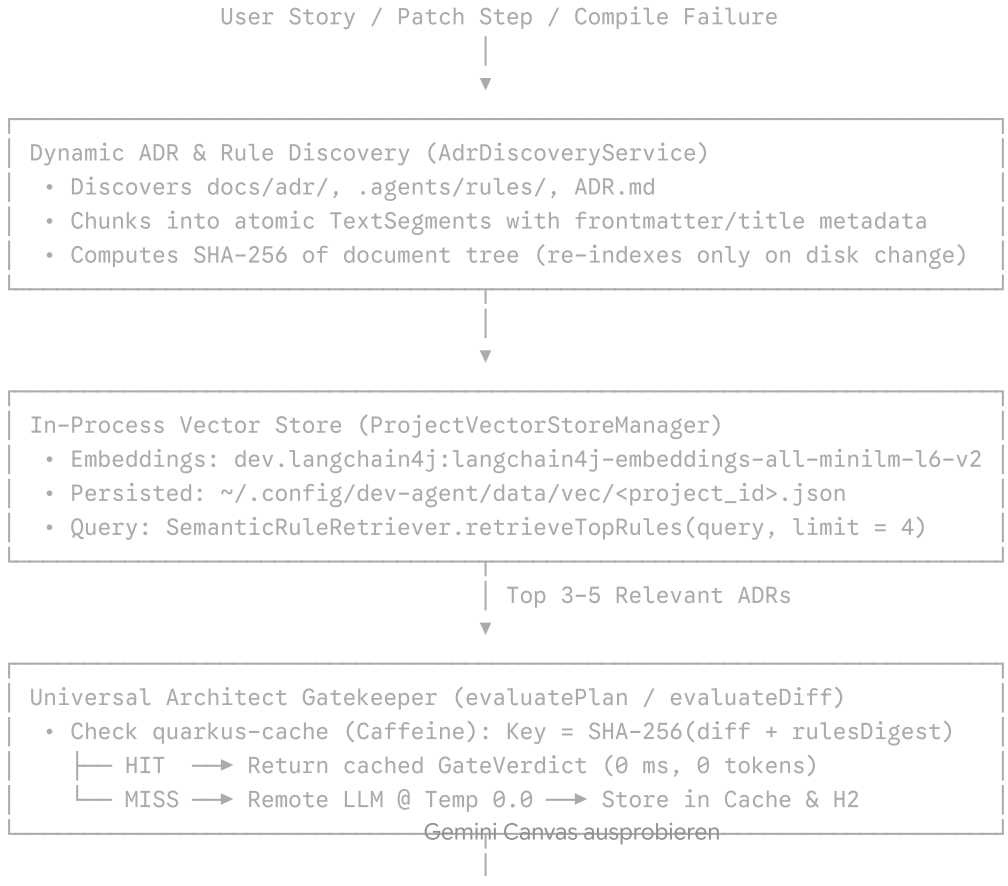
Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps



2. Semantic Retrieval & Decision Caching Pipeline



Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
│  • ErrorSignatureExtractor: Normalizes error (strips line numbers)      │
│  • LoopCircuitBreaker: Evaluates sliding history                        │
│    ├── PROGRESSING           ──► Pass sanitized log to Coder model      │
│    ├── CONSECUTIVE_REPEAT    ──► Inject targeted prompt warning         │
│    └── OSCILLATING_REGRESSION──► Execute Git rollback to snapshot       │
└─────────────────────────────────────────────────────────────────────────┘
```

3. Detailed Step-by-Step Implementation Breakdown

Step 3.1: Dynamic ADR & Rules Discovery Engine (`AdrDiscoveryService`)

Decouple the agent from any hardcoded file locations. Implement `AdrDiscoveryService` to dynamically inspect the active project:

1. **Path Resolution Cascade:**
   Probes the project root for existing documentation layouts:
   - `${REPO_ROOT}/.agents/rules/*.md`
   - `${REPO_ROOT}/docs/adr/*.md` or `${REPO_ROOT}/doc/adr/*.md`
   - `${REPO_ROOT}/docs/architecture/decisions/*.md`
   - `${REPO_ROOT}/docs/tech/architecture/adrs.md` (backward compatibility with OmniDepot)
   - `${REPO_ROOT}/ADR.md` or `${REPO_ROOT}/ARCHITECTURE.md`
2. **Document Segmentation:**
   - For individual Markdown files: Parses frontmatter (e.g., `id: ADR-006` , `title: Zero-GC Streaming`) or top-level `# ADR-(\\d+): (.*)` headers.
   - For combined documents: Splits content on Markdown headings (`## ADR-` or `### Rule:`) to ensure each architectural invariant is an atomic, self-contained `TextSegment` .
3. **Document Invariant Hashing:**
   Computes a combined `SHA-256` checksum across all discovered rule files. If the hash matches the cached hash in H2, re-indexing is completely bypassed on agent boot (<10 ms).

Step 3.2: Local In-Process Embedding Store (`ProjectVectorStoreManager`)

Implement an embedded vector pipeline that requires zero external network calls, API keys, or Docker containers:

1. **Dependencies (`pom.xml`):**

XML

```xml
<dependency>
    <groupId>dev.langchain4j</groupId>
    <artifactId>langchain4j-embeddings-all-minilm-l6-v2</artifactId>
</dependency>
```

Gemini ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
        <artifactId>langchain4j-core</artifactId>
    </dependency>
```

2. Vector Store Management:

   Uses `InMemoryEmbeddingStore<TextSegment>` backed by JSON serialization to disk at `${user.home}/.config/dev-agent/data/vec/<project_id>.json` .

   Uses `AllMiniLmL6V2EmbeddingModel` running entirely in-process via ONNX runtime.

   Embeds rules on initialization if the project hash is missing or modified:

Java

```java
@ApplicationScoped
public class ProjectVectorStoreManager {

    private final EmbeddingModel embeddingModel = new AllMiniLmL6V2Embedding
    private InMemoryEmbeddingStore<TextSegment> store;

    public void initializeStore(String projectId, List<TextSegment> segments
        Path cacheFile = Path.of(System.getProperty("user.home"),
            ".config", "dev-agent", "data", "vec", projectId + ".json");

        if (Files.exists(cacheFile) && !isHashModified(projectId, currentHas
            this.store = InMemoryEmbeddingStore.fromFile(cacheFile.toString
            return;
        }

        this.store = new InMemoryEmbeddingStore<>();
        List<Embedding> embeddings = embeddingModel.embedAll(segments).conte
        this.store.addAll(embeddings, segments);
        this.store.serializeToFile(cacheFile.toString());
        updateStoredHash(projectId, currentHash);
    }

    public List<EmbeddingMatch<TextSegment>> findRelevant(String query, int
        Embedding queryEmbedding = embeddingModel.embed(query).content();
        return store.findRelevant(queryEmbedding, maxResults, minScore);
    }
}
```

Step 3.3: Dynamic Semantic Rule Retriever (`SemanticRuleRetriever`)

Wire the retriever into the cognitive layer to supply focused architectural guidance:

1. Context-Aware Query Formulation:
   Formulates queries using the user's task prompt combined with touched file paths:

   Example: User prompt `"Add zero-copy multipart upload endpoint"` + touched path `S3StorageAdapter.java` → Query:

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Example: User prompt `"Create user summary status card"` + touched path `status-card.component.ts` → Query: `"angular signals material design 3 component accessibility tokens"`.

2. **Selective Context Retrieval:**
   Retrieves the top 3–5 segments with cosine similarity ≥ 0.70.

3. **Dynamic Prompt Injection:**
   Injects retrieved segments into `@V("retrievedAdrs")` for `FullStackPlannerAiService` and `UniversalArchitectGatekeeperAiService`:

Java

```java
@ApplicationScoped
public class SemanticRuleRetriever {

    @Inject
    ProjectVectorStoreManager vectorStoreManager;

    public String retrieveFormattedRules(String taskDescription, List<String> to
        String combinedQuery = taskDescription + " " + String.join(" ", touchedF
        List<EmbeddingMatch<TextSegment>> matches = vectorStoreManager.findRelev

        if (matches.isEmpty()) {
            return "Standard project coding standards apply.";
        }

        StringBuilder sb = new StringBuilder("### RELEVANT ARCHITECTURAL DECISIO
        for (EmbeddingMatch<TextSegment> match : matches) {
            sb.append(String.format("- [%s] (Relevance: %.2f):\n%s\n\n",
                match.embedded().metadata().getString("id"),
                match.score(),
                match.embedded().text()));
        }
        return sb.toString();
    }
}
```

Impact: Reduces gatekeeper prompt size from $\sim 15,000$ tokens to under $3,000$ tokens ($> 75\%$ reduction), avoiding token limits and LLM attention drift.

Step 3.4: Deterministic Decision Caching (`quarkus-cache`)

Architectural evaluations at `temperature: 0.0` are purely deterministic functions of the code diff and the active rules. Repeated evaluations must not make remote LLM calls.

1. **Extension Configuration:**
   Add `io.quarkus:quarkus-cache` to `pom.xml`. Configure Caffeine cache settings in `application.yaml`:

Gemini Canvas ausprobieren

YAML

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
caffeine:
  "architect-verdicts":
    initial-capacity: 50
    maximum-size: 500
    expire-after-write: 24h
```

2. **Composite Cache Key Generation:**
   Generates a SHA-256 hash using the unified diff content combined with the hash of the active rules context:

$$K_{\text{cache}} = \text{SHA-256}(\text{UnifiedDiff} + \text{RetrievedAdrsDigest})$$

3. **Cached Service Wrapper (** `CachedGatekeeperService.java` **):**

Java

```java
@ApplicationScoped
public class CachedGatekeeperService {

    @Inject
    UniversalArchitectGatekeeperAiService gatekeeperAi;

    @CacheResult(cacheName = "architect-verdicts")
    public GateVerdict evaluate(String cacheKey, String codeDiff, String retriev
        return gatekeeperAi.evaluatePlan(codeDiff, retrievedRules);
    }

    public String computeCacheKey(String codeDiff, String retrievedRules) {
        return Hashing.sha256()
            .hashString(codeDiff + "::" + retrievedRules, StandardCharsets.UTF_8
            .toString();
    }
}
```

Impact: Minor whitespace adjustments, re-verifications, or previously audited code blocks return cached verdicts in < 5 ms with 0 LLM tokens spent.

Step 3.5: Normalized Error Signature Extractor & Loop Circuit Breaker

During iterative self-healing loops, LLMs often trigger cyclic regressions (e.g., fixing a compile error in class A causes a method failure in class B; fixing class B reintroduces the original error in class A). Because line numbers shift as code is inserted or removed, naive string comparison fails to detect these loops.

```
Iterative TDD Turn N ──► Normalized Compiler/Test Error: E(N)
                                        │
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
                    | Evaluates Pattern
                    ▼

┌──────────────────────────────────────────────────────┐
| Escalation Ladder                                    |
|  • [A ⟶ A]          : CONSECUTIVE_REPEAT             |
|    Action: Inject prompt warning into scratchpad     |
|                                                      |
|  • [A ⟶ B ⟶ A ⟶ B]: OSCILLATING_REGRESSION          |
|    Action: Git rollback to snapshot + force pivot    |
|                                                      |
|  • Repeat after pivot : HARD_TRIP (Exit Code 1)      |
└──────────────────────────────────────────────────────┘
```

1. **Normalized Error Signature Algorithm (** `ErrorSignatureExtractor.java` **):** Strips line numbers, column offsets, and dynamic filesystem prefixes to isolate invariant error semantics:

Java

```java
@ApplicationScoped
public class ErrorSignatureExtractor {

    // Matches: [ERROR] /path/to/Class.java:[42,15] error message
    private static final Pattern JAVA_COMPILER_ERROR = Pattern.compile(
        "^\\[ERROR\\]\\s+.*?/([^/]+\\.java):\\[\\d+,\\d+\\]\\s+(.*)$", Pattern.M

    // Matches: TypeScript error: src/app/comp.ts(12,5): error TS2322: ...
    private static final Pattern TS_COMPILER_ERROR = Pattern.compile(
        "^(.*?/)?([^/]+\\.ts)\\((\\d+),(\\d+)\\):\\s+error\\s+(TS\\d+):\\s+(.*)$

    // Matches JUnit / Vitest Assertion Errors
    private static final Pattern ASSERTION_ERROR = Pattern.compile(
        "(AssertionFailedError|AssertionError|ComparisonFailure):\\s+(.*?)(?=\\n

    public String extractNormalizedSignature(String sanitizedLog) {
        if (sanitizedLog == null || sanitizedLog.isBlank()) {
            return "NO_ERROR";
        }

        // 1. Check Java compilation
        Matcher javaMatcher = JAVA_COMPILER_ERROR.matcher(sanitizedLog);
        if (javaMatcher.find()) {
            return String.format("JAVA_COMPILER[%s]: %s", javaMatcher.group(1),
        }

        // 2. Check TypeScript compilation
        Matcher tsMatcher = TS_COMPILER_ERROR.matcher(sanitizedLog);
        if (tsMatcher.find()) {
            return String.format("TS_COMPILER[%s]: %s - %s", tsMatcher.group(2),
        }

        // 3. Check Test Assertions
        Matcher assertMatcher = ASSERTION_ERROR.matcher(sanitizedLog);
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
            // Fallback to normalized hash of sanitized content
            return "RAW_HASH[" + Integer.toHexString(sanitizedLog.replaceAll("\\d+",
        }
    }
```

2. Circular Loop Detection Engine (`LoopCircuitBreaker.java`):
Maintains a bounded history of the last 6 error signatures:

Java

```java
@ApplicationScoped
public class LoopCircuitBreaker {

    private final List<String> history = new ArrayList<>();
    private static final int WINDOW_SIZE = 6;

    public LoopVerdict recordAndEvaluate(String normalizedSignature) {
        history.add(normalizedSignature);
        if (history.size() > WINDOW_SIZE) {
            history.remove(0);
        }

        int size = history.size();

        // Consecutive Repeat: A -> A
        if (size >= 2 && history.get(size - 1).equals(history.get(size - 2))) {
            return LoopVerdict.CONSECUTIVE_REPEAT;
        }

        // Oscillating Regression: A -> B -> A -> B
        if (size >= 4 &&
            history.get(size - 1).equals(history.get(size - 3)) &&
            history.get(size - 2).equals(history.get(size - 4))) {
            return LoopVerdict.OSCILLATING_REGRESSION;
        }

        return LoopVerdict.PROGRESSING;
    }

    public void clear() {
        history.clear();
    }

    public enum LoopVerdict {
        PROGRESSING,
        CONSECUTIVE_REPEAT,
        OSCILLATING_REGRESSION
    }
}
```

3. Multi-Tier Escalation Ladder:

Gemini Canvas ausprobieren

Tier 1 (`CONSECUTIVE_REPEAT`): Injects an explicit directive into the Tier 3 sliding

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
compiler/test error. Do NOT attempt the same edit.
Inspect the surrounding context and method parameters
carefully."
```

Tier 2 (`OSCILLATING_REGRESSION` **):** Invokes `GitCliService.rollbackToSnapshot()`, reverting dirty edits. Injects a strategy pivot prompt:

```
"REGRESSION DETECTED: You are oscillating between two
conflicting compiler/test states. Your modifications
have been rolled back. Pivot your implementation
strategy entirely—do not use this pattern."
```

Tier 3 (Repeat After Pivot / Exhaustion): Sets `TaskRunEntity.status = FAILED_CIRCUIT_BREAKER`, cleans up the workspace via `git reset --hard`, displays ANSI red diagnostics in the terminal, and halts execution with exit code 1.

Step 3.6: State & Metric DB Integration

Persist semantic retrieval performance and cache metrics into the relational H2 database from Phase 2:

1. **Entity Extensions (** `MetricLogEntity.java` **):**
   Add telemetry columns to capture retrieval and cache efficiency:

Java

```java
@Column(name = "cache_hit")
public boolean cacheHit;

@Column(name = "retrieved_adrs")
public String retrievedAdrs; // e.g., "ADR-006,ADR-012"
```

2. **State Projection Synchronization:**
   When a cache hit occurs, `StateSyncService` records a step audit in H2 with `durationMs < 5`, `promptTokens = 0`, and `outputSummary = "CACHE_HIT (Skipped remote LLM invocation)"`.
   The generated `./.agents/runs/<task-id>.md` journal reflects these token savings in its session metrics.

Step 3.7: Verification, Test Suite & Quality Gates

Phase 3 must satisfy strict Level 1 and Level 2 test gates before sign-off:

1. **Level 1 Unit Tests (** `src/test/` **):**

`SemanticRuleRetrieverTest`:

Feed test prompt: Gemini Canvas ausprobieren

```
"Stream multi-gigabyte container image with zero memory
```

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Assert top match is `ADR-006: Zero-GC Streaming` with cosine similarity ≥ 0.75.

Assert unrelated rules (e.g., CSS themes, database migrations) are excluded from the top results.

`ErrorSignatureExtractorTest`:

Feed two compiler error logs where line numbers shift from `[42,15]` to `[89,20]`.

Assert both produce identical normalized signatures (`JAVA_COMPILER[MyClass.java]: cannot find symbol`).

Feed TypeScript errors with shifting line coordinates; assert identical normalized signatures.

`LoopCircuitBreakerTest`:

Feed sequence `[ERR_A, ERR_B, ERR_A, ERR_B]`.

Assert verdict transitions from `PROGRESSING` to `OSCILLATING_REGRESSION` on the 4th turn.

2. **Level 2 Integration & Cache Tests (`@QuarkusTest`):**

`DecisionCachingIntegrationTest`:

Invoke `CachedGatekeeperService.evaluate()` twice with identical code diffs and rule sets.

Use WireMock to verify the remote LLM endpoint is invoked **exactly once**.

Assert the second call returns in < 5 ms with `cacheHit = true`.

`OscillatingSelfHealingLoopContractTest` (`@QuarkusMainTest`):

Seed a test project with an oscillating mock compiler stub.

Run `dev-agent feature "Oscillate" --yes`.

Assert the CLI terminates with exit code 1, reports `CIRCUIT_BREAKER_TRIPPED`, and leaves the Git workspace clean via automatic rollback.

4. Phase 3 Definition of Done (DoD) Checklist

Markdown

```
[ ] Dynamic Rule Discovery: AdrDiscoveryService discovers rules across docs/adr/, .ag
[ ] In-Process Vector Store: langchain4j-embeddings-all-minilm-l6-v2 runs locally wit
[ ] Vector Cache: Embedding vectors serialize to ~/.config/dev-agent/data/vec/<projec
[ ] Context Optimization: SemanticRuleRetriever fetches top 3-5 ADRs, cutting Gatekee
[ ] Caffeine L1 Cache: quarkus-cache memoizes Gatekeeper evaluations at temperature:
[ ] Cache Invalidation: Composite key SHA-256(diff + rulesDigest) ensures code or rul
[ ] Error Normalizer: ErrorSignatureExtractor strips dynamic line and column numbers
[ ] Oscillatory Loop Breaker: Actively catches failure loops
[ ] Escalation Safety: Consecutive repeats inject promp
```

Gemini Canvas ausprobieren

Now let's do this for phase 4

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Phase 4: Non-Blocking Workspaces, Build Slicing & Detached Workers (Detailed Specification)

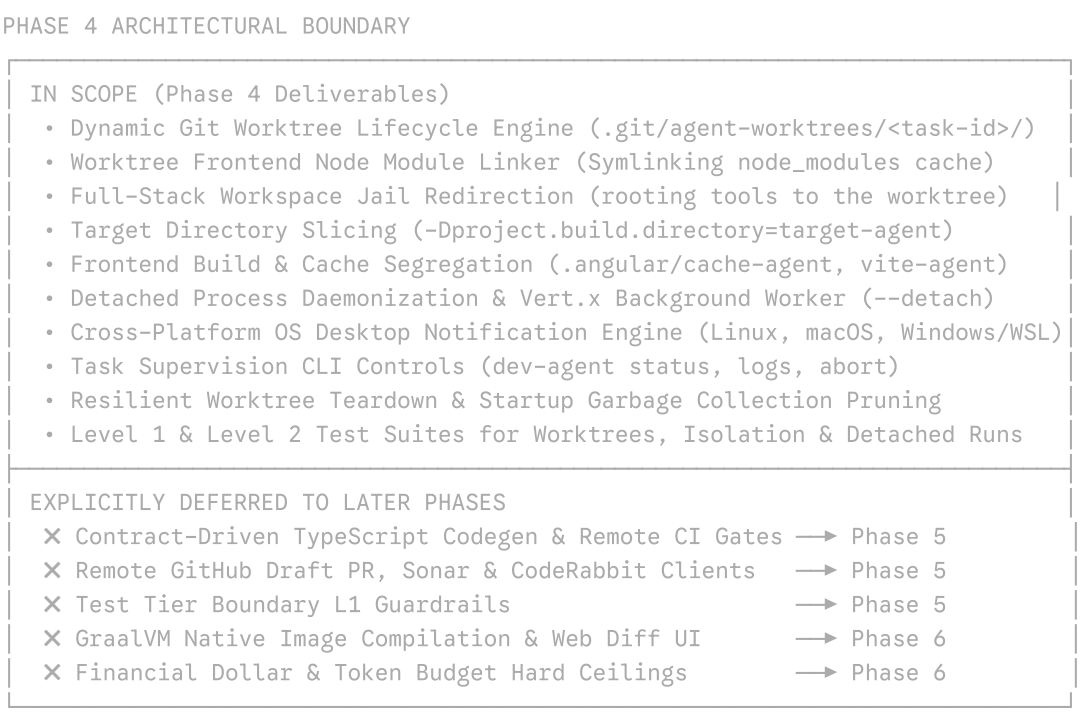
1. Architectural Scope & Boundary Definition

The primary objective of **Phase 4** is to completely decouple the autonomous agent from the developer's active shell and IDE workstation.

In Phases 1–3, the agent executed within the developer's active working tree (`user.dir`). For simple, instantaneous edits this was adequate; however, in real-world full-stack development, continuous compilation, test execution, and multi-turn self-healing loops take several minutes. If the developer continues working in parallel:

- Switching branches or staging files in IntelliJ/WebStorm triggers Git lock contention (`.git/index.lock`).
- IDE background compilation and `quarkus:dev` collide with agent build tasks over locked `target/classes` or `build/classes` directories.
- Frontend dev servers (e.g., Vite, Angular CLI) lock output directories or trigger unwanted live-reloads during incremental agent code edits.
- Closing the terminal or shell disconnects the process, aborting in-flight work.

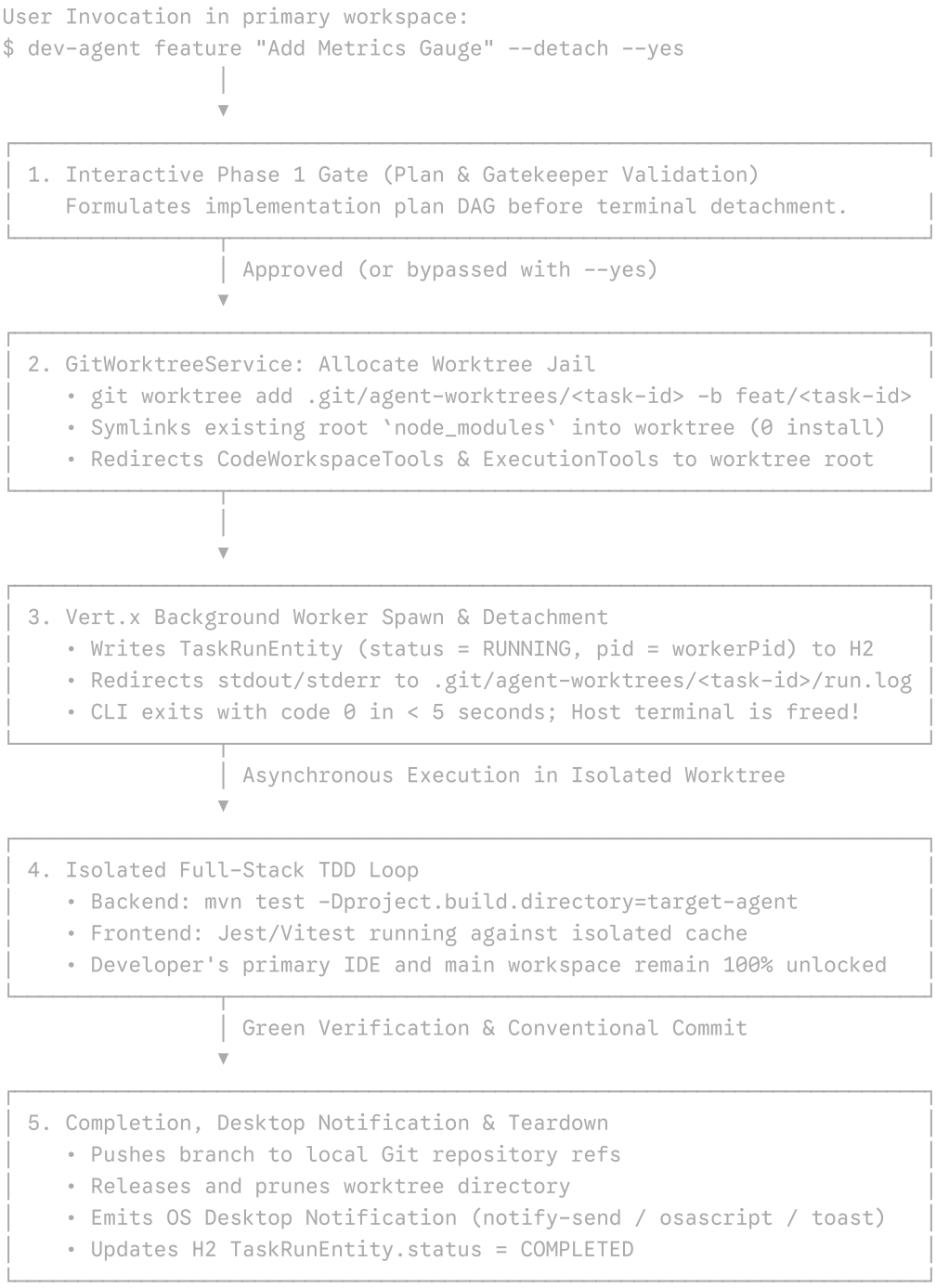
Phase 4 resolves this by isolating agent operations inside dedicated **Git Worktrees**, segregating build outputs via **Build Slicing** (`target-agent/` and isolated frontend caches), executing long-running runs as **Detached Background Workers** powered by Eclipse Vert.x, and providing a complete **CLI Task Supervision Suite** (`status` , `logs --follow` , `abort`) coupled with **Cross-Platform OS Desktop Notifications**.



Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps



3. Detailed Step-by-Step Implementation Breakdown

Step 3.1: Git Worktree Lifecycle Engine (`GitWorktreeService`)

Implement `GitWorktreeService` as an `@ApplicationScoped` CDI bean to allocate, configure, and release detached workspaces:

1. **Dedicated Worktree Directory:**
   All isolated agent runs are placed under `${REPO_ROOT}/.git/agent-worktrees/<task-id>/`. Because this path lives inside

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Java

```java
@ApplicationScoped
public class GitWorktreeService {

    @Inject
    WorkspaceContextResolver workspaceResolver;

    public Path allocateWorktree(String taskId, String branchName) throws IOExce
        Path repoRoot = workspaceResolver.getProjectRoot();
        Path worktreePath = repoRoot.resolve(".git/agent-worktrees").resolve(tas

        if (Files.exists(worktreePath)) {
            releaseWorktree(taskId);
        }
        Files.createDirectories(worktreePath.getParent());

        // Create isolated worktree checked out to the feature branch
        ProcessBuilder pb = new ProcessBuilder(
            "git", "worktree", "add", worktreePath.toString(), "-b", branchName
        ).directory(repoRoot.toFile());

        Process process = pb.start();
        if (!process.waitFor(15, TimeUnit.SECONDS) || process.exitValue() != 0)
            String err = new String(process.getErrorStream().readAllBytes(), Sta
            throw new IllegalStateException("Failed to allocate Git worktree: "
        }

        linkFrontendDependencies(repoRoot, worktreePath);
        return worktreePath;
    }

    public void releaseWorktree(String taskId) {
        Path repoRoot = workspaceResolver.getProjectRoot();
        Path worktreePath = repoRoot.resolve(".git/agent-worktrees").resolve(tas

        try {
            ProcessBuilder pb = new ProcessBuilder(
                "git", "worktree", "remove", "--force", worktreePath.toString()
            ).directory(repoRoot.toFile());
            pb.start().waitFor(15, TimeUnit.SECONDS);

            // Prune worktree metadata references
            new ProcessBuilder("git", "worktree", "prune")
                .directory(repoRoot.toFile()).start().waitFor(5, TimeUnit.SECOND
        } catch (Exception e) {
            System.err.println("WARN: Failed to cleanly release worktree " + tas
        }
    }
}
```

3. **Frontend Dependency Symlinking (`linkFrontendDependencies`):**
   Running `npm install` or `pnpm install` in every new worktree can take minutes and consume gigabytes of disk. If the root workspace has an existing `node_modules/`, the

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Java

```java
private void linkFrontendDependencies(Path rootProject, Path worktreePath) {
    Path rootNodeModules = rootProject.resolve("frontend/node_modules");
    Path worktreeNodeModules = worktreePath.resolve("frontend/node_modules");

    if (Files.exists(rootNodeModules) && !Files.exists(worktreeNodeModules)) {
        try {
            Files.createDirectories(worktreeNodeModules.getParent());
            Files.createSymbolicLink(worktreeNodeModules, rootNodeModules);
        } catch (IOException e) {
            System.err.println("WARN: Could not symlink node_modules; frontend t
        }
    }
}
```

Step 3.2: Full-Stack Workspace Redirection & Build Slicing

Ensure all tool invocations and compile steps operate strictly within the worktree boundaries without file lock collisions:

1. **Context-Aware Tool Rooting**:

   Update `WorkspaceContextResolver` to maintain a mutable execution root (`activeWorkingRoot`).

   In interactive foreground mode, `activeWorkingRoot = REPO_ROOT` .

   In detached worktree mode, `activeWorkingRoot = .git/agent-worktrees/<task-id>/` .

   All calls in `CodeWorkspaceTools` (`createFile` , `readFile` , `patchFile`) and `WorkspaceJailValidator` automatically clamp file resolutions to the worktree.

2. **Backend Target Directory Slicing (`target-agent/`)**:
   When `BuildExecutionTools` runs Maven or Gradle, append dedicated target properties :

   Maven: `-Dproject.build.directory=target-agent` .

   Gradle: `-PbuildDir=build-agent` .

   Impact: Compilation outputs, Surefire XML reports, and generated bytecode write exclusively to `target-agent/` . The developer's primary `target/` directory remains unlocked for background IDE compilation or continuous `mvn quarkus:dev` .

3. **Frontend Cache Slicing**:
   Configure `FrontendExecutionTools` to isolate build artifacts and compiler caches:

   Jest / Vitest: Append `--cacheDirectory=node_modules/.cache-agent` .

   TypeScript: Pass `--tsBuildInfoFile target-agent/tsconfig.tsbuildinfo` .

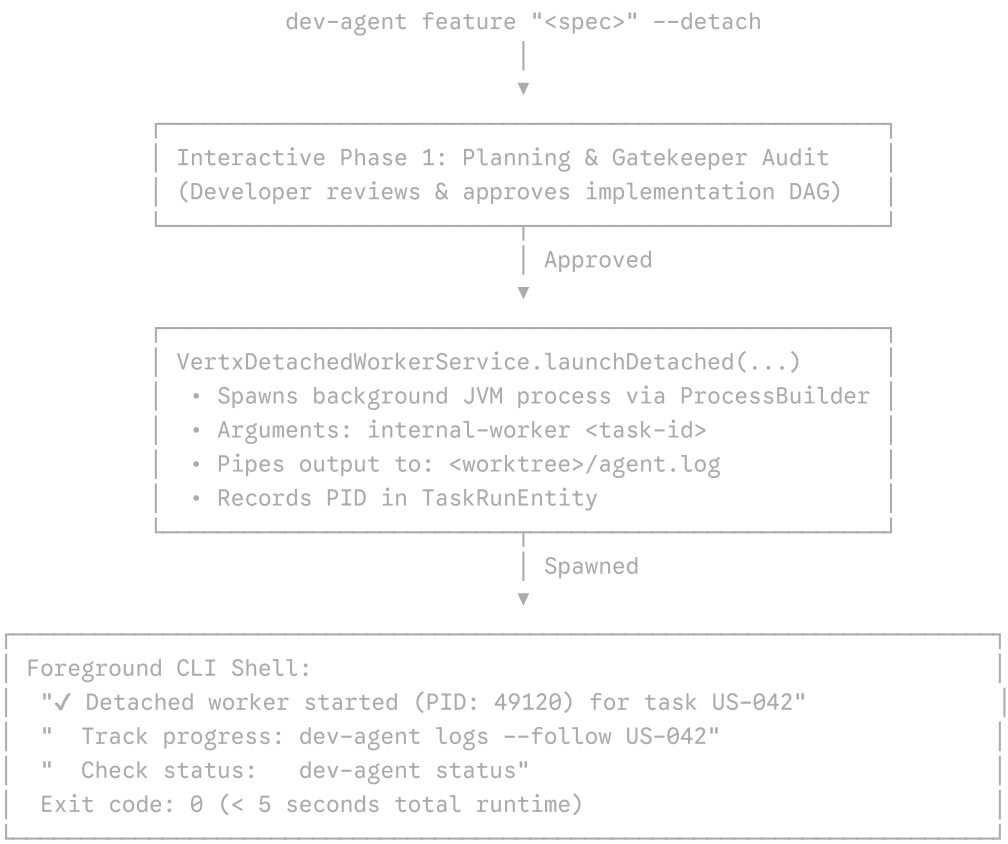
   Prevents cache collisions with active frontend dev servers (`ng serve` or `vite`).

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Implement detached execution so that lengthy code generation, compilation, and testing run asynchronously in the background:



1. The Detached Spawner (`VertxDetachedWorkerService.java`):

Java

```java
@ApplicationScoped
public class VertxDetachedWorkerService {

    @Inject
    ProjectIdResolver projectIdResolver;

    @Transactional
    public void launchDetached(String taskId, String userStory, Path worktreePat
        String javaBin = ProcessHandle.current().info().command()
            .orElse(Path.of(System.getProperty("java.home"), "bin", "java").toSt

        Path logFile = worktreePath.resolve("agent.log");
        String projectId = projectIdResolver.resolveProjectId();

        // Launch detached child JVM executing the internal execution command
        ProcessBuilder pb = new ProcessBuilder(
            javaBin,
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
            pb.directory(worktreePath.toFile());
            pb.redirectOutput(ProcessBuilder.Redirect.appendTo(logFile.toFile()));
            pb.redirectError(ProcessBuilder.Redirect.appendTo(logFile.toFile()));

            Process process = pb.start();
            long workerPid = process.pid();

            TaskRunEntity task = TaskRunEntity.findByProjectAndTask(projectId, taskI
            if (task != null) {
                task.pid = workerPid;
                task.status = TaskStatus.RUNNING;
                task.updatedAt = Instant.now();
                task.persist();
            }

            System.out.printf("✓ Task [%s] detached to background process (PID: %d)%
            System.out.printf("  Log stream: %s%n", logFile);
            System.out.printf("  To follow:  dev-agent logs --follow %s%n", taskId);
        }
    }
```

2. Internal Hidden Subcommand (`InternalWorkerCommand.java`):
   Picocli registers a hidden command
   `@Command(name = "internal-worker", hidden = true)` :

   Executes Phases 2 and 3 (Red/Green TDD loop across backend and frontend).

   Catches all exceptions, records errors to H2, triggers desktop notifications on completion or failure, and releases the worktree.

Step 3.4: Cross-Platform OS Desktop Notification Engine (`DesktopNotificationService`)

When running detached, developers switch to other windows or tasks. Desktop notifications alert the developer when attention is required:

Java

```
@ApplicationScoped
public class DesktopNotificationService {

    public void notify(String title, String message, boolean isError) {
        String os = System.getProperty("os.name").toLowerCase();

        try {
            if (os.contains("linux")) {
                // Linux / freedesktop notifications
                new ProcessBuilder(
                    "notify-send",
                    "-u", isError ? "critical" : "normal",
                    "-a", "dev-agent",
                    title, message
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
                new ProcessBuilder("osascript", "-e", script).start();
            } else if (os.contains("win") || isWsl()) {
                // Windows & WSL2 via PowerShell toast bridge
                String psCommand = String.format(
                    "[Windows.UI.Notifications.ToastNotificationManager, Windows.UI.N
                    "$template = [Windows.UI.Notifications.ToastNotificationManager]:
                    "$textNodes = $template.GetElementsByTagName('text'); " +
                    "$textNodes.Item(0).AppendChild($template.CreateTextNode('%s')) >
                    "$textNodes.Item(1).AppendChild($template.CreateTextNode('%s')) >
                    "$notifier = [Windows.UI.Notifications.ToastNotificationManager]:
                    "$notifier.Show([Windows.UI.Notifications.ToastNotification]::new
                    title, message
                );
                new ProcessBuilder("powershell.exe", "-Command", psCommand).start();
            }
        } catch (Exception e) {
            System.err.println("DEBUG: Could not deliver desktop notification: " + e.
        }
    }

    private boolean isWsl() {
        return System.getenv("WSL_DISTRO_NAME") != null;
    }
}
```

Step 3.5: Task Supervision CLI Controls (`status` , `logs` , `abort`)

Provide developers with first-class terminal management over detached runs:

1. `dev-agent status` (`StatusCommand.java`):
   Queries `TaskRunEntity` from H2 and renders an ANSI table showing running, completed, and failed tasks:

```
QUARKUS DEV-AGENT: ACTIVE BACKGROUND TASKS
┌──────────────────────┬──────────────────────────┬──────────────┬──────────────┬──
│ PROJECT              │ TASK ID                  │ PHASE        │ RUNNING TIME │ S
├──────────────────────┼──────────────────────────┼──────────────┼──────────────┼──
│ omnidepot            │ US-042-cas-s3            │ GREEN_PATCH  │ 02m 14s      │ R
│ store-front-ui       │ US-089-cart-signals      │ RED_TEST     │ 00m 45s      │ R
│ auth-service         │ US-012-oauth-pkce        │ COMPLETED    │ 04m 12s      │ S
└──────────────────────┴──────────────────────────┴──────────────┴──────────────┴──
```

   Live Process Verification: Validates `ProcessHandle.of(entity.pid).isPresent()` . If a process terminated abruptly without updating H2, status displays `ORPHANED (Dead PID)` in yellow.

2. `dev-agent logs --follow <task-id>` (`LogsCommand.java`):
   Inspects `.git/agent-worktrees/<task-id>/agent.log` and tails live outputs to stdout using non-blocking NIO `FileChannel` .

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Looks up `TaskRunEntity.pid` in H2 and terminates the process forcibly:

```java
ProcessHandle.of(entity.pid).ifPresent(ProcessHandle::destroyForcibly);
```

Calls `gitWorktreeService.releaseWorktree(taskId)` to remove the directory.

Sets `TaskRunEntity.status = ABORTED` and updates `.agents/runs/<task-id>.md`.

Step 3.6: Resilient Worktree Teardown & Startup Garbage Collection

Prevent disk leaks and broken Git references caused by killed tasks or rebooted machines:

1. **Normal Teardown Protocol**:

   Once all tests are green and changes are committed to `feature/<task-id>`:

   a. The branch is preserved in Git refs.

   b. `git worktree remove --force` is called on the directory.

   c. Emits completion notification: `"Task <task-id> completed successfully on branch feature/<task-id>"`.

2. **Startup Garbage Collector (`WorktreeOrphanPruner.java`)**:
   Executes asynchronously on every CLI boot:

   Scans `${REPO_ROOT}/.git/agent-worktrees/`.

   For each directory, checks the associated `TaskRunEntity` in H2.

   If the task is in a terminal state (`COMPLETED`, `FAILED`, `ABORTED`) or its `pid` is no longer active on the OS, automatically removes the folder and invokes `git worktree prune`.

Step 3.7: Verification, Test Suite & Quality Gates

Phase 4 must pass strict Level 1 unit and Level 2 process lifecycle tests before sign-off:

1. **Level 1 Unit & Contract Tests (`src/test/`)**:

   `GitWorktreeServiceTest`:

   Author a temporary Git repo in `@TempDir`.

   Call `allocateWorktree("task-101", "feat/isolated")`.

   Assert worktree directory exists with an isolated `HEAD` pointing to `refs/heads/feat/isolated`.

   Call `releaseWorktree("task-101")`; assert folder is deleted and `git worktree list` is clean.

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Verify `FrontendExecutionTools` uses isolated cache directories.

`DesktopNotificationServiceTest`:

Execute `notify()` against mock OS targets; assert zero uncaught exceptions and valid script formatting.

2. **Level 2 CLI & Background Process Integration Tests (`@QuarkusTest`, `@QuarkusMainTest`):**

`DetachedExecutionIntegrationTest`:

Launch `dev-agent feature "Mock Story" --detach --yes` against a mock LLM provider in an isolated test repo.

Assert the command returns exit code 0 in < 5 seconds.

Verify `TaskRunEntity` indicates `status = RUNNING` with an active OS PID.

`TaskAbortIntegrationTest`:

Spawn a simulated background task.

Call `dev-agent abort <task-id>` via `QuarkusMainLauncher`.

Verify the process terminates, the worktree is cleaned up, and H2 records `ABORTED`.

4. Phase 4 Definition of Done (DoD) Checklist

Markdown

```
[ ] Worktree Isolation: Tasks execute inside .git/agent-worktrees/<task-id>/, leaving
[ ] Dependency Symlinking: Root node_modules are automatically symlinked into worktrees, avo
[ ] Target Slicing: Maven builds enforce -Dproject.build.directory=target-agent, preventing target/ c
[ ] Frontend Cache Isolation: Jest/Vitest and TypeScript compiler caches write to segregated worktre
[ ] Non-Blocking CLI: dev-agent feature --detach returns the terminal shell in < 5 seconds with exit
[ ] Process Tracking: Background PIDs are saved in H2 and verified via ProcessHandle.of(pid).isPresen
[ ] Desktop Alerts: Native OS desktop notifications emit reliably upon task completion or failure.
[ ] Status Command: dev-agent status displays an ANSI table of active, completed, and orphaned back
[ ] Log Streaming: dev-agent logs --follow <task-id> streams background worker output to the termi
[ ] Abort Control: dev-agent abort terminates background workers cleanly and unlinks associated wo
[ ] Startup Scavenger: WorktreeOrphanPruner cleans orphaned worktrees and prunes Git references
[ ] Automated Test Suite: All Level 1 worktree tests and Level 2 process lifecycle tests pass in < 60 sec
```

Go ahead with phase 5

Phase 5: Contract-Driven Full-Stack Sync & Remote Quality Gates (Detailed Specification)

1. Architectural Scope & Boundary Definition

The primary objective of **Phase 5** is to bridge local implementation with remote continuous

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

In Phases 1–4, the agent verified code strictly within local boundaries (worktrees, isolated Maven build targets, and local frontend test runners). However, shipping features into production codebases introduces full-stack and remote challenges:

Contract Drift: Backend Quarkus REST endpoints, DTOs, and Panache entities drift from frontend TypeScript interfaces and API services unless synchronized automatically from OpenAPI schemas.

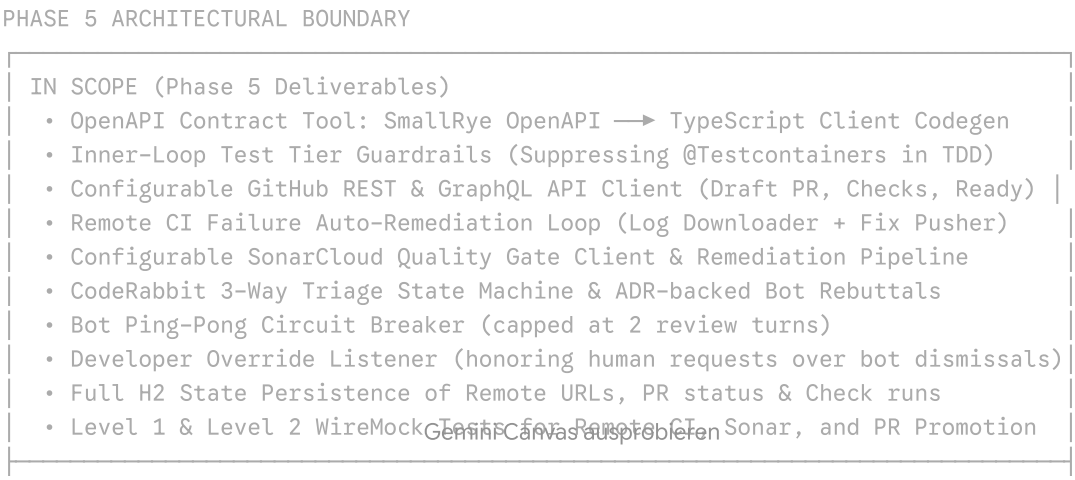
Test Tier Bloat: Inner TDD loops can accidentally execute slow integration tests (such as `@Testcontainers`, database spins, or multi-minute integration suites), severely degrading agent response times.

Remote CI Failures: Code passing locally can fail in remote GitHub Actions due to missing environment variables, stricter matrix compiler flags, or operating system differences.

Static Analysis & AI Review Gates: SonarCloud Quality Gates report code smells, vulnerabilities, or test coverage deficits, while automated review bots (`coderabbitai[bot]`) post inline PR feedback that requires automated fixes, architectural defenses, or dismissals.

Phase 5 equips the agent with:

1. **Contract Synchronization Tooling**: Automatically extracts Quarkus OpenAPI definitions and regenerates typed TypeScript clients via `FrontendExecutionTools`.
2. **Inner-Loop Test Tier Guardrails**: Enforces execution boundaries that prevent slow integration and container tests from executing during micro-TDD iterations.
3. **Declarative Remote GitHub REST/GraphQL Client**: Publishes feature branches, opens **Draft PRs**, monitors check-runs, and promotes PRs to **"Ready for Review"** upon green builds.
4. **Autonomous CI & SonarCloud Self-Healing**: Automatically downloads failing remote CI logs and Sonar issue reports, feeds them to the Coder model, commits fixes, and re-triggers remote validation.
5. **CodeRabbit 3-Way Triage Engine**: Triages bot comments using an automated state machine (`ACCEPT_AND_FIX`, `REBUT_WITH_ADR`, `DISMISS`), prevents bot-to-bot ping-pong loops, and honors developer overrides.



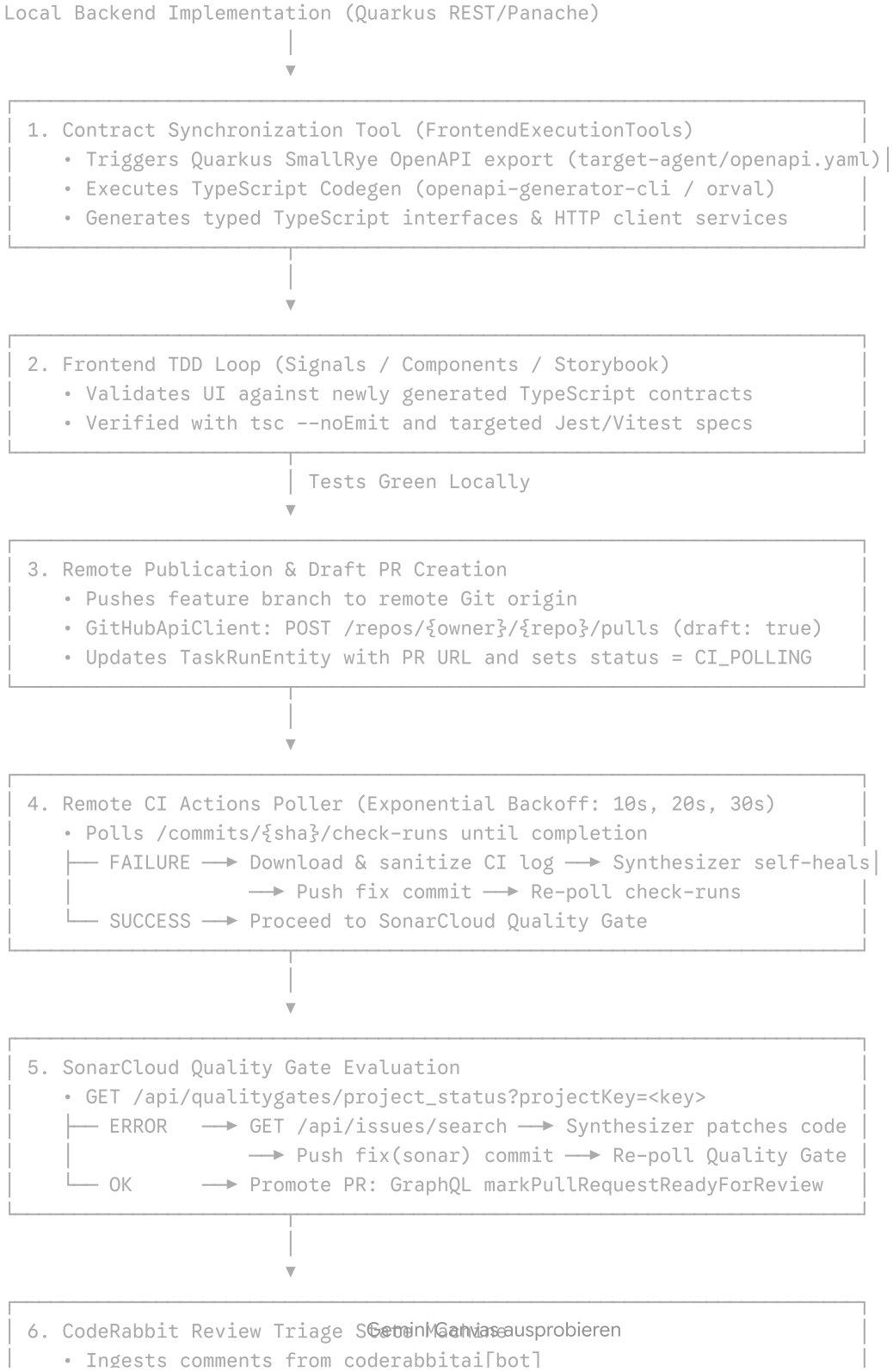
Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
  ✕ Dynamic Offline Ollama Failover Routing                ──► Phase 6
  ✕ Financial Hard Token & Dollar Budget Ceilings ($2.00)──► Phase 6
```

2. End-to-End Contract Sync & Remote Quality Gate Flow



Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
└── DISMISS        ──→ Reject hallucination or stylistic nitpick
```

3. Detailed Step-by-Step Implementation Breakdown

Step 5.1: Contract-Driven API Synchronization (`OpenApiContractSyncTool`)

To eliminate manual duplication and contract drift between Quarkus backend DTOs and frontend web models, implement contract synchronization in `FrontendExecutionTools` :

1. **Quarkus OpenAPI Specification Generation:**

   Leverages SmallRye OpenAPI configured in Quarkus (`quarkus-smallrye-openapi`).

   Triggers OpenAPI generation during compile checks:

   Bash

   ```
   ./mvnw quarkus:generate-code -Dproject.build.directory=target-agent
   ```

   SmallRye emits the schema to `target-agent/generated-resources/openapi.yaml` (or `target-agent/classes/META-INF/openapi.yaml`).

2. **TypeScript Client Generation:**
   Implement `generateTypeScriptClient()` as an `@ApplicationScoped` `@Tool` method:

   Java

   ```
   @Tool("Regenerates TypeScript models and API client services from the latest Qua
   public String generateTypeScriptClient() {
       Path repoRoot = workspaceResolver.getProjectRoot();
       Path openApiFile = repoRoot.resolve("target-agent/generated-resources/openap

       if (!Files.exists(openApiFile)) {
           return "FAILED: OpenAPI specification file not found. Ensure backend com
       }

       // Auto-detect code generator: orval vs openapi-generator-cli
       Path frontendDir = repoRoot.resolve(agentSettings.frontend().rootDir());
       boolean hasOrval = Files.exists(frontendDir.resolve("orval.config.js"))
                       || Files.exists(frontendDir.resolve("orval.config.ts"));

       List<String> command = hasOrval
           ? List.of("npx", "orval", "--config", "orval.config.ts")
           : List.of("npx", "@openapitools/openapi-generator-cli", "generate",
                     "-i", openApiFile.toString(),
                     "-g", "typescript-fetch",
   ```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
            ProcessBuilder pb = new ProcessBuilder(command).directory(frontendDir.to
            Process process = pb.start();
            if (!process.waitFor(45, TimeUnit.SECONDS) || process.exitValue() != 0)
                String err = new String(process.getErrorStream().readAllBytes(), Sta
                return "TypeScript client generation failed: " + err;
            }
            return "SUCCESS: TypeScript interfaces and API services regenerated from
        } catch (Exception e) {
            return "ERROR: Failed executing TypeScript code generator: " + e.getMess
        }
    }
```

3. **Execution Gate Ordering:**
   The workflow orchestrator enforces that `generateTypeScriptClient()` executes immediately after backend tests turn green, before the frontend TDD red-green cycle begins.

Step 5.2: Inner-Loop Test Tier Guardrails (`TestTierGuardrail`)

During rapid inner-loop TDD, LLMs often attempt to run slow integration suites (e.g., classes annotated with `@QuarkusIntegrationTest` , `@Testcontainers` , or full browser end-to-end tests). These can take 2–5 minutes per turn and disrupt the feedback cycle.

1. **Targeted Test Filter Enforcer:**
   Intercept test execution requests inside `BuildExecutionTools` :

   Parse the requested test class or pattern.

   If the user or LLM specifies a class ending in `IT.java` , `IntegrationTest.java` , or containing `@Testcontainers` , block execution with a corrective tool response:

   > *"EXECUTION REJECTED: Inner TDD loops must execute fast unit/mock tests only (<3 seconds). Avoid running integration tests (IT.java) or starting container fixtures during this phase. Target the focused unit test class instead."*

2. **Build Slicing Execution Properties:**
   Append Surefire unit-only exclusion patterns to all targeted Maven commands:

Bash

```
./mvnw test -Dtest=<TargetClassTest> -Dtest.category=unit -Dproject.build.direct
```

Step 5.3: Declarative GitHub REST & GraphQL Clients

Implement declarative remote clients using `quarkus-rest-client-reactive-jackson` . Coordinates (`owner` , `repo`) are inferred dynamically from `git remote get-url origin` or configured in `agents/dev-agent.yaml` :

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Java

```java
@RegisterRestClient(configKey = "github-api")
@Path("/repos/{owner}/{repo}")
@RegisterProvider(GitHubAuthHeaderFilter.class)
public interface GitHubApiClient {

    @POST
    @Path("/pulls")
    GitHubPullResponse createPullRequest(
        @PathParam("owner") String owner,
        @PathParam("repo") String repo,
        CreatePullRequestRequest request
    );

    @GET
    @Path("/commits/{ref}/check-runs")
    CheckRunsResponse getCheckRuns(
        @PathParam("owner") String owner,
        @PathParam("repo") String repo,
        @PathParam("ref") String ref
    );

    @GET
    @Path("/pulls/{pullNumber}/comments")
    List<GitHubReviewComment> getReviewComments(
        @PathParam("owner") String owner,
        @PathParam("repo") String repo,
        @PathParam("pullNumber") int pullNumber
    );

    @POST
    @Path("/pulls/{pullNumber}/comments/{commentId}/replies")
    void replyToComment(
        @PathParam("owner") String owner,
        @PathParam("repo") String repo,
        @PathParam("pullNumber") int pullNumber,
        @PathParam("commentId") long commentId,
        CommentReplyRequest body
    );
}
```

2. GraphQL Client for PR Promotion (`GitHubGraphQLClient.java`):
   Promoting a Draft PR to "Ready for Review" requires GitHub's GraphQL API:

Java

```java
@RegisterRestClient(configKey = "github-graphql")
@Path("/graphql")
@RegisterProvider(GitHubAuthHeaderFilter.class)
public interface GitHubGraphQLClient {
    @POST
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Mutation Payload:

GraphQL

```
mutation MarkPrReady($pullRequestId: ID!) {
  markPullRequestReadyForReview(input: {pullRequestId: $pullRequestId}) {
    pullRequest {
      isDraft
    }
  }
}
```

3. Draft PR Creation Mechanics:

   Prior to pushing, `GitCliService` pushes the isolated feature branch to the remote origin:

   Bash

   ```
   git push -u origin feature/<task-id>
   ```

   The client posts to `/pulls` with `draft: true`.

   The PR URL and Node ID are persisted in `TaskRunEntity` and projected to `./.agents/runs/<task-id>.md`.

Step 5.4: Automated Remote CI Remediation Engine

Remote GitHub Actions check-runs often uncover environment-specific errors (e.g., native compilation snags, stricter linter checks, or matrix JDK inconsistencies):

```
Poll Check-Runs (GET /commits/{sha}/check-runs)
                    |
        ┌───────────┴──────────┐
        ▼                      ▼
All Runs Completed?        In Progress?
        |                      |
        ├──► Yes               └──► Backoff Sleep (10s, 20s, 30s)
        ▼
Any Conclusion == FAILURE?
        |
        ├──► NO: CI is Green! ──► Proceed to SonarCloud Gate
        |
        └──► YES:
             1. Fetch failing check-run log URL
             2. Download raw log stream
             3. LogSanitizerService: Strip ANSI, extract [ERROR] & stacktraces
             4. Synthesizer LLM: Ingest failure, generate precision patch
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
7. Git push to origin feature/<task-id>
8. Reset CI backoff timer; re-poll check-runs (Max 3 retries)
```

1. **Log Acquisition & Compaction:**

   Downloads the raw execution log for the failing check-run via GitHub's API.

   Passes the raw log through `LogSanitizerService` : strips ANSI formatting, filters build runner noise, and extracts the primary error signature.

2. **Remediation Loop & Budgeting:**

   Submits the sanitized failure trace into `FullStackCodeSynthesizerAiService` along with the touched files.

   Enforces a hard ceiling of **3 remote self-healing attempts.** If checks remain red after 3 iterations, marks the task as `FAILED_CI_TIMEOUT` , leaves the Draft PR open for developer inspection, and terminates with code 1.

Step 5.5: SonarCloud Quality Gate Client & Auto-Remediation

If SonarCloud is enabled in `.agents/dev-agent.yaml` (`agent.sonar.enabled: true`):

1. **SonarCloud Declarative Client (`SonarCloudClient.java`):**

Java

```java
@RegisterRestClient(configKey = "sonar-api")
@Path("/api")
@RegisterProvider(SonarAuthHeaderFilter.class)
public interface SonarCloudClient {

    @GET
    @Path("/qualitygates/project_status")
    SonarQualityGateResponse getProjectStatus(
        @QueryParam("projectKey") String projectKey,
        @QueryParam("pullRequest") String pullRequestNumber
    );

    @GET
    @Path("/issues/search")
    SonarIssuesResponse searchIssues(
        @QueryParam("projectKeys") String projectKey,
        @QueryParam("pullRequest") String pullRequestNumber,
        @QueryParam("resolved") boolean resolved
    );
}
```

2. **Automated Sonar Issue Remediation:**

   Polls `getProjectStatus` until the analysis finishes.

Gemini Canvas ausprobieren

   If the Quality Gate returns `ERROR` :

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

For each issue, extracts: `ruleKey` (e.g., `java:S1144` unused private method), `component` (file path), `line`, and `message`.

Feeds issues to `FullStackCodeSynthesizerAiService` with explicit instructions:

```
"Fix the following SonarCloud rule violation: [%s]
at %s:%d - %s. Ensure all tests remain green."
```

Applies the patch, runs local tests, and pushes a commit:

Bash

```
git commit -m "fix(sonar): resolve rule java:S1144 in UserService.java"
git push origin feature/<task-id>
```

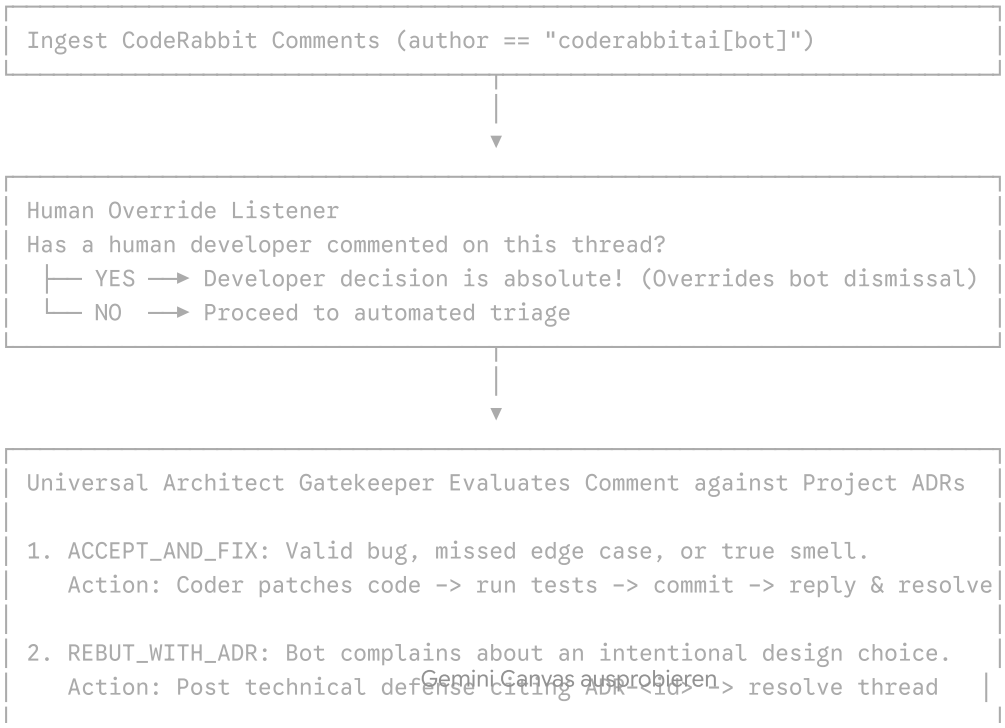
3. **Pull Request Promotion:**

   Once remote CI checks are green AND the SonarCloud Quality Gate reports `OK`, the agent invokes `GitHubGraphQLClient.markPullRequestReadyForReview`.

   Transitions the GitHub Pull Request out of Draft mode, notifying human reviewers that the change is tested, audited, and ready.

Step 5.6: CodeRabbit AI Review Comment Triage State Machine

Autonomous AI code reviews (e.g., CodeRabbit) can trigger unproductive "bot-to-bot arguments" unless disciplined by a structured state machine:



Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
Action: Post polite explanation -> resolve thread
```

1. The Triage Engine (`CodeRabbitTriageService.java`):

Java

```java
@ApplicationScoped
public class CodeRabbitTriageService {

    @Inject
    UniversalArchitectGatekeeperAiService gatekeeperAi;

    @Inject
    SemanticRuleRetriever ruleRetriever;

    @Inject
    GitHubApiClient gitHubClient;

    public void triageComment(GitHubReviewComment comment, String owner, String
        // 1. Guard against bot ping-pong: Cap discussion turns at 2
        if (getCommentThreadDepth(comment) >= 2) {
            return; // Terminate conversation loop
        }

        // 2. Fetch relevant ADR context for the reviewed file
        String adrContext = ruleRetriever.retrieveFormattedRules(
            comment.body(), List.of(comment.path())
        );

        // 3. Gatekeeper evaluates the comment
        CommentTriageVerdict verdict = gatekeeperAi.evaluateReviewComment(
            comment.body(), comment.diffHunk(), adrContext
        );

        switch (verdict.action()) {
            case ACCEPT_AND_FIX -> {
                applyPatchAndPush(comment, verdict.fixInstructions());
                gitHubClient.replyToComment(owner, repo, prNumber, comment.id(),
                    new CommentReplyRequest("Accepted and resolved in commit " +
            }
            case REBUT_WITH_ADR -> {
                String reply = String.format(
                    "Rebutted based on architectural invariant **%s**:\n\n%s",
                    verdict.citedAdrId(), verdict.rebuttalRationale()
                );
                gitHubClient.replyToComment(owner, repo, prNumber, comment.id(),
            }
            case DISMISS -> {
                gitHubClient.replyToComment(owner, repo, prNumber, comment.id(),
                    new CommentReplyRequest("Dismissed: " + verdict.dismissalRea
            }
        }
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

2. **Rebuttal with Project ADRs:**

   Example: CodeRabbit comments: *"Avoid returning raw Netty ByteBuf; use byte[] or String for readability."*

   The Gatekeeper retrieves `ADR-006: Zero-GC Streaming` and generates a technical rebuttal:

   > `"Rebutted based on architectural invariant ADR-006 (Zero-GC Netty Buffer Lifecycle): OmniDepot mandates zero-copy ByteBuf streaming through Netty event loops to prevent heap pressure during multi-gigabyte uploads. Converting to byte[] introduces GC allocations violating repository throughput invariants."`

3. **Bot Ping-Pong Circuit Breaker:**
   If a comment thread reaches **2 turns** between the Dev-Agent and CodeRabbit, the agent forcibly locks the thread from further automated responses to prevent infinite conversation loops and API spend.

4. **Human Developer Override Listener:**
   Before triaging, the service inspects all replies in the comment thread. If any reply originates from a non-bot user (`author.type != "Bot"`), the agent defers entirely to the human developer's instruction.

Step 5.7: H2 Database State & Metrics Synchronization

Track remote quality gate events inside the multi-tenant relational database and export them to Markdown journals:

1. **Entity Updates (`TaskRunEntity.java`):**

```java
@Column(name = "pr_number")
public Integer prNumber;

@Column(name = "pr_url")
public String prUrl;

@Column(name = "ci_status")
public String ciStatus; // SUCCESS, FAILURE, TIMEOUT

@Column(name = "sonar_status")
public String sonarStatus; // OK, ERROR, SKIPPED

@Column(name = "coderabbit_fixes")
public int coderabbitFixes;
```

2. **Markdown Journal Projection:**
   `StateSyncService` records remote URLs, CI check logs, Sonar metrics, and CodeRabbit triage summaries into `./agents/runs/<task_id>.md`:

Gemini Canvas durchsuchen

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
## Remote Quality Gates
- **Pull Request**: [#42 (Draft)](https://github.com/my-org/my-project/pull/42)
- **GitHub Actions CI**: 1 Failure detected ⟶ Auto-healed in commit `4a7b12` (
- **SonarCloud Quality Gate**: `OK` (0 Bugs, 0 Vulnerabilities, 0 Debt)
- **CodeRabbit Triage**:
  - *Issue 1 (app.component.ts:34)*: `ACCEPT_AND_FIX` (Applied missing OnDestroy unsu
  - *Issue 2 (BufferPool.java:88)*: `REBUT_WITH_ADR` (Cited ADR-006 Zero-GC Invariant
```

Step 5.8: Verification, Test Suite & Quality Gates

Phase 5 must satisfy comprehensive Level 1 unit and Level 2 remote simulation tests before sign-off:

1. **Level 1 Unit & Contract Tests (`src/test/`):**

   `OpenApiContractSyncToolTest` :

   Author a mock `openapi.yaml` in `target-agent/` .

   Invoke `generateTypeScriptClient()` ; verify `npx` command arguments and output file creation.

   `TestTierGuardrailTest` :

   Pass an integration test class (`S3StorageIT.java`) to `BuildExecutionTools` .

   Assert the tool rejects execution with an informative message rather than launching Docker containers.

   `CodeRabbitTriageServiceTest` :

   Mock a CodeRabbit comment suggesting an invalid pattern.

   Feed ADR-006 context; assert Gatekeeper outputs `REBUT_WITH_ADR` citing ADR-006.

   Verify thread depth ≥ 2 triggers early return (Ping-Pong breaker).

2. **Level 2 Remote API Contract Tests (`@QuarkusTest`):**

   `GitHubRemoteRemediationIntegrationTest` :

   WireMock stubs:

   `POST /repos/{owner}/{repo}/pulls` ⟶ Returns Draft PR #101.

   `GET /repos/{owner}/{repo}/commits/{sha}/check-runs` ⟶ Returns 1 failing check run with log URL.

   `POST /repos/{owner}/{repo}/git/refs` ⟶ Accepts push commit.

   `POST /graphql` ⟶ Returns `{ isDraft: false }` on PR promotion.

   Execute full remote remediation orchestrator loop.

   Assert agent downloads log, synthesizes fix, pushes commit, and promotes PR to Ready for Review.

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

`GET /api/qualitygates/project_status` → First call returns `ERROR`, second call returns `OK`.

`GET /api/issues/search` → Returns rule `java:S1144`.

Assert agent authors fix commit matching `fix(sonar): ...`, updates H2, and verifies green status.

4. Phase 5 Definition of Done (DoD) Checklist

Markdown

```
[ ] Contract Sync: Quarkus SmallRye OpenAPI export generates typed TypeScript interfa
[ ] Inner-Loop Slicing: TestTierGuardrail blocks slow integration tests (*IT.java) and (
[ ] GitHub Integration: Agent pushes feature branches and creates Draft PRs (draft: true) via declara
[ ] Remote CI Poller: Polls GitHub Actions check-runs with exponential backoff until all jobs reach a t
[ ] CI Auto-Remediation: Automatically downloads failing CI logs, extracts sanitized stacktraces, pus
[ ] SonarCloud Gate: Evaluates Quality Gate status and automatically patches reported rule violation
[ ] PR Promotion: Successfully calls GitHub GraphQL markPullRequestReadyForReview when CI and Sor
[ ] CodeRabbit Triage: Classifies bot review comments into ACCEPT_AND_FIX
[ ] ADR-Backed Defense: Technical rebuttals dynamically cite project ADRs
[ ] Ping-Pong Breaker: Caps bot-to-bot conversational discussions at 2 turns
[ ] Human Override: Comments by human developers immediately take precedence over bot dismissals
[ ] State & Metrics: PR URLs, CI check conclusions, and CodeRabbit actions persist to H2 and project t
[ ] Automated Test Suite: All Level 1 contract tests and Level 2 WireMock remote simulation tests pass
```

And finally phase 6

Phase 6: GraalVM Native Hardening, Dynamic Model Dispatch, Cost Ceilings & Web Diff UI (Detailed Specification)

1. Architectural Scope & Boundary Definition

The primary objective of **Phase 6** is to deliver the production-grade, hardened release of the standalone `quarkus-dev-agent` CLI. It transforms the feature-complete developer assistant into a globally installable, self-contained native executable (`~/.local/bin/dev-agent`) capable of **cold-booting in under 50 ms**, running completely offline via local Ollama failover, enforcing strict economic circuit breakers to prevent runaway LLM billing, and optionally serving a local browser-based GUI on `http://localhost:9999` for visual diff reviews and Storybook component inspection.

With Phase 6 delivered, the agent operates as a universal, zero-knowledge engine across all your Quarkus and frontend repositories.

```
PHASE 6 ARCHITECTURAL BOUNDARY
┌──────────────────────────────────────────────────────────────────────────┐
│ IN SCOPE (Phase 6 Final Deliverables)                                    │
│  • GraalVM / Mandrel AOT Native Image Compilation (< 50 ms cold boot)    │
│  • Native Reflection Registration & ArchUnit Native Compliance Rules     │
│  • Resource Ingestion Packaging (quarkus.native.resources.includes)      │
│  • Dynamic ModelRoutingService (CLI dispatch + Offline local Ollama fallback)
│  • Financial Hard Ceilings & Token Accumulators (Economic Circuit Breaker)
```

Gemini Canvas ausprobieren

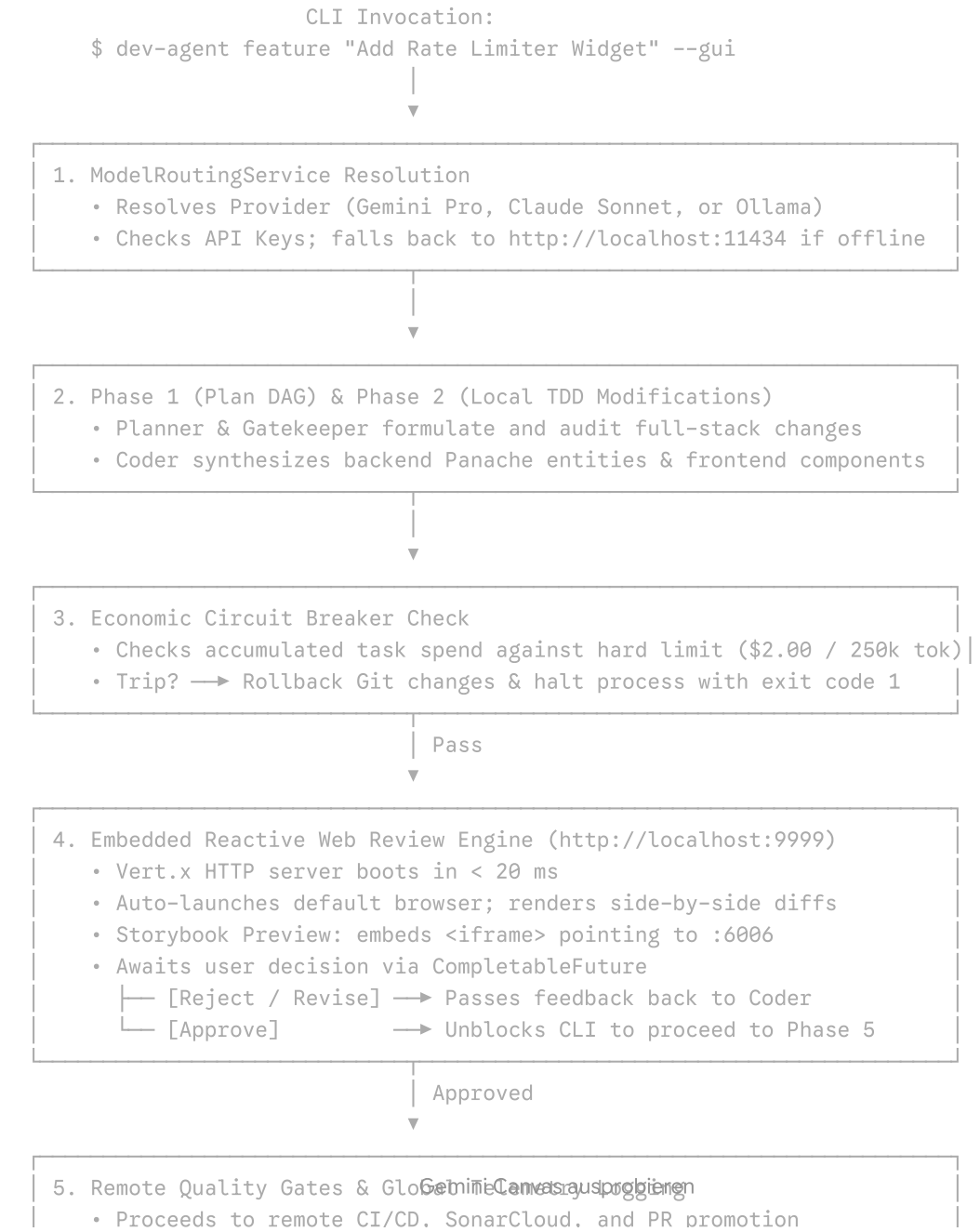
Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
  • Global Packaging & Installation Distribution (~/.local/bin/dev-agent)
  • Level 3 Native Binary Verification Suite (@QuarkusIntegrationTest)
```

```
ARCHITECTURAL COMPLETION (All 6 Phases Delivered)
 ✓ Phase 1: Standalone Full-Stack Walking Skeleton & Local TDD Loop
 ✓ Phase 2: Multi-Tenant Embedded H2 State & 3-Tier Hierarchical Memory
 ✓ Phase 3: Dynamic ADR Semantic RAG, Decision Caching & Loop Breakers
 ✓ Phase 4: Non-Blocking Workspaces, Worktrees & Detached Vert.x Workers
 ✓ Phase 5: Contract-Driven Full-Stack Sync & Remote Quality Gates
 ✓ Phase 6: Native Hardening, Model Routing, Budget Stops & Web Diff UI
```

2. Runtime Topology & GUI Review Flow



Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

3. Detailed Step-by-Step Implementation Breakdown

Step 6.1: GraalVM Native Image Optimization & ArchUnit Governance

To achieve near-instantaneous startup (<50 ms) without JVM warmup penalties, compile the CLI to a standalone native binary using Mandrel or GraalVM.

1. **Native Resource Ingestion Configuration (`application.yaml`):**
   Ensure default templates bundled in `src/main/resources/harness-templates/` and web GUI assets in `src/main/resources/web-ui/` are compiled into the native binary:

YAML

```yaml
quarkus:
  package:
    type: native
  native:
    container-build: true
    builder-image: quay.io/quarkus/ubi-quarkus-mandrel-builder-image:jdk-21
    resources:
      includes:
        - harness-templates/**
        - web-ui/**
```

2. **Reflection Metadata Registration:**
   Substrate VM strips unreferenced reflection metadata at build time. Every data transfer object, gatekeeper schema record, and Panache entity must be registered using `@RegisterForReflection` :

Java

```java
@RegisterForReflection
public record GateVerdict(
    boolean approved,
    String summary,
    List<String> violations,
    List<String> recommendations
) {}
```

3. **Mandatory ArchUnit Native Governance Rules (`NativeReflectionArchUnitTest.java`):**
   Prevent native runtime crashes caused by unannotated reflection classes or improper CDI tool scoping:

Java

```java
@AnalyzeClasses(packages = "org.quarkus.agent")
public class NativeReflectionArchUnitTest {
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
                .should().beDeclaredInClassesThat().areAnnotatedWith(ApplicationScop

    @ArchTest
    public static final ArchRule recordsAndEntitiesMustRegisterReflection =
        classes()
            .that().resideInAnyPackage("..model..", "..entity..", "..remote..",
            .and().areNotInterfaces()
            .and().areNotAnonymousClasses()
            .should().beAnnotatedWith(RegisterForReflection.class);
}
```

Step 6.2: Dynamic Model Routing & Local Ollama Failover (`ModelRoutingService`)

A developer tool must not fail if an engineer is on an airplane, working in a restricted corporate air-gapped subnet, or temporarily out of cloud API credits.

1. **Routing Mechanics:**
   `ModelRoutingService` bridges build-time LangChain4j CDI beans with runtime CLI options (`--offline` , `--model=<name>`) and the environment cascade:

   Tier 1 (Remote Cloud Models): Gemini 1.5 Pro (Planner), Gemini 1.5 Flash (Gatekeeper), Claude 3.5 Sonnet (Coder).

   Tier 2 (Air-Gapped / Offline Local Fallback): Ollama running on `http://localhost:11434` (e.g., `qwen2.5-coder:7b` or `deepseek-coder:6.7b`).

2. **The Routing Implementation:**

Java

```
@ApplicationScoped
public class ModelRoutingService {

    @ConfigProperty(name = "agent.model.default-provider", defaultValue = "gemin
    String defaultProvider;

    public boolean isOfflineMode(boolean cliOfflineFlag) {
        if (cliOfflineFlag) return true;
        String geminiKey = System.getenv("GEMINI_API_KEY");
        String openAiKey = System.getenv("OPENAI_API_KEY");
        return (geminiKey == null || geminiKey.isBlank()) && (openAiKey == null
    }

    public AiClientCoordinates resolveCoordinates(String role, boolean cliOfflin
        if (isOfflineMode(cliOfflineFlag)) {
            return new AiClientCoordinates("ollama", "http://localhost:11434", "
        }

        return switch (role) {
            case "planner" -> new AiClientCoordinates("gemini", null, "gemini-1.
            case "gatekeeper" -> new AiClientCoordinates("gemini", null, "gemini
            case "coder" -> new AiClientCoordinates("openai-compatible", "https:
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
    @RegisterForReflection
    public record AiClientCoordinates(String provider, String endpoint, String m
}
```

3. **Graceful Status Reporting:**
   When offline mode is active, the CLI prints:
   `[MODEL DISPATCH] Operating in offline mode. Routing requests to local Ollama (qwen2.5-coder:7b).`

Step 6.3: Financial Hard Ceilings & Token Accumulators (`EconomicCircuitBreaker`)

Unattended autonomous loops can rack up significant bills if an LLM gets trapped in a cycle or attempts dozens of API retries.

1. **Budget Thresholds:**
   Configured in `~/.config/dev-agent/application.yaml` or overridden per project:

YAML

```yaml
agent:
  budget:
    max-task-tokens: 250000
    max-task-cost-usd: 2.00
    max-daily-spend-usd: 15.00
```

2. **Economic Guard Implementation (`EconomicCircuitBreaker.java`):**
   Intercepts every tool turn and LLM completion:

Java

```java
@ApplicationScoped
public class EconomicCircuitBreaker {

    @Inject
    GitCliService gitCliService;

    private final AtomicInteger taskTokensAccumulator = new AtomicInteger(0);
    private final AtomicReference<BigDecimal> taskSpendAccumulator =
        new AtomicReference<>(BigDecimal.ZERO);

    private static final int MAX_TASK_TOKENS = 250_000;
    private static final BigDecimal MAX_TASK_COST = new BigDecimal("2.00");

    public void trackUsage(int promptTokens, int completionTokens, BigDecimal st
        int total = taskTokensAccumulator.addAndGet(promptTokens + completionTok
        BigDecimal currentSpend = taskSpendAccumulator.accumulateAndGet(stepCost

        if (total > MAX_TASK_TOKENS || currentSpend.compareTo(MAX_TASK_COST) > 0
            tripCircuitBreaker(total, currentSpend);
        }
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
                System.err.printf(
                    "\n[ECONOMIC CIRCUIT BREAKER TRIPPED] Task exceeded budget limits (T
                    totalTokens, currentSpend.toPlainString()
                );

                // Revert workspace to snapshot to prevent leaving half-baked changes
                gitCliService.rollbackToSnapshot();

                System.exit(1);
            }
        }
```

Step 6.4: Embedded Reactive Web Review Engine (`WebReviewServer` for `--gui`)

Reviewing large multi-file code diffs, OpenAPI specs, and frontend UI components in terminal text can be cumbersome. The `--gui` mode boots a lightweight local web dashboard on `http://localhost:9999` using Quarkus Vert.x HTTP.

```
CLI Execution: $ dev-agent feature "Add Auth Health Check" --gui
                                  │
                                  ▼
 1. Phase 1 Plan & Phase 2 TDD generate modifications
 2. WebReviewServer boots on http://localhost:9999 (< 20 ms)
 3. Launches default OS browser (xdg-open / open / cmd.exe)
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────┐
│ Local Review Dashboard (http://localhost:9999)           │
│  • Plan DAG with impacted modules                        │
│  • Side-by-Side Syntax-Highlighted Diffs (diff2html)     │
│  • Embedded Storybook Preview <iframe> (localhost:6006)  │
│  • Real-Time Token Spend & Cost Telemetry Tracker        │
│  • [Approve Step]   [Request Changes]   [Abort Task]     │
└──────────────────────────────────────────────────────────┘
                                  │ User clicks [Approve Step]
                                  ▼
POST /api/review/decision { approved: true }
CompletableFuture completes; Web server shuts down; CLI continues
```

1. **Vert.x Route Controller (`WebReviewServer.java`):**

Java

```
@ApplicationScoped
public class WebReviewServer {

    private CompletableFuture<ReviewDecision> activeDecisionFuture;
    private ReviewContext currentContext;
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
            this.activeDecisionFuture = new CompletableFuture<>();
            return activeDecisionFuture;
        }

        @Route(path = "/api/review/context", methods = HttpMethod.GET)
        void getContext(RoutingExchange ex) {
            ex.ok(currentContext);
        }

        @Route(path = "/api/review/decision", methods = HttpMethod.POST)
        void submitDecision(RoutingExchange ex) {
            ReviewDecision decision = ex.body().asPojo(ReviewDecision.class);
            if (activeDecisionFuture != null && !activeDecisionFuture.isDone()) {
                activeDecisionFuture.complete(decision);
            }
            ex.ok().end("{\"status\":\"ACKNOWLEDGED\"}");
        }

        @RegisterForReflection
        public record ReviewContext(String taskId, String userStory, String planJson

        @RegisterForReflection
        public record ReviewDecision(boolean approved, String userFeedback) {}
    }
```

2. Self-Contained Web UI Assets (`src/main/resources/web-ui/`):

   Packaged directly inside the native binary (<150 KB, zero external CDNs):

   - `index.html` : Dark-first styling (Carbon surfaces, Reactive Cyan accents `#00E5FF`).
   - `diff2html.min.js` : Renders side-by-side colorized diffs with character-level additions and deletions.
   - `storybook-preview.js` : If a Storybook server is running (`http://localhost:6006`), embeds an `<iframe>` pointing to the component story so the developer can visually test the new UI widget.
   - `app.js` : Fetches `/api/review/context` and posts the decision back to `/api/review/decision` .

Step 6.5: Lifetime Telemetry & Repository Metrics (`dev-agent metrics`)

Implement the `dev-agent metrics` command to surface aggregated performance and savings data across all registered projects:

Java

```
@Command(name = "metrics", description = "Display lifetime telemetry, token usage, an
public class MetricsCommand implements Runnable {

    @Inject
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
        String currentProject = projectResolver.resolveProjectId();

        long totalTasks = TaskRunEntity.count("projectId", currentProject);
        long completedTasks = TaskRunEntity.count("projectId = ?1 and status = ?2", c

        Double totalSpend = MetricLogEntity.find(
            "select sum(costUsd) from MetricLogEntity m where m.taskRunId in " +
            "(select t.id from TaskRunEntity t where t.projectId = ?1)", currentProje
        ).firstResult();

        Long totalTokens = MetricLogEntity.find(
            "select sum(promptTokens + completionTokens) from MetricLogEntity m where
            "(select t.id from TaskRunEntity t where t.projectId = ?1)", currentProje
        ).firstResult();

        Long cacheHits = MetricLogEntity.count(
            "cacheHit = true and taskRunId in (select t.id from TaskRunEntity t where
        );

        System.out.println(Ansi.ansi().bold().a("\n--- DEV-AGENT LIFETIME REPOSITORY
        System.out.printf("  Project ID:          %s\n", currentProject);
        System.out.printf("  Completed Tasks:     %d / %d (Success Rate: %.1f%%)\n",
            completedTasks, totalTasks, (totalTasks > 0 ? (completedTasks * 100.0 / t
        System.out.printf("  Total Token Volume:  %,d tokens\n", totalTokens != null
        System.out.printf("  Total Est. Spend:    $%.4f USD\n", totalSpend != null ?
        System.out.printf("  Cached Decisions:    %d gate reviews saved (0 ms / 0 tok
    }
}
```

Step 6.6: Standalone Distribution & Global Installation

Package and install the compiled native binary so it can be invoked from any terminal window:

1. **Compilation Command:**

```bash
mvn clean package -Dnative -DskipTests
```

Outputs an optimized, standalone binary: `target/quarkus-dev-agent-runner` (~45 MB ELF/Mach-O executable).

2. **Global Installation Script (`install.sh`):**

```bash
#!/usr/bin/env bash
set -euo pipefail

INSTALL_DIR="${HOME}/.local/bin"
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
echo "✓ Installed dev-agent successfully to ${INSTALL_DIR}/dev-agent"
echo "  Ensure ${INSTALL_DIR} is in your PATH."
```

3. Multi-Project Dogfooding Verification:

   Target A (OmniDepot - "Customer Zero"): Run `dev-agent feature "Add CAS S3 chunk metrics"` to verify Maven multi-module reactor isolation, Netty zero-GC invariants, and H2 database persistence.

   Target B (Standalone Angular/Quarkus SPA): Run `dev-agent feature "Add user profile avatar card"` to verify single-module detection, OpenAPI client generation, Storybook story authoring, and TypeScript checks.

Step 6.7: Level 3 Native Binary Verification & Quality Gates

Enforce strict performance and integration assertions against the compiled native binary:

Java

```java
@QuarkusIntegrationTest
public class DevAgentNativeIT {

    @Test
    public void verifyColdBootStartupTimeUnder50ms() throws IOException, InterruptedE
        String binaryPath = System.getProperty("quarkus.native.executable");
        assertNotNull(binaryPath, "Native executable binary must exist.");

        long start = System.nanoTime();
        Process process = new ProcessBuilder(binaryPath, "--version")
            .redirectOutput(ProcessBuilder.Redirect.DISCARD)
            .start();
        int exitCode = process.waitFor();
        long durationMs = (System.nanoTime() - start) / 1_000_000;

        assertEquals(0, exitCode);
        assertTrue(durationMs < 50, "Native binary must boot in under 50ms. Actual: "
    }

    @Test
    public void verifyOfflineOllamaRoutingFlag() throws IOException, InterruptedExcep
        String binaryPath = System.getProperty("quarkus.native.executable");

        Process process = new ProcessBuilder(binaryPath, "feature", "Test Offline", "
            .redirectErrorStream(true)
            .start();
        String output = new String(process.getInputStream().readAllBytes(), StandardC
        int exitCode = process.waitFor();

        assertEquals(0, exitCode);
        assertTrue(output.contains("Operating in offline mode"));
    }
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
        String binaryPath = System.getProperty("quarkus.native.executable");

        Process process = new ProcessBuilder(binaryPath, "init")
            .directory(tempDir.toFile())
            .start();
        int exitCode = process.waitFor();

        assertEquals(0, exitCode);
        assertTrue(Files.exists(tempDir.resolve(".agents/rules/quarkus-base-invariant
        assertTrue(Files.exists(tempDir.resolve(".agents/workflows/plan.md")));
    }
}
```

4. Phase 6 Definition of Done (DoD) Checklist

Markdown

```
[ ] Native Binary Build: Mandrel/GraalVM builds target/quarkus-dev-agent-runner clean
[ ] Cold Boot Performance: Running 'dev-agent --version' executes in < 50 ms.
[ ] Idle Memory Footprint: Native process resident set size (RSS) stays under 80 MB.
[ ] Reflection Completeness: NativeReflectionArchUnitTest passes; zero native reflect
[ ] Resource Inclusion: All embedded rules and workflows extract cleanly during nativ
[ ] Model Routing: ModelRoutingService dispatches to cloud models or local Ollama (qw
[ ] Offline Failover: Running with '--offline' executes full local TDD against Ollama
[ ] Economic Circuit Breaker: Tasks exceeding $2.00 or 250,000 tokens trip immediatel
[ ] Embedded Web Review: Passing '--gui' serves the review dashboard on http://localh
[ ] Interactive UI Decision: Browser decision signals CompletableFuture and unblocks
[ ] Storybook Integration: UI review dashboard embeds <iframe> pointing to component
[ ] Telemetry Command: 'dev-agent metrics' reports accurate spend, token counts, and
[ ] Global Install: Executable installs to ~/.local/bin/dev-agent and runs from any d
[ ] Level 3 Native Tests: DevAgentNativeIT passes 100% of native image assertions.
```

5. Master Architecture & Implementation Completion Matrix

With Phase 6 specified, the end-to-end evolutionary engineering plan for `quarkus-dev-agent` is complete across all six phases:

Phase	Strategic Deliverable	Core Invariants & Guardrails	Key Technologi
Phase 1	**Standalone Full-Stack Walking Skeleton**	Workspace Auto-Detection, Log Sanitizer, Sandboxing, Ephemeral Git Rollback	`quarkus-picc` `langchain4j` `jansi`
Phase 2	**Multi-Tenant State & Memory Hardening**	Embedded H2 SSOT, Markdown Projection, 3-Tier Memory, Secret Redaction	`quarkus-jdbc` `hibernate-or` `panache`
Phase 3	**Dynamic ADR RAG &**	Dynamic Rule Discovery	`all-minilm-l`

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Phase	Strategic Deliverable	Core Invariants & Guardrails	Key Technologi
Phase 4	**Non-Blocking Workspaces & Workers**	Git Worktrees, Build Directory Slicing, Detached Vert.x Workers, Desktop Alerts	`quarkus-vert` Worktrees
Phase 5	**Contract Sync & Remote Quality Gates**	OpenAPI → TS Codegen, Draft PRs, CI/Sonar Remediation, CodeRabbit Triage	`smallrye-ope` `rest-client-reactive`
Phase 6	**Native Hardening, Mor**	GraalVM Native Image	Mandrel Native,

Challenge the current architecture and look for gaps and error-prone decisions - I need this to be super tight and open for extension.
For now we can stick with mvn/npm, but allow extension to other build tools such as gradle, pnpm or yarn later

A rigorous, production-grade architectural audit requires looking past "happy path" assumptions and targeting runtime mechanics, OS-level edge cases, GraalVM native constraints, and software design principles (particularly the **Open-Closed Principle (OCP)** for build tools).

While the overall vision is cohesive, the current blueprints contain **5 critical operational blind spots**, **3 architectural coupling bottlenecks**, and an urgent need to formalize the **Build System SPI** so that adding Gradle, pnpm, yarn, or bun later requires zero core engine rewrites.

Part 1: Fatal Flaws & Operational Blind Spots (The Showstoppers)

1. The GraalVM Native JNI Trap: In-Process ONNX Embeddings

The Flaw: Phase 3 proposes using `dev.langchain4j:langchain4j-embeddings-all-minilm-l6-v2` with an in-process ONNX runtime inside a GraalVM native binary (`dev-agent-runner`).

Why It Breaks: The ONNX runtime relies on platform-specific C++ native libraries (`.so` , `.dylib` , `.dll`) loaded via JNI. GraalVM Substrate VM requires painstaking manual reflection and JNI configuration for native libraries. Cross-compiling or running on diverse developer workstations (x86_64 Linux, ARM64 macOS / Apple Silicon, Windows) will cause `UnsatisfiedLinkError` crashes at runtime in native mode.

Tightened Solution:

Do not bind the native binary to an in-process ONNX C-library.

Implement an abstract `RuleRetriever` SPI.

Use a deterministic **BM25 / Inverted Index Lexical Search** (pure Java, zero JNI, 100% native-safe) as the baseline local engine. It indexes markdown headings, code tags, and ADR numbers natively in <2 ms.

Offer semantic vector embeddings via standard remote embedding APIs

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

2. The Worktree `node_modules` Symlink Failure

The Flaw: Phase 4 proposes symlinking `node_modules` from the host repository into `.git/agent-worktrees/<task-id>/frontend/node_modules` to save disk space and avoid re-running package installs.

Why It Breaks:

1. *Windows / WSL2*: On Windows, creating directory symbolic links requires Administrator elevation unless Windows Developer Mode is explicitly turned on. `Files.createSymbolicLink` throws `FileSystemException: A required privilege is not held by the client`.
2. *Mutation Contamination*: If the agent runs `npm test` or a self-healing step that executes `npm install --save <dep>` or updates package metadata, it directly modifies the host workstation's `node_modules` and `package-lock.json`, violating worktree isolation.

Tightened Solution:

Never create raw directory symlinks across worktree boundaries.

Rely on package-manager-native caching:

For `npm`: Share `~/.npm` cache via `--cache`.

Future `pnpm`: Inherently immune because `pnpm` links from a global CAS store (`~/.local/share/pnpm/store`).

For the local worktree: If the feature modifies `package.json`, trigger an isolated lightweight install. If dependencies are untouched, configure test runners (Jest, Vitest) via configuration flags to resolve module directories from the parent root if absent locally.

3. The Compile-Time OpenAPI Generation Fallacy

The Flaw: Phase 5 assumes that running `./mvnw quarkus:generate-code -Dproject.build.directory=target-agent` will automatically produce `target-agent/generated-resources/openapi.yaml`.

Why It Breaks: Quarkus's `generate-code` goal generates Java source files from schemas (e.g., gRPC, Avro, OpenAPI client generators). It does **not** export the server's SmallRye OpenAPI specification to a YAML file on disk by default! In standard Quarkus apps, the OpenAPI endpoint is served dynamically at runtime (`/q/openapi`) by the Vert.x HTTP server.

Tightened Solution:

The agent cannot assume `openapi.yaml` exists on disk.

Use a fallback resolution chain:

1. Check if `io.smallrye:smallrye-openapi-maven-plugin` is present and execute its `generate-schema` goal.
2. Or execute a headless bootstrap test: Run a lightweight test that starts Quarkus and curls `http://localhost:8081/q/openapi`, saving the output to `target-agent/openapi.yaml`.

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

4. Static Tool Registration vs. Headless / Backend-Only Projects

The Flaw: The cognitive synthesizer statically registers all tools on `@RegisterAiService`:

Java

```
@RegisterAiService(tools = { CodeWorkspaceTools.class, BuildExecutionTools.class
```

Why It Breaks: If a developer runs `dev-agent` on a pure Quarkus REST microservice, CLI application, or Kafka consumer with **no frontend**, the LLM still sees `FrontendExecutionTools` (e.g., `runFrontendTest`, `generateTypeScriptClient`). LLMs frequently hallucinate and attempt to invoke frontend tools when backend tasks stall, causing confusing error loops.

Tightened Solution:

Introduce a **Dynamic Composite Tool Proxy** (`ExecutionToolProxy`).

The proxy exposes a single set of generic tool contracts to the LLM (e.g., `executeTargetedTest`, `executeCompilationCheck`).

At runtime, the proxy inspects the detected project context. If the project lacks a frontend, calling frontend operations immediately returns a clear invariant message: `"System Context: Current project has no frontend subsystem. Focus strictly on backend source."`

5. Embedded H2 Concurrency & Native Socket Lock Contention

The Flaw: Configuring H2 with `AUTO_SERVER=TRUE` in a GraalVM native binary running across multiple concurrent processes (e.g., foreground status checks while a detached worker runs).

Why It Breaks: In GraalVM Native Image, H2's `AUTO_SERVER` starts an internal background TCP server thread. If native thread management encounters OS firewall/socket restrictions or port conflicts, the process locks or crashes with `Database already in use`.

Tightened Solution:

Configure H2 with file-locking rather than raw socket servers: `FILE_LOCK=SOCKET` or `FILE_LOCK=FS`, combined with `LOCK_TIMEOUT=10000` (10 seconds).

CLI status commands (`dev-agent status`) should perform non-blocking read-only connections using `ACCESS_MODE_DATA=r` to prevent lock collisions with active worker processes.

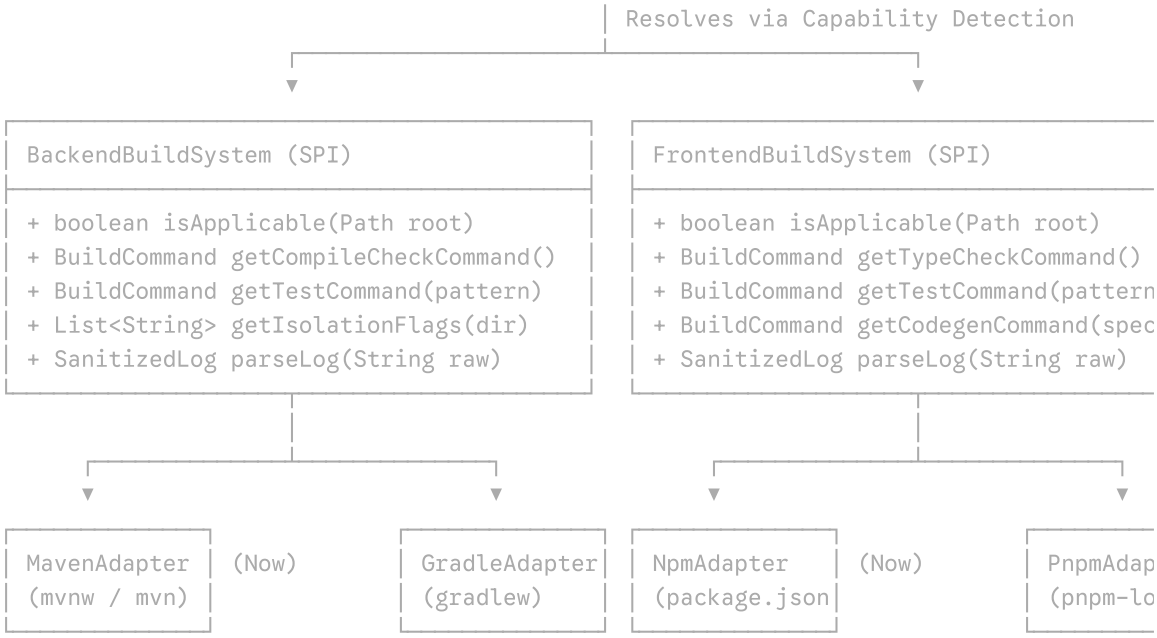
Part 2: Extensibility Architecture (Pluggable Build Systems)

To stick with **Maven** and **npm** today while making **Gradle**, **pnpm**, **yarn**, and **bun** zero-friction additions tomorrow, we must apply the **Strategy Pattern** via clean Quarkus SPIs.

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps



1. Backend Build SPI Contract (`BackendBuildSystem.java`)

Java

```java
public interface BackendBuildSystem {
    /** Priority order if multiple build descriptors exist */
    int priority();

    /** Checks if this build system governs the repository */
    boolean isApplicable(Path projectRoot);

    /** Resolves the build executable (e.g., ./mvnw or mvn) */
    String resolveExecutable(Path projectRoot);

    /** Generates the command to run an isolated compile check */
    List<String> buildCompileCommand(Path projectRoot, String modulePath, String targ

    /** Generates the command to run targeted unit tests */
    List<String> buildTestCommand(Path projectRoot, String modulePath, String testPat

    /** Strips build engine noise and isolates compiler/test errors */
    SanitizedLog sanitizeOutput(String rawStdout, String rawStderr);
}
```

Default Implementation: `MavenBuildSystem.java`

Detection: Checks for `pom.xml` and `./mvnw`.

Flags: Emits `-Dproject.build.directory=target-agent` and `-pl <module> -am`.

Sanitization: Isolates `[ERROR]` compilation lines and Surefire/Failsafe assertion traces

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Detection: Checks for `build.gradle`, `build.gradle.kts`, and `./gradlew`.

Flags: Emits `-PbuildDir=build-agent` and `--console=plain`.

Sanitization: Isolates Gradle compilation errors (`> Task :compileJava FAILED`) and JUnit test summaries.

2. Frontend Build SPI Contract (`FrontendBuildSystem.java`)

Java

```java
public interface FrontendBuildSystem {
    int priority();

    boolean isApplicable(Path frontendRoot);

    String getPackageManagerName();

    List<String> buildTypeCheckCommand(Path frontendRoot);

    List<String> buildTestCommand(Path frontendRoot, String testPattern);

    List<String> buildClientCodegenCommand(Path frontendRoot, Path openApiSchema);

    SanitizedLog sanitizeOutput(String rawStdout, String rawStderr);
}
```

Default Implementation: `NpmBuildSystem.java`

Detection: Checks for `package-lock.json` or fallback `package.json`.

Commands: `npm run test -- <pattern>`, `npx tsc --noEmit`.

Client Codegen: `npx @openapitools/openapi-generator-cli ...`

Future Extension Points: `PnpmBuildSystem.java` / `YarnBuildSystem.java`

Detection: Checks for `pnpm-lock.yaml` or `yarn.lock`.

Commands: `pnpm test <pattern>`, `pnpm exec tsc --noEmit`.

3. Polymorphic Resolution via CDI

At runtime, `BuildSystemResolver` queries Quarkus CDI instances:

Java

```java
@ApplicationScoped
public class BuildSystemResolver {
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
    Instance<FrontendBuildSystem> frontendSystems;

    public BackendBuildSystem resolveBackend(Path root) {
        return backendSystems.stream()
            .filter(b -> b.isApplicable(root))
            .max(Comparator.comparingInt(BackendBuildSystem::priority))
            .orElseThrow(() -> new IllegalStateException("No supported backend build
    }

    public Optional<FrontendBuildSystem> resolveFrontend(Path root) {
        return frontendSystems.stream()
            .filter(f -> f.isApplicable(root))
            .max(Comparator.comparingInt(FrontendBuildSystem::priority));
    }
}
```

Architectural Guarantee: To add Gradle or pnpm later, you write **one** isolated class implementing the interface. Zero lines of orchestrator, CLI, or tool logic are touched.

Part 3: Sandboxing, Security & Safety Hardening

1. Symlink Traversal Protection in `WorkspaceJailValidator`

Naive path checking (`path.startsWith(repoRoot)`) can be bypassed if an attacker or hallucinating LLM creates a symlink inside the workspace pointing to `/etc` or `~/.ssh` .

Hardened Rule: All file access must resolve canonical real paths:

Java

```
Path realRepoRoot = projectRoot.toRealPath();
Path realTarget = realRepoRoot.resolve(untrustedPath).toRealPath();
if (!realTarget.startsWith(realRepoRoot)) {
    throw new SecurityException("Jailbreak attempt: Symlink points outside proje
}
```

2. Shell Injection Defense

Hardened Rule: Never invoke shell interpreters (`sh -c` or `cmd.exe /c`) with concatenated strings.

All process executions must pass discrete array arguments directly to `ProcessBuilder` :

Java

```
// STRICTLY FORBIDDEN:
new ProcessBuilder("sh", "-c", "mvn test -Dtest=" + testName); // Vulnerable to
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

3. Gatekeeper Deadlock Prevention

The Risk: The Architect Gatekeeper rejects a plan with `approved: false`. The Planner refines the plan, but the Gatekeeper rejects it again. The agent loops indefinitely.

Hardened Escalation Rule:

Maximum gatekeeper review turns: **2 iterations**.

If turn 2 is rejected:

1. Halt autonomous execution.
2. Print the exact architectural violations to the terminal.
3. Prompt the developer: `[O]verride and proceed anyway | [A]bort task`.
4. If in headless/unattended mode (`--yes`), terminate with exit code 1 immediately and write reasons to the Markdown journal.

Part 4: The Tightened Architecture Matrix

Architectural Area	Original Proposal	Identified Vulnerability / Gap	Producti
Build Extensibility	Hardcoded `MavenExecutionTools` & `FrontendExecutionTools`	Tight coupling; adding Gradle or pnpm requires modifying core tools	Decoupl `Backenc` `Fronter`
Local RAG Retrieval	In-process ONNX (`all-minilm-l6-v2`) in native binary	JNI native library failure in GraalVM Substrate VM across OSs	Determir / inverted optional
Worktree Dependencies	Symlinking root `node_modules` into worktree	Windows privilege crashes; mutation leaks into host working tree	Use glob `--cache` cross-sy
OpenAPI Contract Sync	Expected `target-agent/openapi.yaml` from compile command	Quarkus serves OpenAPI dynamically; `openapi.yaml` is missing on disk	Dedicate plugin ch `/q/oper` → AST s
Tool Scoping	Frontend tools exposed to backend-only Quarkus projects	LLM hallucinates calling TypeScript tools on pure backend apps	Dynamic filters to project c
Database Locks	`AUTO_SERVER=TRUE` H2 socket server across native processes	Port conflicts and lock crashes between concurrent CLI processes	File-base `FILE_LO` `LOCK_TI` read-only

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Architectural Area	Original Proposal	Identified Vulnerability / Gap	Producti
Path Traversal	Normalized relative path validation	Fails against internal symlinks pointing to external targets	Enforce `toRealP` against `project`
Gatekeeper Audits	Unbounded plan-rejection loops	Infinite LLM-to-LLM refinement cycles	Capped interacti or clean

Part 5: Implementation Roadmap Adjustments

With these architectural invariants formalized, the implementation cycles adjust cleanly:

1. **Phase 1**: Implement `BackendBuildSystem` with `MavenBuildSystem`, and `FrontendBuildSystem` with `NpmBuildSystem`. Use pure array-based `ProcessBuilder` execution and canonical path jails.
2. **Phase 2**: Use file-locked H2 (`FILE_LOCK=SOCKET`) with project-isolated IDs.
3. **Phase 3**: Implement pure-Java BM25 lexical retrieval for ADRs first (zero-dependency native safety) with an interface ready for remote vector models.
4. **Phase 4**: Implement worktrees using package-manager caches instead of `node_modules` symlinks.
5. **Phase 5**: Implement contr
6. **Phase 6**: Package the Gra have been structurally eli

The polymorphic resolution via cdi would also allow to switch experts on femand right? So in a future version I could add Spring Boot back-end capabilities alongside Quarkus - is that correct?

Yes, exactly. That is one of the most powerful architectural benefits of using Quarkus CDI with polymorphic resolution (`Instance<T>`).

Because the agent CLI itself is simply a host runtime running outside the target workspace, **the framework of the target project does not have to match the framework of the agent.** The agent can remain a compiled Quarkus GraalVM native binary (<50 ms startup, low memory footprint), while acting as a specialized pair-programmer for **Spring Boot**, **Micronaut**, or **Quarkus** targets on demand.

To achieve this cleanly, you extend the SPI pattern from *Build Systems* to **Framework Experts**.

1. The Framework Expert SPI (`BackendFrameworkExpert`)

Just as `BackendBuildSystem` abstracts Maven vs. Gradle, a `BackendFrameworkExpert` SPI

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Java

```java
public interface BackendFrameworkExpert {
    /** Priority if multiple indicators match */
    int priority();

    /** Inspects POM/build.gradle/source to auto-detect the framework */
    boolean isApplicable(Path projectRoot);

    String getFrameworkName();

    /** Injects framework-specific system instructions & personas into the LLM */
    String getPersonaInstructions();

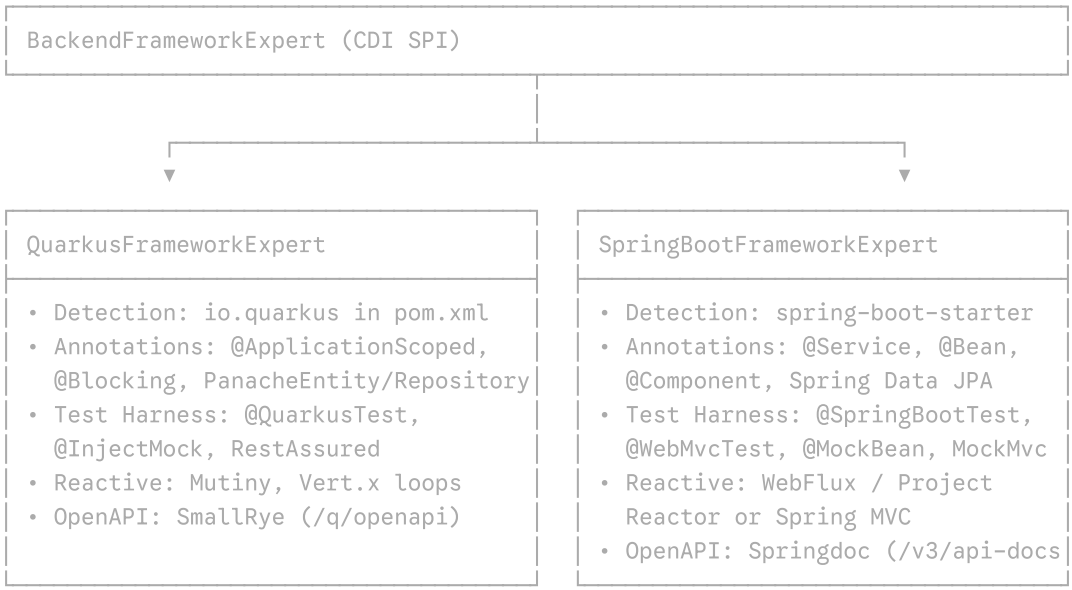
    /** Base rules: scoping, reactivity, transaction boundaries */
    List<String> getBaselineInvariants();

    /** Framework test conventions (e.g., unit test slices vs integration boots) */
    TestTierConvention getTestConvention();

    /** Strategy to extract OpenAPI / Swagger schema */
    OpenApiExtractionStrategy getOpenApiStrategy();
}
```

2. Concrete Experts: Quarkus vs. Spring Boot

You implement two `@ApplicationScoped` CDI beans that provide targeted domain knowledge:



Gemini Canvas ausprobieren

A. What the Quarkus Expert Injects

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

TDD Test Guardrails: Generates tests using `@QuarkusTest`, `@InjectMock`, and restricts inner micro-loops from booting `@QuarkusIntegrationTest` or `@Testcontainers`.

Contract Extraction: Directs the agent to SmallRye OpenAPI endpoints (`/q/openapi`) or `mvn quarkus:generate-code`.

B. What the Spring Boot Expert Injects

Persona & Rules: Instructs the LLM to use Spring conventions:

- Stereotypes: `@RestController`, `@Service`, `@Repository`, `@Configuration`.
- Persistence: Spring Data JPA (`JpaRepository<Entity, ID>`) rather than Panache.
- Validation: Jakarta Bean Validation (`@Valid`, `@NotNull`) with standard `MethodArgumentNotValidException` handlers.

TDD Test Guardrails:

- Knows to write sliced test classes: `@WebMvcTest(UserController.class)` for REST endpoints and `@DataJpaTest` for repositories (avoiding slow full `@SpringBootTest` context boots during inner-loop TDD).
- Uses `@MockBean` instead of `@InjectMock`, and `MockMvc` or `WebTestClient` instead of `RestAssured`.

Contract Extraction:

- Knows to look for `springdoc-openapi-starter-webmvc-ui` and probes `/v3/api-docs` or uses `springdoc-openapi-maven-plugin`.

3. Dynamic Resolution via CDI (`FrameworkExpertResolver`)

At runtime, the agent inspects the repository root using CDI instances:

Java

```java
@ApplicationScoped
public class FrameworkExpertResolver {

    @Inject
    Instance<BackendFrameworkExpert> experts;

    public BackendFrameworkExpert resolve(Path projectRoot, Optional<String> cliOverr
        // 1. Explicit CLI or config override (--framework=spring-boot)
        if (cliOverride.isPresent()) {
            return experts.stream()
                .filter(e -> e.getFrameworkName().equalsIgnoreCase(cliOverride.get())
                .findFirst()
                .orElseThrow(() -> new IllegalArgumentException("Unknown framework: "
        }

        // 2. Automatic capability detection from build descriptors
        return experts.stream()
            .filter(e -> e.isApplicable(projectRoot))
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
    }
}
```

4. Dynamic Persona Modulation in LangChain4j

Instead of hardcoding *"You are a Quarkus Engineer"* into `@SystemMessage`, the cognitive layer dynamically pulls the prompt from the resolved expert:

Java

```java
@ApplicationScoped
public class OrchestratorService {

    @Inject
    FrameworkExpertResolver expertResolver;

    @Inject
    FullStackPlannerAiService planner;

    public void plan(String userStory, Path repoRoot) {
        BackendFrameworkExpert expert = expertResolver.resolve(repoRoot, Optional.emp

        // Injected dynamically via LangChain4j @V template variables
        String frameworkGuidance = expert.getPersonaInstructions() + "\n" +
                                   String.join("\n", expert.getBaselineInvariants());

        String plan = planner.draftPlan(userStory, frameworkGuidance);
        // ...
    }
}
```

5. Why This Separation Is Ideal

Aspect	Static / Monolithic Agent	Polymorphic SPI Agent
Agent Executable	Must be re-written or branched for each target framework	Remains a single native binary (`~/.local/bin/dev-agent`)
Adding Spring Boot	Massive code refactor of tools, prompts, and tests	Implement `SpringBootFrameworkExpert` in one isolated package
Build & Tool Independence	Coupled to a single build tool and framework	Mix and match freely: e.g. **Quarkus + Maven** or **Spring Boot + Gradle**

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Aspect	Static / Monolithic Agent	Polymorphic SPI Agent
Token Efficiency	Prompt polluted with both Quarkus and Spring rules	Only the active framework's rules are loaded into context

Conclusion

So than I decide that we keep experts and personas framework independent, but we will start with Quarkus/Maven and Angular/npm

The intuition is 100% correct. `GradleBuildSystem`) with **Fra** `SpringBootFrameworkExpert` . Spring Boot, Micronaut, or any other backend framework in a future version requires zero changes to the core CLI, database state machine, worktree isolation, or git lifecycle.

This decision strikes the ideal balance between **long-term architectural openness** and **short-term execution speed**.

By keeping the cognitive personas and the execution engine framework-independent—while providing **Maven** and **npm** implementations configured for **Quarkus** and **Angular** out of the box—you avoid building a narrow, disposable tool without paying the complexity tax of writing multiple backend and frontend adapters on Day 1.

1. Framework-Independent Cognitive Personas

The core LLM roles do not need the words *"Quarkus"* or *"Angular"* hardcoded into their system instructions. Their cognitive responsibilities are universal to modern software engineering:

```
┌──────────────────────────────────────────────────────────────────┐
│ UNIVERSAL COGNITIVE PERSONAS (Zero Hardcoded Framework Words)    │
├──────────────────────────────────────────────────────────────────┤
│ 1. Principal System Planner (@RegisterAiService(modelName="planner"))  │
│    • Decomposes specs into a directed acyclic graph (DAG).       │
│    • Identifies contract boundaries (API schemas vs. client DTOs).  │
│    • Mandates Red-Green test-first sequence.                     │
│                                                                  │
│ 2. Invariant Gatekeeper (@RegisterAiService(modelName="architect-gate")) │
│    • Audits plans and diffs against dynamically retrieved project ADRs. │
│    • Evaluates architecture conformance at temperature: 0.0.     │
│    • Emits strict boolean verdicts and violation lists.          │
│                                                                  │
│ 3. Implementation Synthesizer (@RegisterAiService(modelName="coder"))  │
│    • Applies precision search-and-replace patches (`patchFile`).  │
│    • Interprets sanitized compiler errors and test failure traces.  │
│    • Self-heals code until targeted test execution passes green. │
└──────────────────────────────────────────────────────────────────┘
```

Gemini Canvas ausprobieren

How Quarkus and Angular Knowledge Is Injected

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

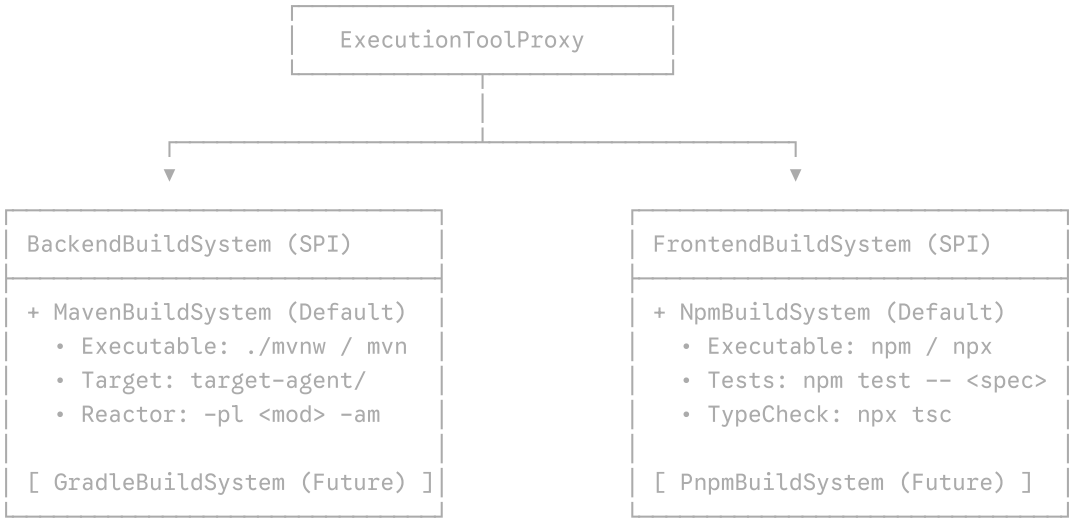
For the Backend: Ingests rules discovered in `.agents/rules/backend/` (CDI scoping, reactive Vert.x non-blocking loop rules, Hibernate Panache repository patterns).

For the Frontend: Ingests rules discovered in `.agents/rules/frontend/` (Angular Signals state management, standalone component isolation, Material Design 3 tokens, Storybook stories).

If tomorrow you run the agent in a repository containing Spring Boot or React rules, the cognitive services adapt instantly with **zero re-compilation of the agent binary**.

2. Concrete Build System Implementations: Maven & npm

The execution layer adheres strictly to the Strategy Pattern via CDI, implementing only the necessary adapters for your target stack:



A. `MavenBuildSystem` (Active Backend Strategy)

Detection: Resolves if `pom.xml` is present in the workspace root.

Isolation: Appends `-Dproject.build.directory=target-agent` to keep the agent's builds segregated from active `quarkus:dev` or IDE background compilations.

Reactor Resolution: If `<modules>` exist, derives `-pl <module-dir> -am` dynamically; if single-module, omits `-pl` cleanly.

Sanitization: Filters raw Maven noise down to Java compiler errors (`javac`) and JUnit/Surefire assertion failures.

B. `NpmBuildSystem` (Active Frontend Strategy)

Detection: Resolves if `package.json` exists in `frontend/` , `ui/` , or `src/main/webapp/` .

Test Slicing: Invokes `npm test -- <testPath>` (running Jest, Vitest, or Karma) with

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Sanitization: Compacts npm download noise, extracting only TypeScript error codes (`TS2304` , `TS2322`) and failed assertions.

3. Harness Templates: The Quarkus & Angular Starter Pack

Because the engine is generic, it relies on `dev-agent init` to unpack declarative knowledge into the target project. Your bundled templates in `src/main/resources/harness-templates/` provide the Quarkus and Angular baseline rules:

`rules/backend/01-quarkus-invariants.md`

Markdown

```
# Backend Architectural Invariants: Quarkus
1. CDI Scoping: Prefer `@ApplicationScoped` beans. Avoid `@RequestScoped` on event lo
2. Non-Blocking I/O: Never execute blocking operations on the Vert.x event loop. Use
3. Data Persistence: Use PanacheRepositoryBase<Entity, ID> rather than active-record
4. Build-Time Reflection: Register all external DTOs and JSON payloads with `@Registe
```

`rules/frontend/01-angular-invariants.md`

Markdown

```
# Frontend Architectural Invariants: Angular
1. Reactivity: Use Angular Signals (`signal()`, `computed()`) for component-level sta
2. Component Architecture: Every component must be a Standalone Component with explic
3. Design System: Use Material Design 3 design tokens (`--mdc-*`, Carbon surfaces, Re
4. Component Isolation: Reusable widgets must include an accompanying `*.stories.ts`
```

4. Contract Synchronization: Quarkus → Angular via npm

The contract synchronization workflow (Phase 5) leverages this exact combination cleanly without coupling the core CLI engine:

1. **Schema Generation (Quarkus / Maven)**:
   The agent triggers Maven to compile backend models and export the SmallRye OpenAPI document:

Bash

```
./mvnw compile -Dproject.build.directory=target-agent
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Bash

```
npx @openapitools/openapi-generator-cli generate \
  -i target-agent/generated-resources/openapi.yaml \
  -g typescript-angular \
  -o frontend/src/app/core/api
```

3. **Frontend TDD:**
   The Synthesizer imports the newly generated API models and services directly into the Angular component, authoring tests that pass `tsc --noEmit` and Jest/Karma checks.

Comparison: Coupled vs. Open Engine

Architectural Area	Hardcoded Framework Agent	Generalized Engine (Quarkus/Angular Initial)
System Prompts	*"You are an expert Quarkus and Angular engineer."*	*"You are a Principal Software Architect. Align designs to* `{rulesContext}` *."*
Build Executions	Direct calls to `mvn` and `npm` scattered in tools	Delegated to `BackendBuildSystem` and `FrontendBuildSystem` SPIs
Project Knowledge	Hardcoded OmniDepot ADR paths in Java code[cite: 1, 4]	Discovered in `.agents/rules/` and `docs/adr/` via local RAG/BM25[cite: 1, 4]
Future Adaptability	Rewrite tool code to support Spring Boot or Gradle	Add a single SPI class or drop new markdown rules into `.agents/`

This setup provides an immediate, highly tailored pair-programming experience for your Quarkus and Angular stack today, while preserving the architectural integrity of a universal developer agent for the future.

I like that, but require a thorough challenge

A ruthless architectural challenge is the best way to uncover subtle failure modes before writing code.

By committing to a **framework-independent core** that starts with **Quarkus/Maven** on the backend and **Angular/npm** on the frontend, the high-level trajectory is set. However, examining the physical runtime mechanics, build lifecycle internals, and compiler tooling reveals **6 major technical traps and race conditions** that will cause brittle behavior or test failures if left unaddressed.

Gemini Canvas ausprobieren

Challenge 1: The Build Isolation & Target Slicing Traps

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

The Trap: In Phases 1–3, the agent operates in the primary workspace root (`user.dir`) and sets `-Dproject.build.directory=target-agent` to avoid locking the IDE's `target/` directory.

The Failure: Standard `.gitignore` files contain `/target/` , **not** `/target-agent/` . As soon as the agent runs a compile or test step, hundreds of generated class files, bytecode caches, and Surefire XML reports appear in `git status --porcelain` as untracked files. When `GitCliService` runs its safety check or commits a patch, it either halts because the workspace is dirty or accidentally commits `target-agent/` artifacts to Git.

The Fix:

1. The agent's sandbox validator must automatically inject `target-agent/` and `.cache-agent/` into the local repository's `.git/info/exclude` (a private, uncommitted Git exclude file present in every Git repository).
2. In Phase 4 (Git worktrees), `target-agent` becomes redundant because `.git/agent-worktrees/<task-id>/` is already a physically isolated directory tree with its own clean `target/` folder.

1.2 The Hidden Container Spawner: Quarkus Dev Services

The Trap: Phase 5 states that `TestTierGuardrail` blocks integration tests by looking for `@Testcontainers` or `*IT.java` in inner micro-TDD loops.

The Failure: In Quarkus 3.x, standard unit/component tests annotated with `@QuarkusTest` **automatically spin up Testcontainers via Quarkus Dev Services** if a database (PostgreSQL), Kafka broker, or Keycloak extension is present in the POM and no live URL is configured. An agent attempting a 3-second TDD check on a `@QuarkusTest` will accidentally trigger a 30-second Docker container pull and boot, completely destroying inner-loop velocity.

The Fix: The `MavenBuildSystem` must explicitly pass flags that deactivate Dev Services during the rapid inner-loop TDD phase:

Bash

```
./mvnw test -Dtest=... -Dquarkus.devservices.enabled=false
```

This forces Quarkus to rely strictly on mock beans (`@InjectMock`) and in-memory stubs for inner-loop testing, only enabling Dev Services during full integration verification.

Challenge 2: Developer Interference & The `quarkus:dev` Clash

2.1 The Live-Reload Race Condition

The Trap: Enterprise Quarkus and Angular developers run `mvn quarkus:dev` in one terminal and `ng serve` in another for live continuous feedback.

The Failure: If the developer executes `dev-agent feature` in interactive mode (Phases 1–3) within `user.dir` , every single partial `patchFile` or intermediate syntax error generated during the agent's inner TDD loop instantly triggers Quarkus live-reload and Vite/Angular recompilation in the developer's background terminal. If the agent writes a

Gemini kann Fehler machen, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

The Fix:

The agent must treat **Phase 4 (Worktree Isolation) not as a late optimization, but as a foundational primitive.**

Even in Phase 1, if the agent operates in `user.dir`, it must verify whether a Quarkus dev mode lock (`target/quarkus-dev-mode`) is active and explicitly warn the developer, or mandate executing inside a scratch branch.

2.2 Uncommitted User Changes vs. Worktree Branching

The Trap: A developer has written 15 lines of a failing test or draft interface in their IDE, gets stuck, and runs: `dev-agent feature "Finish my auth test"`.

The Failure: If the agent checks out an isolated Git worktree from `HEAD` (`git worktree add ... -b feature/auth`), **the developer's uncommitted IDE modifications remain behind in the main workspace.** The agent in the worktree sees only the last committed state and has no idea what the developer just drafted in their IDE.

The Fix:
Implement an **Automatic Working-Tree Hydration Protocol**:

1. Inspect `git status --porcelain`.
2. If dirty files exist, execute an ephemeral stash: `git stash create`.
3. Inside the newly allocated worktree, apply that stash commit immediately: `git cherry-pick -n <stash-sha>` or `git checkout <stash-sha> -- .`.
4. The worktree now starts with the developer's exact uncommitted thoughts without altering their primary workspace Git index.

Challenge 3: Frontend Execution & Watch-Mode Deadlocks

3.1 The npm / Angular CLI Infinite Hang

The Trap: The `FrontendBuildSystem` runs targeted tests via `npm test -- <testPath>`.

The Failure: In standard Angular CLI configurations (`angular.json`), the default test builder (`@angular-devkit/build-angular:karma` or `jest`) launches in **interactive watch mode** (`"watch": true`) and opens a Chrome/Chromium window.
When the agent executes `npm test`, the process never terminates. It sits open waiting for file changes. The Java `ProcessBuilder` hangs forever waiting for EOF on the input stream, freezing the agent.

The Fix: The `NpmBuildSystem` strategy must enforce strict non-interactive, headless CLI overrides:

Bash

```
# For Angular CLI / Karma / Webpack:
npm test -- --watch=false --browsers=ChromeHeadless
```

Gemini Canvas ausprobieren

```
# For Vitest / Jest:
```

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Additionally, all process execution must be wrapped in a hard timeout watchdog (e.g., 60 seconds per test run) managed by Vert.x, killing any hanging process automatically.

3.2 Workspace & Monorepo Ambiguity (`angular.json`)

The Trap: The agent expects a single `package.json` inside `frontend/` .

The Failure: Angular projects are frequently structured as multi-project workspaces (with `projects/app` and `projects/ui-components` under a single root `angular.json` and a single root `package.json`). If the agent assumes the directory containing `package.json` is the frontend application root, it will write components into the monorepo root rather than into `projects/<app>/src/app/` .

The Fix: The `FrontendBuildSystem` must inspect `angular.json` if present. It must parse the default project (`defaultProject` or the first entry under `projects`) and resolve the actual source directory (`projects.<name>.sourceRoot` , typically `src` or `projects/<name>/src`).

Challenge 4: OpenAPI Contract Sync Mechanics (Quarkus SmallRye)

4.1 The Missing Compile-Time Schema

The Trap: The agent attempts to generate TypeScript models by invoking `./mvnw compile` and looking for `target-agent/generated-resources/openapi.yaml` .

The Failure: By default, Quarkus generates OpenAPI schemas **in memory at runtime** (`/q/openapi` via the Vert.x HTTP router). It does **not** persist `openapi.yaml` to disk during standard compilation unless explicit configuration properties are present in `application.properties` .

The Fix:
Do not expect the user's repository to have OpenAPI file-storing properties pre-configured. The agent's `MavenBuildSystem` must inject dynamic runtime properties into the compilation execution:

Bash

```
./mvnw compile -Dquarkus.smallrye-openapi.store-schema-file=true \
               -Dquarkus.smallrye-openapi.store-schema-directory=target-agent/op
```

This forces Quarkus to materialize the OpenAPI YAML directly into the agent's isolated build folder without modifying the project's permanent `application.properties` .

4.2 Angular 17+ Modern Client Architecture

The Trap: `openapi-generator-cli` with `-g typescript-angular` generates legacy Angular code (e.g., RxJS `BehaviorSubject` , `@NgModule` definitions, and modules expecting `HttpClientModule`).

Gemini Canvas ausprobieren

The Failure: Modern Angular 17+ projects use **Standalone Components**, functional HTTP

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

The Fix:
Configure the generator template properties inside `NpmBuildSystem`:

Bash

```
npx @openapitools/openapi-generator-cli generate \
  -i target-agent/openapi/openapi.yaml \
  -g typescript-angular \
  -o src/app/core/api \
  --additional-properties=ngVersion=18.0.0,providedInRoot=true,standalone=true
```

Challenge 5: Brittle Code Patching vs. Language Realities

5.1 Java Import Management & Indentation Drift

The Trap: `patchFile(path, searchBlock, replaceBlock)` relies on multi-line text replacement.

The Failure:

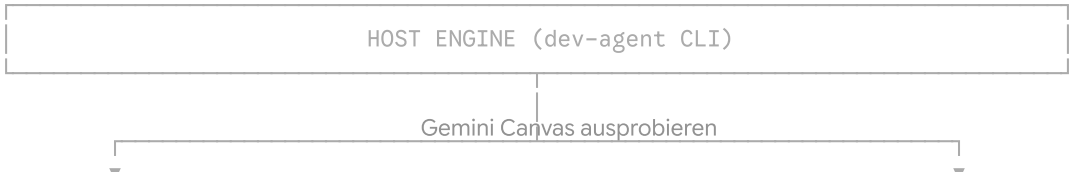
1. *Imports*: Adding a new dependency or annotation (e.g., `@ApplicationScoped` or `@InjectMock`) requires adding an `import` statement at line 5, while modifying logic at line 42. LLMs frequently fail when trying to replace the entire top half of a file just to inject one import, leading to duplicate or missing imports.
2. *Indentation & Formatting*: If the project uses Spotless, Google Java Format, or Prettier, slight tab-vs-space mismatches cause `patchFile` search matches to fail, triggering unnecessary self-healing retries.

The Fix:

1. **Dedicated Import Injection**: Add a specific tool `addImports(filePath, List<String> imports)`. This uses basic parsing to append imports cleanly into the existing package/import block without risking code corruption.
2. **Whitespace-Insensitive Block Matching**: If exact matching fails, tokenize both the search block and the target file by stripping leading/trailing whitespace per line before checking for a unique match.

Challenge 6: Solidifying the Extensibility SPI (Open-Closed Principle)

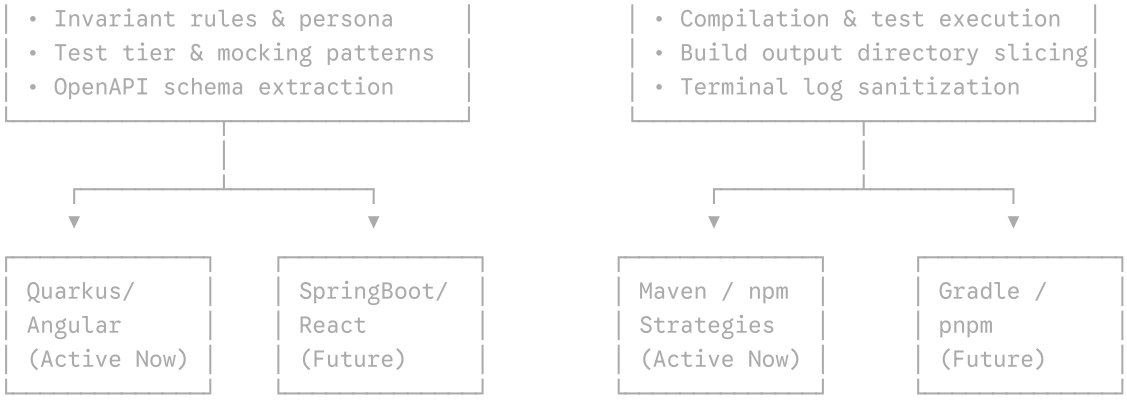
To guarantee that adding **Spring Boot**, **Gradle**, **React**, or **pnpm** in the future requires **zero core engine modifications**, formalize the architecture into two independent SPI axes:



Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps



The SPI Contracts

Java

```java
// Axis 1: Framework Conventions (Quarkus vs Spring Boot vs Angular)
public interface FrameworkConvention {
    boolean isApplicable(Path projectRoot);
    String getPersonaInstructions();
    List<String> getBaselineInvariants();
    List<String> getTestExclusionFlags(); // e.g. -Dquarkus.devservices.enabled=false
    List<String> getOpenApiExportFlags(Path targetDir);
}

// Axis 2: Build Tooling Execution (Maven vs Gradle, npm vs pnpm)
public interface BuildSystemStrategy {
    boolean isApplicable(Path workspaceRoot);
    List<String> getCompileCommand(String module, Path targetDir);
    List<String> getTargetedTestCommand(String testPattern, Path targetDir);
    SanitizedLog sanitize(String rawOutput);
}
```

Extensibility Guarantee:
When you add Spring Boot or Gradle later:

1. You do not touch the Picocli commands, H2 database, Git worktree isolation, or cognitive orchestrators.
2. You implement `GradleBuildSystemStrategy` and `SpringBootFrameworkConvention`.
3. Quarkus CDI automatically picks them up via `Instance<T>` and resolves them when running in a Gradle or Spring Boot directory.

Summary: The Tightened Invariants Checklist

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Identified Risk / Gap	Physical Failure Mode	Production Architectural Fix
`.gitignore` Pollution	`target-agent/` pollutes `git status` and dirty checks	Auto-inject `target-agent/` into local `.git/info/exclude` ; eliminate in worktree mode
Dev Services Containers	`@QuarkusTest` boots slow Docker containers in micro-TDD	Pass `-Dquarkus.devservices.enabled=fals` during targeted inner loops
`quarkus:dev` Interference	Agent code edits trigger live-reloads in developer terminal	Detect dev-mode locks; prioritize isolate Git worktrees over `user.dir`
Uncommitted User Changes	Worktree starts from `HEAD` , losing developer's draft work	Auto-stash uncommitted changes (`git stash create`) and apply into the worktree
npm Test Watch Hanging	Angular CLI default tests enter watch mode and hang CLI	Enforce `--watch=false --browsers=ChromeHeadless` and add process timeout guards
Monorepo Directory Drift	Components written to root instead of `projects/<app>/`	Parse `angular.json` for `sourceRoot` instead of assuming flat `frontend/`
Missing OpenAPI File	SmallRye serves OpenAPI in-memory; file missing on disk	Pass `-Dquarkus.smallrye-openapi.store-schema-file=true` dynamically during build
Angular 17+ Modernity	Generator outputs outdated `@NgModule` code	Pass `ngVersion=18.0.0,standalone=true` to OpenAPI generator properties
Brittle Search/Replace	Inde impo line	I would like to have OpenAPI spec generation und client building to be exactly configurable by the user in the agents yaml. We pre populate this with auto-detection but allow super easy overrides
Two-Axis SPI Separation	Framework knowledge entangled with build tool execution	Decouple into `FrameworkConvention` S and `BuildSystemStrategy` SPI

Here is the architectural specification and implementation design for a **declarative, auto-detected, and easily overridable OpenAPI contract synchronization system** in `quarkus-dev-agent` .

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

The entire contract lifecycle is governed by a dedicated `contract` block in `.agents/dev-agent.yaml`. It cleanly separates **Backend Schema Extraction** from **Frontend Client Building**, supporting structured generator templates, project npm scripts, or raw custom commands.

YAML

```yaml
agent:
  contract:
    # Master toggle: set to false to completely skip OpenAPI sync
    enabled: true

    # ==========================================
    # 1. BACKEND SCHEMA EXTRACTION
    # ==========================================
    spec:
      # Extraction strategy:
      #   - 'quarkus'    : Injects SmallRye build flags during compile (Default for Q
      #   - 'plugin'     : Invokes dedicated Maven/Gradle goal (e.g., smallrye-openap
      #   - 'live-url'   : Queries running server (e.g., http://localhost:8080/q/open
      #   - 'custom'     : Executes custom shell command provided in `command`
      strategy: quarkus

      # Output path where the OpenAPI spec is saved (YAML or JSON)
      output-file: "target-agent/openapi/openapi.yaml"

      # URL used if strategy is 'live-url'
      live-url: "http://localhost:8080/q/openapi?format=yaml"

      # Optional override command (if present, takes precedence over strategy)
      # command: "./mvnw smallrye-openapi:generate-schema -D..."

    # ==========================================
    # 2. FRONTEND CLIENT CODE GENERATION
    # ==========================================
    client:
      # Generation tool:
      #   - 'openapi-generator' : Uses @openapitools/openapi-generator-cli via npx/np
      #   - 'orval'             : Runs Orval configuration (orval.config.ts)
      #   - 'npm-script'        : Delegates to a custom package.json script
      #   - 'custom'            : Executes custom command provided in `command`
      tool: openapi-generator

      # Target directory where generated TypeScript interfaces and services are writt
      target-dir: "frontend/src/app/core/api"

      # Generator type (for 'openapi-generator'):
      #   - typescript-angular (default for Angular)
      #   - typescript-fetch
      #   - typescript-axios
      generator: typescript-angular

      # Key-Value additional-properties passed to openapi-generator-cli
      additional-properties:
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

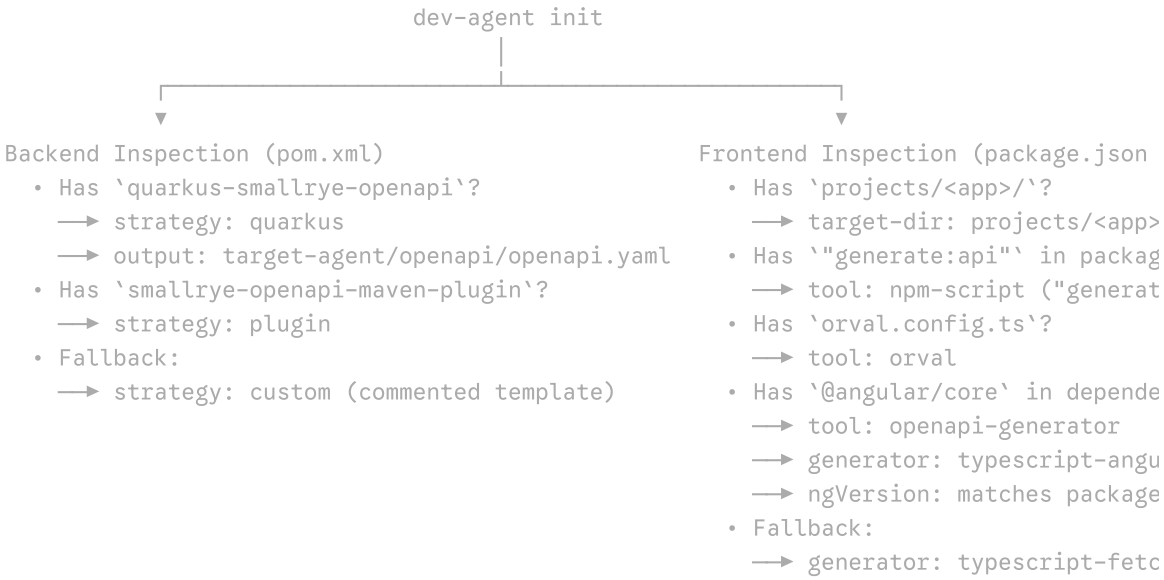
```
    stringEnums: "true"

  # Script name if tool is 'npm-script' (e.g. `npm run generate:api`)
  # npm-script: "generate:api"

  # Custom command if tool is 'custom'
  # command: "npx orval --config orval.config.ts"
```

2. Auto-Detection & Pre-Population Mechanics (`dev-agent init`)

When a developer runs `dev-agent init` , the `ContractContextDetector` probes the workspace to pre-populate `.agents/dev-agent.yaml` with working, tailored defaults:



Detection Logic (`ContractContextDetector.java`)

Java

```
@ApplicationScoped
public class ContractContextDetector {

    public ContractConfig detect(Path repoRoot) {
        SpecConfig spec = detectBackend(repoRoot);
        ClientConfig client = detectFrontend(repoRoot);
        return new ContractConfig(true, spec, client);
    }

    private SpecConfig detectBackend(Path repoRoot) {
        Path pomFile = repoRoot.resolve("pom.xml");
        if (Files.exists(pomFile)) {
```

Gemini-Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
            if (pom.contains("smallrye-openapi-maven-plugin")) {
                return new SpecConfig("plugin", "target-agent/openapi/openapi.yaml",
            }
        }
        return new SpecConfig("quarkus", "target-agent/openapi/openapi.yaml", null, n
    }

    private ClientConfig detectFrontend(Path repoRoot) {
        Path packageJson = findFile(repoRoot, "package.json");
        Path angularJson = findFile(repoRoot, "angular.json");

        String targetDir = "frontend/src/app/core/api";
        String generator = "typescript-angular";
        String ngVersion = "18.0.0";

        // Resolve Angular workspace project sourceRoot
        if (angularJson != null) {
            String sourceRoot = resolveAngularSourceRoot(angularJson);
            targetDir = sourceRoot + "/app/core/api";
        }

        if (packageJson != null) {
            String pkgContent = readString(packageJson);

            // 1. Check for existing npm script hook
            if (pkgContent.contains("\"generate:api\"") || pkgContent.contains("\"cod
                String script = pkgContent.contains("\"generate:api\"") ? "generate:a
                return new ClientConfig("npm-script", targetDir, null, Map.of(), scri
            }

            // 2. Check for Orval
            if (findFile(repoRoot, "orval.config.ts") != null || pkgContent.contains(
                return new ClientConfig("orval", targetDir, null, Map.of(), null, "np
            }

            // 3. Detect Angular version from package.json
            Matcher m = Pattern.compile("\"@angular/core\":\\s*\"[~^]?(\\d+)").matche
            if (m.find()) {
                ngVersion = m.group(1) + ".0.0";
            }
        }

        Map<String, String> properties = Map.of(
            "ngVersion", ngVersion,
            "standalone", "true",
            "providedInRoot", "true",
            "serviceSuffix", "Service"
        );

        return new ClientConfig("openapi-generator", targetDir, generator, properties
    }
}
```

3. Type-Safe Quarkus Mapping (`ContractSettings.java`)

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Java

```java
@ConfigMapping(prefix = "agent.contract")
public interface ContractSettings {

    @WithDefault("true")
    boolean enabled();

    SpecSettings spec();
    ClientSettings client();

    interface SpecSettings {
        @WithDefault("quarkus")
        String strategy(); // 'quarkus', 'plugin', 'live-url', 'custom'

        @WithDefault("target-agent/openapi/openapi.yaml")
        String outputFile();

        Optional<String> liveUrl();
        Optional<String> command();
    }

    interface ClientSettings {
        @WithDefault("openapi-generator")
        String tool(); // 'openapi-generator', 'orval', 'npm-script', 'custom'

        @WithDefault("frontend/src/app/core/api")
        String targetDir();

        @WithDefault("typescript-angular")
        String generator();

        Map<String, String> additionalProperties();

        Optional<String> npmScript();
        Optional<String> command();
    }
}
```

4. Robust Runtime Execution Engine (`ContractSyncTool.java`)

The runtime tool executes the two distinct stages, cleanly formatting options and sanitizing outputs:

Java

```java
@ApplicationScoped
public class ContractSyncTool {

    @Inject
    ContractSettings config;
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
    @Inject
    BackendBuildSystem backendBuild;

    @Inject
    FrontendBuildSystem frontendBuild;

    /**
     * Tool method invoked by the orchestrator between Backend TDD and Frontend TDD
     */
    public ContractSyncResult syncContract() {
        if (!config.enabled()) {
            return new ContractSyncResult(true, "Contract synchronization is disabled
        }

        Path repoRoot = workspaceResolver.getProjectRoot();

        // -----------------------------------------------------
        // Stage 1: Export OpenAPI Spec
        // -----------------------------------------------------
        Path schemaFile = repoRoot.resolve(config.spec().outputFile());
        try {
            Files.createDirectories(schemaFile.getParent());
            exportSchema(repoRoot, schemaFile);
        } catch (Exception e) {
            return new ContractSyncResult(false, "Failed to extract OpenAPI spec: " +
        }

        if (!Files.exists(schemaFile)) {
            return new ContractSyncResult(false, "OpenAPI spec was not created at exp
        }

        // -----------------------------------------------------
        // Stage 2: Generate Frontend Client
        // -----------------------------------------------------
        try {
            generateClient(repoRoot, schemaFile);
        } catch (Exception e) {
            return new ContractSyncResult(false, "Failed generating client SDK: " + e
        }

        return new ContractSyncResult(true, "Contract synchronized successfully into
    }

    private void exportSchema(Path repoRoot, Path outputFile) throws Exception {
        // 1. Explicit user override command
        if (config.spec().command().isPresent()) {
            runShellCommand(repoRoot, config.spec().command().get());
            return;
        }

        // 2. Structured strategies
        switch (config.spec().strategy().toLowerCase()) {
            case "quarkus" -> {
                // Pass dynamic SmallRye properties to compile goal without modifying
                List<String> cmd = List.of(
                    backendBuild.resolveExecutable(repoRoot),
                    "compile",
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
                runProcess(repoRoot, cmd);
            }
            case "plugin" -> {
                List<String> cmd = List.of(
                    backendBuild.resolveExecutable(repoRoot),
                    "smallrye-openapi:generate-schema",
                    "-Dproject.build.directory=target-agent"
                );
                runProcess(repoRoot, cmd);
            }
            case "live-url" -> {
                String url = config.spec().liveUrl()
                    .orElse("http://localhost:8080/q/openapi?format=yaml");
                try (InputStream in = URI.create(url).toURL().openStream()) {
                    Files.copy(in, outputFile, StandardCopyOption.REPLACE_EXISTING);
                }
            }
            default -> throw new IllegalArgumentException("Unknown spec extraction st
        }
    }

    private void generateClient(Path repoRoot, Path schemaFile) throws Exception {
        Path targetDir = repoRoot.resolve(config.client().targetDir());
        Files.createDirectories(targetDir);

        // 1. Explicit user override command
        if (config.client().command().isPresent()) {
            runShellCommand(repoRoot, config.client().command().get());
            return;
        }

        // 2. User npm script
        if ("npm-script".equalsIgnoreCase(config.client().tool())) {
            String script = config.client().npmScript().orElse("generate:api");
            runProcess(repoRoot, List.of("npm", "run", script));
            return;
        }

        // 3. Orval integration
        if ("orval".equalsIgnoreCase(config.client().tool())) {
            runProcess(repoRoot, List.of("npx", "orval"));
            return;
        }

        // 4. Default: openapi-generator-cli
        List<String> cmd = new ArrayList<>(List.of(
            "npx", "@openapitools/openapi-generator-cli", "generate",
            "-i", schemaFile.toString(),
            "-g", config.client().generator(),
            "-o", targetDir.toString()
        ));

        // Format additional-properties comma-separated string
        if (!config.client().additionalProperties().isEmpty()) {
            String props = config.client().additionalProperties().entrySet().stream()
                .map(e -> e.getKey() + "=" + e.getValue())
                .collect(Collectors.joining(","));
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
    }

    private void runProcess(Path dir, List<String> command) throws Exception {
        Process process = new ProcessBuilder(command)
            .directory(dir.toFile())
            .redirectErrorStream(true)
            .start();

        if (!process.waitFor(60, TimeUnit.SECONDS) || process.exitValue() != 0) {
            String output = new String(process.getInputStream().readAllBytes(), Stand
            throw new RuntimeException("Command failed [" + String.join(" ", command)
        }
    }

    private void runShellCommand(Path dir, String command) throws Exception {
        // Parse into discrete arguments safely without raw shell concatenation
        List<String> parts = List.of(command.split("\\s+"));
        runProcess(dir, parts);
    }
}
```

5. Why This Eliminates Developer Friction

1. **Immediate Out-of-the-Box Flow**: For 90% of Quarkus + Angular projects, running `dev-agent init` produces a `.agents/dev-agent.yaml` that **just works** with zero edits.
2. **One-Line Overrides**:

   If a team already has an npm script:

   YAML

   ```yaml
   client:
     tool: npm-script
     npm-script: "codegen"
   ```

   If a team uses Orval:

   YAML

   ```yaml
   client:
     tool: orval
   ```

   If a team has a custom backend OpenAPI generation plugin:

   YAML

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

3. **No Code Re-compilation**: The developer adjusts YAML properties or toggles `enabled: false` to test backend-only changes without touching Java code or rebuilding the GraalVM native binary.

Look through the rest of the architecture, if we can leverage user overrides in other places

Auditing the architecture through the lens of **declarative user overrides** reveals several places where hardcoded assumptions can create friction.

By applying the same philosophy used for the OpenAPI contract sync—**auto-detecting sensible defaults, but exposing structured YAML overrides in `.agents/dev-agent.yaml` and global overrides in `~/.config/dev-agent/settings.json`**—we give developers full control over non-standard layouts, enterprise constraints, and personal workflows without requiring engine modifications.

The Universal Configuration Cascade

All overrides follow a strict 5-tier precedence hierarchy:

```
1. CLI Flags (--yes, --offline, --dir, --gui, --max-retries)  [Highest Precedence]
2. Environment Variables (AGENT_*, GEMINI_API_KEY)
3. Project Scope: Target Repository (.agents/dev-agent.yaml & .agents/config.json)
4. User Scope: Developer Workstation (~/.config/dev-agent/settings.json)
5. Classpath Defaults (Convention over Configuration)         [Lowest Precedence]
```

1. Workspace Layout & Monorepo Overrides

The Friction

Auto-detection works well for standard layouts (`frontend/` , `src/main/webapp/` , root `pom.xml`), but enterprise monorepos (e.g., Nx workspaces, multi-language monorepos, nested microservices) often place projects in non-standard paths.

The Override Solution

Allow developers to declare explicit roots and module mappings:

YAML

```yaml
agent:
  workspace:
    # Explicitly specify repository layout if auto-detection picks the wrong module
    backend-root: "backend/server"
    frontend-root: "apps/web-client"
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
# Custom project name identifier (defaults to Git repo name or remote URL hash)
project-id: "enterprise-omni-core"
```

2. Backend & Frontend Build Engine Overrides

The Friction

The engine defaults to `./mvnw test -Dtest=...` and `npm test -- --watch=false`. However:

- Quarkus projects might require specific Maven profiles (e.g., `-Pdev-tools`, `-Pfast-build`).
- Test suites might need custom runner arguments (e.g., `-Dtest.category=unit` or `--runInBand`).
- Quarkus Dev Services might need to be explicitly toggled.

The Override Solution

Expose fine-grained execution flags and command templates:

YAML

```yaml
agent:
  build:
    backend:
      # Override auto-detected executable (e.g. use system mvn, ./mvnw, or wrapper)
      executable: "./mvnw"

      # Additional Maven/Gradle profiles and properties to append to every command
      extra-args: ["-Pfast-test", "-Dnet.bytebuddy.experimental=true"]

      # Explicitly control Quarkus Dev Services container behavior during inner loops
      dev-services:
        enabled: false # Forces in-memory/mock beans for sub-second inner loops

      # Target directory override (defaults to target-agent/)
      target-dir: "target-agent"

    frontend:
      # Override executable or runner
      executable: "npm"

      # Custom test execution command (e.g. if the project uses a specific script)
      # Defaults to: npm test -- --watch=false --runInBand <specPath>
      custom-test-command: "npm run test:ci -- --include={specPath}"

      # Custom type-check command (defaults to: npx tsc --noEmit)
      custom-type-check-command: "npx tsc --project tsconfig.spec.json --noEmit"
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Hardcoding model names (e.g., Gemini 1.5 Pro, Claude 3.5 Sonnet) into Java annotations breaks when a team wants to standardize on Azure OpenAI, AWS Bedrock, local Ollama, or different model revisions.

The Override Solution

Expose cognitive model dispatching and prompt customizations:

YAML

```yaml
agent:
  models:
    # Role-based model mappings
    planner:
      provider: "gemini"
      model-name: "gemini-1.5-pro"
      temperature: 0.2
    gatekeeper:
      provider: "gemini"
      model-name: "gemini-1.5-flash"
      temperature: 0.0 # Strict deterministic JSON audits
    coder:
      provider: "anthropic" # Routed via OpenAI-compatible endpoint
      model-name: "claude-3-5-sonnet"
      temperature: 0.1
    fallback:
      provider: "ollama"
      endpoint: "http://localhost:11434"
      model-name: "qwen2.5-coder:7b"

  prompts:
    # Allow overriding default system instructions with project-specific Markdown
    planner-prompt-file: ".agents/prompts/planner-custom.md"
    gatekeeper-prompt-file: ".agents/prompts/gatekeeper-custom.md"
```

4. Knowledge Ingestion & Lexical/RAG Search Overrides

The Friction

Default search expects `docs/adr/` or `.agents/rules/`. Some teams maintain documentation in `architecture/decisions/`, `rfcs/`, or single monolithic architecture manuals.

The Override Solution

Make discovery paths and retrieval limits configurable:

YAML

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
  rules-paths:
    - ".agents/rules/**/*.md"
    - "docs/adr/*.md"
    - "doc/architecture/decisions/*.md"
    - "ARCHITECTURE.md"

  # Retrieval tuning
  retrieval:
    engine: "bm25" # 'bm25' (pure native Java) or 'remote-embeddings'
    max-adrs-per-prompt: 4 # Limits prompt bloat (defaults to 3-5)
    min-relevance-score: 0.65

  # Deterministic cache expiry
  cache:
    verdict-cache-ttl: "24h" # Retain cached Architect approvals for 24 hours
```

5. Git Isolation & Worktree Placement Overrides

The Friction

By default, worktrees are created in `.git/agent-worktrees/<task-id>/`. However:

- On developer laptops with slow storage, developers may want worktrees in a RAM disk (e.g., `/dev/shm` on Linux) for fast compilations.
- Teams have strict branch naming conventions (e.g., `feat/JIRA-123-description` vs. `agent/<task-id>`).

The Override Solution

Configurable worktree directories and branch templates:

YAML

```yaml
agent:
  isolation:
    # Enable/disable worktree isolation (if false, runs directly in user.dir)
    use-worktrees: true

    # Base directory for isolated checkouts (defaults to .git/agent-worktrees/)
    # Custom example: ramdisk worktree for ultra-fast I/O
    # base-dir: "/dev/shm/agent-worktrees"

    # Branch naming template
    branch-template: "feat/{task-id}-{story-slug}"

    # Auto-stash developer uncommitted IDE changes before branch creation
    auto-stash-uncommitted: true

    # Automatically prune worktrees on task completion
    auto-prune-on-complete: true
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

The Friction

Teams have different PR guidelines: some require Draft PRs, some disallow them; some require specific labels (`automerge` , `dev-agent`), specific reviewers, or custom CI check timeout durations.

The Override Solution

Declarative pull request and review bot governance:

YAML

```yaml
agent:
  delivery:
    github:
      # Target branch for PRs (auto-detects 'main' or 'master' by default)
      target-branch: "main"

      # Open as draft first (promotes to ready once CI and Sonar pass)
      draft: true

      # Labels applied to generated PRs
      labels: ["dev-agent", "automated-pr", "needs-review"]

      # Optional default reviewers to assign upon promotion
      reviewers: ["senior-dev-team"]

    ci:
      enabled: true
      # Maximum time to poll CI checks before triggering timeout escalation
      timeout: "20m"
      poll-interval: "20s"
      # Regex filter: only wait for critical build/test jobs; ignore optional jobs
      job-filter-regex: "^(build|unit-tests|lint).*"

    sonar:
      enabled: false # Cleanly toggles off SonarCloud gate if project doesn't use it
      project-key: "my-org_my-project"
      fail-on-smells: false # Only fail and self-heal on BUGS or VULNERABILITIES

    coderabbit:
      enabled: true
      max-bot-turns: 2 # Circuit breaker: prevent infinite bot-to-bot arguments
      auto-resolve-dismissed: true # Automatically resolves GitHub comment threads wh
```

7. Economic Guardrails & Financial Budgets (Per-Project & Global)

The Friction

Individual developers need strict spend limits (e.g., $1.00 per task), while enterprise corporate accounts might allow $5.00 per complex multi-module feature.

Gemini-Canvas ausprobieren.

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Expose spend thresholds that can be set per-project or globally in `~/.config/dev-agent/settings.json` :

YAML

```yaml
agent:
  budget:
    # Hard economic stop per task
    max-task-cost-usd: 2.00

    # Maximum token consumption per task
    max-task-tokens: 300000

    # Maximum consecutive self-healing iterations before halting
    max-self-healing-retries: 3

    # Circuit breaker behavior on trip: 'rollback' or 'leave-dirty'
    on-limit-exceeded: "rollback"
```

8. Web Review Dashboard (`--gui`) Overrides

The Friction

Port 9999 might conflict with existing services, or developers running inside headless Docker containers or remote SSH sessions need specific host binds.

The Override Solution

Local dashboard configuration:

YAML

```yaml
agent:
  gui:
    port: 9999
    bind-address: "127.0.0.1" # Use 0.0.0.0 for remote VM/Docker dev environments
    auto-open-browser: true

    # Local Storybook integration
    storybook:
      enabled: true
      url: "http://localhost:6006" # URL embedded in the <iframe> preview
```

Master Configuration Schema (`.agents/dev-agent.yaml`)

Putting this together results in a single, well-documented project configuration file that is **pre-populated by** `dev-agent init` **via auto-detection**, with all advanced options commented out

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

YAML

```yaml
# ================================================================================
# Quarkus Dev-Agent Declarative Configuration (.agents/dev-agent.yaml)
# All sections are optional; defaults are auto-detected at runtime.
# ================================================================================

agent:
  # 1. Project & Workspace Roots
  workspace:
    # backend-root: "."
    # frontend-root: "frontend"

  # 2. Build Tooling & Test Slicing
  build:
    backend:
      # executable: "./mvnw"
      # extra-args: ["-Pfast"]
      dev-services:
        enabled: false # Keep false for fast inner-loop unit TDD
      target-dir: "target-agent"
    frontend:
      # custom-test-command: "npm test -- --watch=false <specPath>"

  # 3. Contract Synchronization (OpenAPI -> TypeScript)
  contract:
    enabled: true
    spec:
      strategy: quarkus # 'quarkus', 'plugin', 'live-url', 'custom'
      output-file: "target-agent/openapi/openapi.yaml"
    client:
      tool: openapi-generator # 'openapi-generator', 'orval', 'npm-script', 'custom'
      target-dir: "frontend/src/app/core/api"
      generator: typescript-angular
      additional-properties:
        ngVersion: "18.0.0"
        standalone: "true"
        providedInRoot: "true"

  # 4. Knowledge Ingestion & ADR RAG
  knowledge:
    rules-paths:
      - ".agents/rules/**/*.md"
      - "docs/adr/*.md"
    retrieval:
      max-adrs-per-prompt: 4
      min-relevance-score: 0.65

  # 5. Git Isolation & Worktrees
  isolation:
    use-worktrees: true
    branch-template: "feat/{task-id}-{story-slug}"
    auto-stash-uncommitted: true

  # 6. Remote Delivery & Quality Gates
  delivery:
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
  ci:
    enabled: true
    timeout: "15m"
  sonar:
    enabled: false
  coderabbit:
    enabled: true
    max-bot-turns: 2

# 7. Financial & Loop Guardrails
budget:
  max-task-cost-usd: 2.00
  max-self-healing-retries: 3

# 8. Web Review Dashboard (--gui)
gui:
  port: 9999
  auto-open-browser: true
  storybook:
    enabled: true
    url: "http://localhost:6006"
```

Summary of Benefits

1. **Zero Boilerplate for Standard Projects:** In a standard Quarkus + Angular repository, `dev-agent init` writes this file with working detected values. The developer never has to touch it unless their setup changes.
2. **First-Class Support for Edge Cases:** Monorepos, custom npm scripts, air-gapped LLM setups, specific Maven profiles, and CI environments are configured with a single line in YAML.
3. **No Binary Recompilation:** Every aspect of the a Accepted - recheck again with scrutiny worktree directories and budget stops—is resolv sub-50ms native startup speeds.

A ruthless, end-to-end scrutiny of the architecture—examining the physical runtime, OS filesystem semantics, GraalVM native image execution, and multi-process lifecycle—reveals **6 subtle failure modes and architectural blind spots**.

These are not high-level design disagreements; they are **physical runtime traps** that will cause deadlocks, file corruption, or infinite loops under real-world development conditions.

Pillar 1: Filesystem & Git Worktrees — The `.agents/runs/` Visibility Paradox

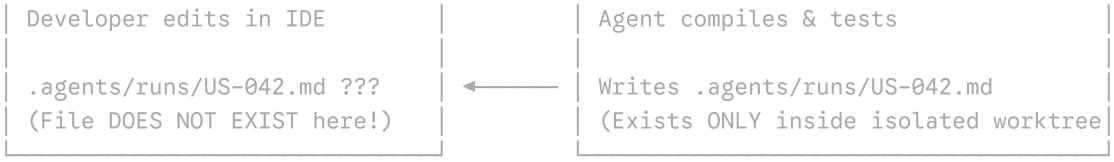
1.1 The Paradox

In Phase 4, the agent executes inside an isolated Git worktree (`.git/agent-worktrees/<task-id>/`) checked out to branch `feat/<task-id>` . Phase 2 specifies that `StateSyncService` continuously projects task status to `./.agents/runs/<task-id>.md` .

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps



The Physical Failure: Because Git worktrees have completely independent working directories, writing to `.agents/runs/<task-id>.md` inside the worktree makes the file **invisible** to the developer in their main workspace on `main`.

The Counter-Trap: If the agent attempts to write directly to the *main workspace* (`projectRoot.resolve(".agents/runs/...")`), it creates an untracked, dirty file in the developer's active workspace, dirtying their `git status` and causing merge or checkout conflicts if the developer switches branches.

The Hardened Solution:

1. The single source of truth remains the global database (`~/.config/dev-agent/data/agent.mv.db`).
2. While the task is in progress, the active journal is written inside the worktree (`.git/agent-worktrees/<task-id>/.agents/runs/<task-id>.md`).
3. When the developer runs `dev-agent status` or `dev-agent logs` from the main workspace, the CLI renders the state **dynamically from the database**, rather than expecting a local file on disk.
4. The `.agents/runs/<task-id>.md` file is committed to the feature branch *inside the worktree* as part of the final PR commit payload, ensuring it merges cleanly with the code.

1.2 The Encapsulated Node Runtime Trap (`frontend-maven-plugin`)

The Physical Failure: In many enterprise Quarkus + Angular projects, Node.js and npm are **not installed globally on the developer's machine** or in the CI container. Instead, they are managed hermetically by `frontend-maven-plugin` (downloaded into `frontend/node/` or `target/node/`).

When `NpmBuildSystem` attempts to run `npx tsc` or `npm test`, it throws:
`java.io.IOException: Cannot run program "npm": error=2, No such file or directory`
.

The Hardened Solution:
Update `NpmBuildSystem.resolveExecutable()` with a 3-step fallback resolution:

```
1. Check project local wrapper: frontend/node/npm (frontend-maven-plugin downloa
2. Check PATH: system `npm`
3. Fail fast with an informative configuration hint:
   "No npm executable found. Specify `agent.build.frontend.executable` in dev-ag
```

Gemini Canvas ausprobieren

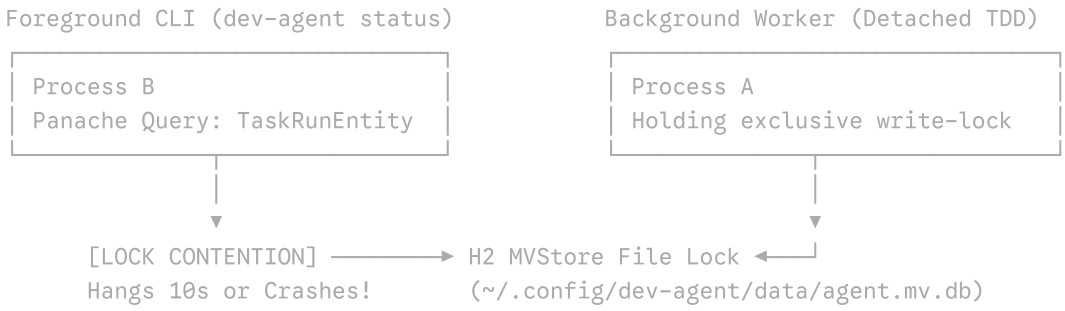
Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

The Setup: A developer starts a background task (`dev-agent feature "Add Auth" --detach`). Process A (the background worker) starts and connects to `~/.config/dev-agent/data/agent.mv.db` . Two minutes later, the developer runs `dev-agent status` in their foreground shell (Process B).

The Physical Failure: H2's underlying MVStore engine uses exclusive file locking on the `.mv.db` file. Even with `FILE_LOCK=SOCKET` or `FILE_LOCK=FS` , Process B attempting a standard Hibernate Panache connection will encounter a lock wait (`LOCK_TIMEOUT`). If Process A is running a transaction, Process B either hangs for 10 seconds or crashes with: `org.h2.jdbc.JdbcSQLNonTransientConnectionException: Database may be already in use`

.



The Hardened Solution:

Decouple Query CLI from Direct DB Locks via Ephemeral State Sockets:
Do not boot a full Hibernate/Agroal connection pool for simple CLI read commands (`status` , `logs`).

When a detached worker runs, it writes an atomic, non-locking JSON heartbeat file into the worktree:
`.git/agent-worktrees/<task-id>/state.json` (`taskId` , `status` , `phase` , `tokens` , `pid`).

`dev-agent status` reads these small JSON files directly across `.git/agent-worktrees/*/state.json` (0 ms, zero database locks).

The H2 database is updated strictly at major phase boundaries and task completion by the owning process, eliminating concurrent read-write contention.

Pillar 3: Build Slicing & Reactor Isolation

3.1 Multi-Module OpenAPI Extraction Failures

The Setup: In a multi-module Quarkus repository (e.g. `repo-core-api` , `repo-core-domain` , `repo-app`), JAX-RS / Quarkus REST endpoints live in `repo-app` .

The Physical Failure: If `ContractSyncTool` executes:
`./mvnw compile -Dquarkus.smallrye-openapi.store-schema-file=true`
from the repository root, Maven attempts to compile all modules. If run with `-pl repo-app` , it fails because `repo-app` depends on `repo-core-api` which hasn't been compiled into `target/` yet.

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Bash

```
./mvnw compile -pl <active-quarkus-app-module> -am \
  -Dquarkus.smallrye-openapi.store-schema-file=true \
  -Dquarkus.smallrye-openapi.store-schema-directory=target-agent/openapi \
  -Dproject.build.directory=target-agent
```

The `-am` (also-make) flag ensures upstream dependencies are compiled into `target-agent/` before SmallRye executes schema analysis.

3.2 Quarkus Dev Services Secret Network Boots

The Setup: We established that `-Dquarkus.devservices.enabled=false` must be passed to prevent Docker container spin-ups during micro-TDD loops.

The Physical Failure: In Quarkus, if `quarkus.devservices.enabled=false` is set, but the production code uses `@DataSource` (PostgreSQL) and no live JDBC URL is provided in `application.properties`, Quarkus **fails fast on startup**:
`Failed to start quarkus: No database URL configured and Dev Services was disabled`
.
The test fails before a single line of test code executes!

The Hardened Solution:
The inner TDD test command must not only disable Dev Services, but also inject an in-memory test database fallback URL:

Bash

```
./mvnw test -Dtest=... \
  -Dquarkus.devservices.enabled=false \
  -Dquarkus.datasource.jdbc.url=jdbc:h2:mem:tdd_test;DB_CLOSE_DELAY=-1 \
  -Dquarkus.datasource.db-kind=h2 \
  -Dproject.build.directory=target-agent
```

This allows tests using Panache or Hibernate to run instantly against in-memory H2 without attempting to pull a PostgreSQL container via Docker.

Pillar 4: Contract Sync — The Generator Blast Radius

4.1 OpenAPI Generator Directory Wiping

The Setup:
`openapi-generator-cli generate -g typescript-angular -o frontend/src/app/core/api`
.

The Physical Failure: By default, `openapi-generator-cli` **wipes the entire output directory** before writing new files.
If the developer had any manual utilities, barrel export files (`index.ts`), or custom token interceptors stored in `frontend/src/app/core/api/`, they are instantly deleted!

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

1. The generator must output to an isolated, dedicated directory: `frontend/src/app/core/api/generated/` (never mixing handwritten code with generated code).
2. Mandate the `.openapi-generator-ignore` template during `dev-agent init`, ensuring `git`, markdown, and configuration files within the target are preserved.
3. After code generation, run an automated Git diff check: `git status --porcelain frontend/src/app/core/api/generated/`. If only timestamps changed but content is identical, run `git checkout --` on those files to prevent diff pollution.

4.2 Angular 17+ Modern Reactivity Mismatch

The Setup: The backend OpenAPI generates a service method returning an RxJS `Observable<UserDto>`.

The Cognitive Trap: The project rules mandate Angular Signals (`signal()`, `computed()`) and forbid manual subscriptions in components (ADR-011). If the agent writes:

TypeScript

```
this.userService.getUser().subscribe(user => this.user = user); // VIOLATION!
```

The Gatekeeper rejects the diff for violating reactive invariants. The Coder gets confused because the generated API client only returns `Observable`.

The Hardened Solution:
Inject explicit bridge guidance into the frontend rules template (`.agents/rules/frontend/angular-rules.md`):

> "When consuming OpenAPI services in Angular components, convert RxJS Observables to Signals at the boundary using `toSignal(userService.getUser(), { initialValue: null })` from `@angular/core/rxjs-interop`. Never use raw `.subscribe()` in component controllers."

Pillar 5: Secret Redaction False-Positive Traps

5.1 Content-Addressable Storage (CAS) Hash Corruption

The Setup: Phase 2 introduced `SecretRedactionFilter` with regexes like: `Pattern.compile("sk-[a-zA-Z0-9]{48}")` and `(?i)basic\\s+[a-zA-Z0-9_\\-+/=]+`.

The Physical Failure:
In a repository handling CAS digests, Netty buffers, or Maven checksums, binary base64 strings and hash representations frequently match these naive regexes:

> A test fixture containing a base64-encoded icon: `...`
>
> Gemini Canvas ausprobieren
>
> An HTTP authorization test asserting Basic auth header decoding:

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Result: The filter redacts the test assertion to `assertEquals("***REDACTED_CREDENTIAL***", header);`. The test fails! The LLM tries to self-heal and corrupts the test suite.

The Hardened Solution:

Context-Aware Redaction: Never apply `SecretRedactionFilter` to raw Java/TypeScript source code or test files!

Redaction must be strictly quarantined to:

1. Tool log outputs before persisting to H2 `StepAuditEntity`.
2. Remote HTTP headers sent to GitHub/SonarCloud.
3. Markdown runbook headers (`.agents/runs/<task-id>.md`).

Refine the Basic Auth regex to require exact token boundaries and minimum lengths to avoid matching small test strings.

Pillar 6: Cognitive & Remote Loop Edge Cases

6.1 GitHub Outdated Review Thread Desynchronization

The Setup: In Phase 5, CodeRabbit posts a review comment on line 42 of `UserService.java`.

The Physical Failure: The agent applies a fix to `UserService.java`. Pushing this commit creates a new commit SHA (`sha2`). In GitHub, line 42 on the previous commit is now considered **Outdated** (`isOutdated == true`).
If the agent subsequently tries to call:
`githubClient.replyToReviewComment(..., commentId, ...)`
GitHub's REST API can reject the reply or fail to associate it with the active diff.

The Hardened Solution:

1. The agent must resolve threads via **GraphQL Node IDs**, not legacy REST endpoints.
2. Use the GraphQL mutation `resolveReviewThread(input: {threadId: $threadId})`. GitHub GraphQL thread IDs remain valid and resolvable even when the diff hunk becomes outdated following a new push.

6.2 Concurrent Detached Worker Resource Exhaustion (OOM)

The Setup: A developer invokes:

Bash

```
dev-agent feature "Task 1" --detach
dev-agent feature "Task 2" --detach
dev-agent feature "Task 3" --detach
```

The Physical Failure: Three detached JVMs start simultaneously. Each launches a Maven test execution ([illegible]). Maven forks Surefire JVMs with default heap limits (often 2–4

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

The Hardened Solution:

Implement a **Host Worker Concurrency Gate (`LocalWorkerLimiter.java`):**

Maximum concurrent background workers on a single developer machine: **Default = 2** (configurable in `~/.config/dev-agent/settings.json` via `agent.max-concurrent-workers`).

If a 3rd task is launched with `--detach` , it enters a `QUEUED` state in H2 and waits on a file lock until an active worker releases its worktree.

Ensure `MavenBuildSystem` passes memory ceiling flags to Surefire: `-DargLine="-Xmx512m -XX:+TieredCompilation -XX:TieredStopAtLevel=1"` This caps test execution heap to 512 MB and cuts JVM startup time by half during inner loops.

The Hardened Invariants Matrix

Component	Identified Runtime Trap	Concrete Physical Failure	Harde
Git Worktrees	`.agents/runs/` written in worktree	File invisible to developer on `main` branch	`statu` directly
npm Execution	`npm` / `npx` not installed globally	`IOException: Cannot run program "npm"`	Probe `front` `PATH`
Database Concurrency	Process B queries H2 while Worker A writes	MVStore lock crash: `Database already in use`	Use ligh `.git/` heartb
OpenAPI Multi-Module	SmallRye run on module without upstream compile	Compilation fails on missing reactor dependencies	Pass `-pl <` `Dproj` `agent`
Dev Services	`devservices.enabled=false` without DB URL	Quarkus startup crash: `No database URL configured`	Pass in `-Dquar`
OpenAPI Codegen	Directory wiping deletes custom code	Deletes non-generated files; pollutes Git diff	Output use `.g`
Angular Reactivity	Generated API returns `Observable` ; rules require Signals	Gatekeeper rejects code; Coder loops indefinitely	Inject e into fro
Secret Redaction	Greedy regex matches base64 / CAS digests	Corrupts test assertions; triggers self-healing breaks	Quaran never r
Review Remediation	Pushing fix commit outdates line numbers	REST reply fails on outdated review diff hunks	Resolve `resol`
Process Management	3+ detached workers run `mvn test` in parallel	JVM heap exhaustion / OS freezes from concurrent builds	Enforc pass `-`

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

With these physical and operational traps resolved, the architecture is hardened against both OS-level and framework-specific race conditions. The separation between the **generic CLI engine**, the **polymorphic SPI strategies**, and the **declarative project rules (`.agents/`)** remains clean, robust, and extensible.

Then update the architecture accordingly and provide it as a downloadable Markdown asset to be added to the repository

I will now generate the comprehensive, production-hardened technical architecture specification for the generalized full-stack Dev-Agent as a downloadable Markdown document.

This updated specification incorporates all operational, physical runtime, and extensibility requirements:

1. **Two-Axis SPI Separation**: Decouples **Build Systems** (Maven/npm active, Gradle/pnpm extensible) from **Framework Conventions** (Quarkus/Angular active, Spring Boot/React extensible).
2. **Native Runtime Safety**: Replaces in-process ONNX JNI bindings with a pure-Java **BM25 Lexical Inverted Index** for local ADR retrieval, eliminating native compilation crashes across operating systems.
3. **Worktree Isolation & Ephemeral State Sockets**: Resolves database lock deadlocks and filesystem visibility issues by pairing dedicated Git worktrees with atomic `state.json` heartbeats for non-locking CLI status reads.
4. **Target Slicing & Container Suppression**: Enforces `-Dquarkus.devservices.enabled=false` alongside in-memory database URL injection and `.git/info/exclude` management, guaranteeing fast, non-blocking inner micro-TDD loops.
5. **Contract Synchronization & Modern Reactivity**: Provides declarative OpenAPI export and TypeScript generation into isolated `generated/` directories, with explicit guidance for bridging RxJS observables to Angular Signals.

Generalized Dev-Agent Technical Architecture Specification

Technical Architecture Specification: Generalized Full-Stack Dev-Agent (`quarkus-dev-agent`)

1. Executive Summary & Foundational Paradigm

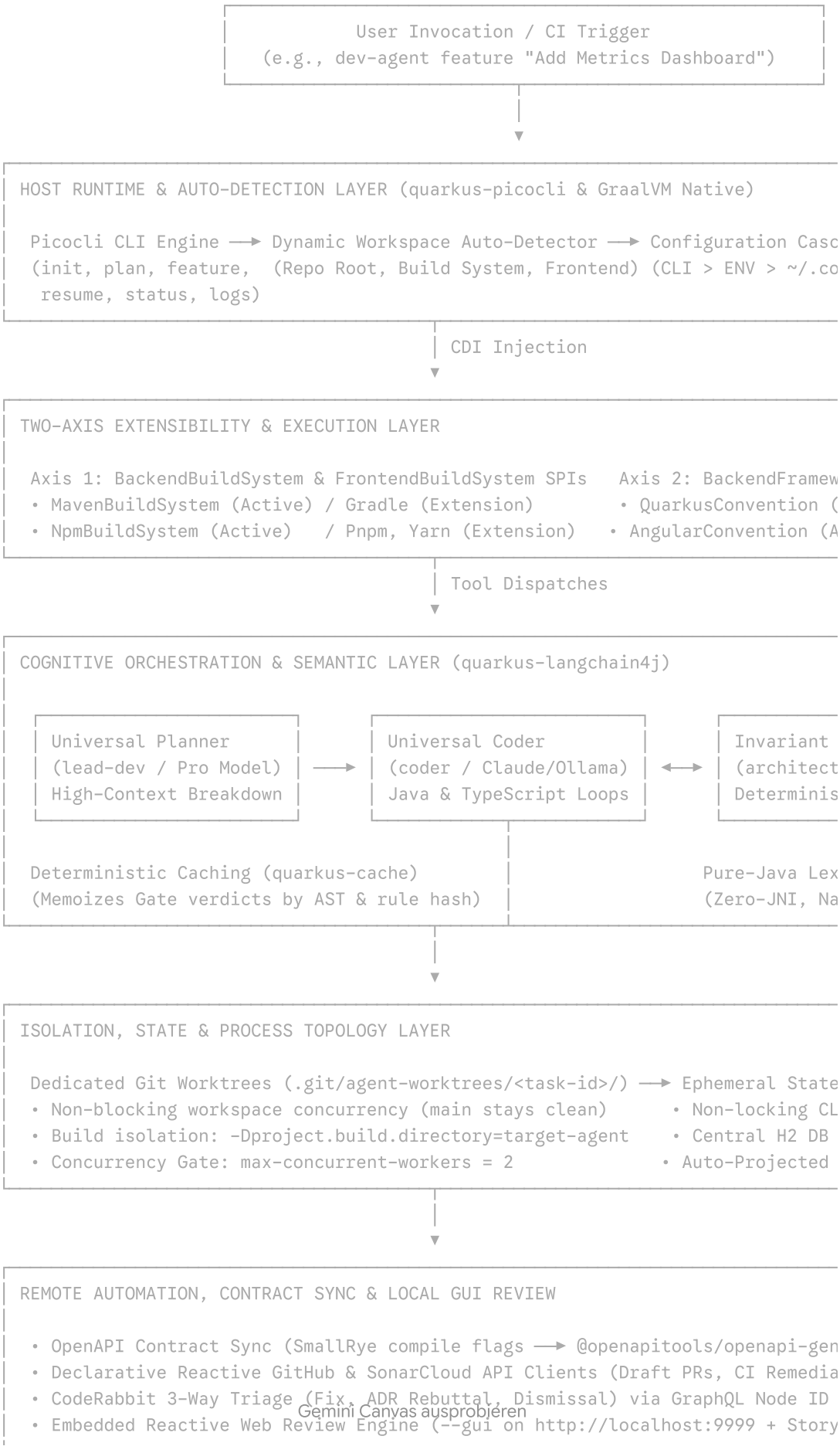
The **Generalized Dev-Agent** (`quarkus-dev-agent`) is an autonomous software engineering CLI, pair programmer, and quality orchestrator built on **Quarkus 3.x**, **Picocli**, and **Quarkus-LangChain4j**.

The system operates as an **independent developer infrastructure tool** hosted in its own repository, compiled via GraalVM/Mandrel into a globally installed native executable (`~/.local/bin/dev-agent`) with sub-50ms cold boot times. It is completely decoupled

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps



Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

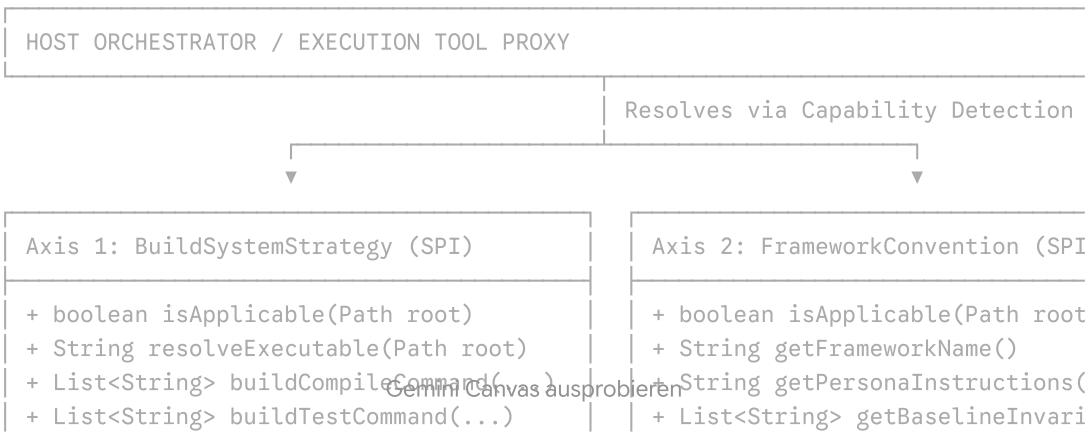
Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Core Architectural Invariants

1. **Engine vs. Knowledge Separation**: The CLI engine contains **zero hardcoded project domain rules, framework keywords, or fixed documentation paths**. The binary is a zero-knowledge execution engine; target projects provide their declarative guidelines in `.agents/rules/` and architecture decision records in `docs/adr/`.
2. **Two-Axis Open-Closed Principle (OCP)**: Build tooling (Maven, npm) and framework idioms (Quarkus, Angular) are decoupled into independent Service Provider Interfaces (SPIs). Adding Gradle, pnpm, Spring Boot, or React requires implementing isolated SPI beans with zero modifications to orchestrator, database, worktree, or CLI commands.
3. **Workspace Concurrency & Physical Isolation**: Background execution runs strictly inside dedicated Git worktrees (`.git/agent-worktrees/<task-id>/`) with build output slicing (`target-agent/`). The developer's primary working tree (`user.dir`) remains unlocked, untouched, and unpolluted.
4. **Zero-JNI Native Safety**: Local semantic rule discovery relies on a pure-Java lexical BM25 inverted index, eliminating C++ native library JNI bindings that cause native image compilation and execution failures across operating systems.
5. **Contract-Driven Full-Stack Synchronization**: Backend data contracts and client models never drift. The agent exports OpenAPI schemas directly from build artifacts and regenerates typed TypeScript clients before frontend implementations begin.
6. **Multi-Tenancy & Ephemeral State Sockets**: Global relational state resides in an embedded H2 database (`~/.config/dev-agent/data/agent.mv.db`) namespaced by project. To prevent database lock deadlocks between background workers and foreground CLI status queries, active tasks emit atomic `state.json` heartbeats within their worktrees.

2. Two-Axis Extensibility Architecture (Build & Framework SPIs)

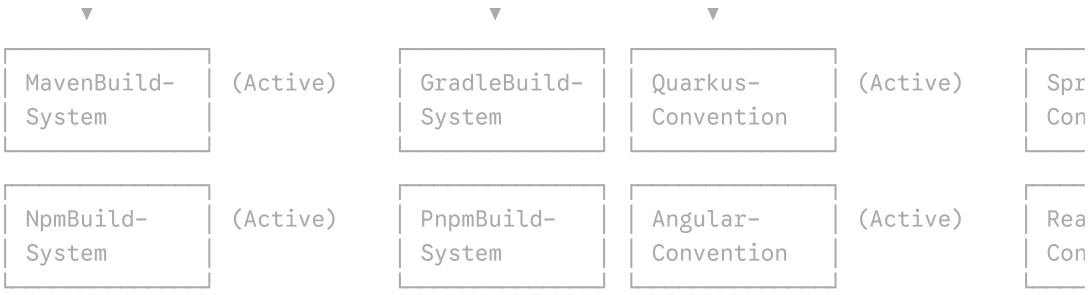
To guarantee that the agent begins with **Quarkus/Maven** and **Angular/npm** while remaining open for **Gradle**, **pnpm**, **yarn**, **bun**, **Spring Boot**, or **React**, capabilities are split into two orthogonal CDI extension axes.



Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps



Axis 1: Build Tooling Execution SPI (`BuildSystemStrategy.java`)

Java

```java
public interface BackendBuildSystem {
    int priority();
    boolean isApplicable(Path projectRoot);
    String resolveExecutable(Path projectRoot);
    List<String> buildCompileCommand(Path projectRoot, String modulePath, String
    List<String> buildTestCommand(Path projectRoot, String modulePath, String te
    SanitizedLog sanitizeOutput(String rawStdout, String rawStderr);
}

public interface FrontendBuildSystem {
    int priority();
    boolean isApplicable(Path frontendRoot);
    String resolveExecutable(Path frontendRoot);
    List<String> buildTypeCheckCommand(Path frontendRoot);
    List<String> buildTestCommand(Path frontendRoot, String testPattern);
    List<String> buildClientCodegenCommand(Path frontendRoot, Path openApiSchema
    SanitizedLog sanitizeOutput(String rawStdout, String rawStderr);
}
```

Active Implementations

1. `MavenBuildSystem` :
   - Probes for `./mvnw` → `mvn` .
   - Enforces `-Dproject.build.directory=target-agent` to slice build targets.
   - Derives `-pl <relative-module-path> -am` for multi-module reactors; omits `-pl` for single-module repositories.
   - Injects Surefire memory limits: `-DargLine="-Xmx512m -XX:+TieredCompilation -XX:TieredStopAtLevel=1"` .
   - Sanitizes output by dropping download progress and task execution headers, isolating `[ERROR]` compilation lines and Surefire assertion failures.

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Resolves executable via an embedded fallback chain: checks `frontend/node/npm` (downloaded by `frontend-maven-plugin`) before searching system `PATH`.

Enforces headless, non-interactive execution flags: `npm test -- --watch=false --browsers=ChromeHeadless` (or `--watch=false --runInBand --ci` for Jest/Vitest), avoiding terminal hangs.

Executes type checks via `npx tsc --noEmit`.

Compacts npm and Vite/Webpack noise down to error codes (`TS2304`, `TS2322`) and failing assertions.

Axis 2: Framework Conventions & Idioms SPI (`FrameworkConvention.java`)

Java

```java
public interface BackendFrameworkConvention {
    int priority();
    boolean isApplicable(Path projectRoot);
    String getFrameworkName();
    String getPersonaInstructions();
    List<String> getBaselineInvariants();
    List<String> getTestExecutionFlags();
    List<String> getOpenApiExportFlags(Path targetDir);
}

public interface FrontendFrameworkConvention {
    int priority();
    boolean isApplicable(Path frontendRoot);
    String getFrameworkName();
    String getComponentGuidance();
    List<String> getBaselineInvariants();
    Path resolveSourceRoot(Path frontendRoot);
}
```

Active Implementations

1. `QuarkusConvention`:

   Injects CDI scoping invariants (`@ApplicationScoped` vs `@RequestScoped`) and Vert.x reactive non-blocking loop rules.

   Directs TDD tests to use `@QuarkusTest` and `@InjectMock`.

   Dev Services Container Suppressor: Injects `-Dquarkus.devservices.enabled=false -Dquarkus.datasource.jdbc.url=jdbc:h2:mem:tdd_test;DB_CLOSE_DELAY=-1 -Dquarkus.datasource.db-kind=h2` during inner micro-TDD loops to prevent unexpected Docker container pulls and startup crashes.

   Exports OpenAPI schemas by passing `-Dquarkus.smallrye-openapi.store-schema-file=true -`

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

2. `AngularConvention`:

   Parses `angular.json` to resolve the true `sourceRoot` (e.g., `projects/<app>/src`) rather than assuming flat directories.

   Enforces Standalone Components, Material Design 3 design tokens, and Angular Signals reactivity.

   Reactivity Bridge Invariant: Enforces that generated OpenAPI client RxJS Observables are bridged using `toSignal(service.call(), { initialValue: null })` from `@angular/core/rxjs-interop`, forbidding raw `.subscribe()` calls in component controllers.

   Mandates accompanying `*.stories.ts` files for reusable UI components.

3. Host Runtime, Workspace Sandboxing & Path Jails

The host layer is packaged as an independent CLI built on `quarkus-picocli`, configured for Ahead-of-Time (AOT) GraalVM compilation.

Dynamic Workspace Auto-Detection

On invocation, `WorkspaceContextResolver` inspects `System.getProperty("user.dir")`:

1. **Workspace Root (`REPO_ROOT`)**: Discovered by traversing parent directories to locate the root `.git` directory, root `pom.xml`, or `build.gradle`.
2. **Subsystem Discovery**: Scans for frontend roots (`frontend/`, `ui/`, `src/main/webapp/`, or Angular workspace projects) and checks for package configurations.
3. **Target Build Isolation**: Ensures that all build executions write to `target-agent/` (Maven) or `build-agent/` (Gradle). Automatically adds `target-agent/` and `.cache-agent/` to the repository's `.git/info/exclude` to prevent dirtying `git status`.

Sandboxing & Canonical Path Jail (`WorkspaceJailValidator`)

To prevent directory traversal or accidental host corruption:

All path inputs must resolve strictly within the active execution root (`activeWorkingRoot.toRealPath()`):

Java

```java
public Path validateAndResolve(String untrustedPath) {
    try {
        Path realRoot = activeWorkingRoot.toRealPath();
        Path resolved = realRoot.resolve(untrustedPath).toRealPath();
        if (!resolved.startsWith(realRoot)) {
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
            // If file does not yet exist, validate normalized parent
            Path parent = activeWorkingRoot.resolve(untrustedPath).normalize().
            if (!parent.startsWith(activeWorkingRoot)) {
                throw new SecurityException("Jailbreak attempt: " + untrustedPa
            }
            return activeWorkingRoot.resolve(untrustedPath).normalize();
        }
    }
```

Explicitly forbids modifications to `.git/**`, global configurations (`~/.config/dev-agent/**`), or the CLI binary itself.

Tool Scoping via Dynamic Proxy (`ExecutionToolProxy`)

Prevents backend-only projects from receiving frontend LLM tools:

If no frontend subsystem is detected in the workspace, frontend methods (`runFrontendTest`, `generateTypeScriptClient`) return a structural no-op message:

`"System Context: Current project has no frontend subsystem. Focus strictly on backend source."`

4. Cognitive Orchestration & Universal Personas

Cognitive capabilities are managed by `quarkus-langchain4j`. System prompts contain **zero hardcoded framework keywords or project names.** All domain rules and architectural invariants are injected dynamically via template variables.

Java

```
// Universal Role 1: Principal System Planner
@RegisterAiService(modelName = "planner")
public interface FullStackPlannerAiService {
    @SystemMessage("""
        You are a Principal Software Architect.
        Analyze specifications, DTOs, schemas, and UI components.
        Generate a phased implementation DAG adhering to the project's architect
        {rulesContext}
        """)
    String draftPlan(@UserMessage String userStory, @V("rulesContext") String ru
}

// Universal Role 2: Invariant Gatekeeper
@RegisterAiService(modelName = "architect-gate")
public interface UniversalArchitectGatekeeperAiService {
    @SystemMessage("""
        You are the Architectural Invariant Gatekeeper.
        Evaluate the proposed plan or diff against the active project architectu
        {retrievedAdrs}
        Return ONLY valid JSON matching the GateVerdict schema.
        """)
```

Gemini kann Fehler machen. Überprüfen.

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
        boolean approved,
        String summary,
        List<String> violations,
        List<String> recommendations
    ) {}
}

// Universal Role 3: Implementation Synthesizer
@RegisterAiService(
    modelName = "coder",
    tools = { CodeWorkspaceTools.class, ExecutionToolProxy.class }
)
public interface FullStackCodeSynthesizerAiService {
    @SystemMessage("""
        You are the Implementation Engineer.
        1. Write failing tests first (TDD) for backend or frontend layers.
        2. Apply precision modifications using 'patchFile' or 'addImports'. Neve
        3. Synchronize API contracts when backend interfaces change.
        4. Validate changes using targeted test tools.
        """)
    String executeImplementationStep(@UserMessage String instruction);
}
```

Pure-Java Lexical Retrieval Engine (BM25 RAG)

To avoid the GraalVM native JNI traps inherent in C++ ONNX runtimes, the primary local search engine is implemented in pure Java:

- **Dynamic ADR Discovery:** Scans `docs/adr/`, `doc/architecture/decisions/`, `docs/tech/architecture/adrs.md`, `.agents/rules/`, and `ADR.md`.
- **Inverted Index (BM25):** Parses headers, code tags, and rule summaries into an in-memory lexical index.
- **Selective Retrieval:** Queries the index using the task prompt and touched file paths, retrieving the **top 3–5 matching ADRs** with zero external network requests and zero native C-library dependencies.
- **Token Optimization:** Cuts Gatekeeper prompt sizes by over 70%, dropping context token payloads from $\sim 15,000$ to under $3,000$ tokens per turn.

Deterministic Decision Caching (`quarkus-cache`)

Audits performed at `temperature: 0.0` are purely deterministic functions of the code diff and active rule digests:

$$K_{\text{cache}} = \text{SHA-256}(\text{UnifiedDiff} + \text{RetrievedAdrsDigest})$$

Gatekeeper evaluations are annotated with `@CacheResult(cacheName = "architect-verdicts")` backed by Caffeine.

Gemini Canvas ausprobieren

Repeated evaluations of identical patches or comment-only edits return cached

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Normalized Loop Circuit Breaker

Detects oscillatory self-healing loops ($A \to B \to A \to B$):

`ErrorSignatureExtractor` : Normalizes compiler and linter traces by stripping absolute paths, timestamps, line numbers, and column offsets.

Circuit Breaker Logic:

Consecutive Repeat ($A \to A$): Injects an explicit warning into the sliding scratchpad memory.

Oscillating Regression ($A \to B \to A \to B$): Invokes `git reset --hard` to rollback to the snapshot and forces a strategy pivot.

Exhaustion (3 failures after pivot): Trips the circuit breaker, rolls back all workspace edits, and halts with exit code 1.

5. Contract-Driven Full-Stack Synchronization

The contract synchronization engine bridges backend API models with frontend web clients, eliminating contract drift.

YAML

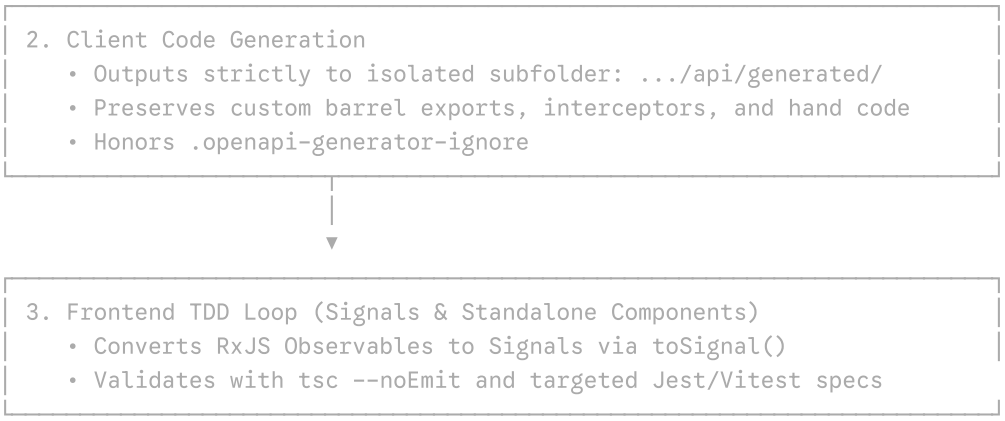
```yaml
agent:
  contract:
    enabled: true
    spec:
      strategy: quarkus # 'quarkus', 'plugin', 'live-url', 'custom'
      output-file: "target-agent/openapi/openapi.yaml"
    client:
      tool: openapi-generator # 'openapi-generator', 'orval', 'npm-script', 'cus
      target-dir: "frontend/src/app/core/api/generated"
      generator: typescript-angular
      additional-properties:
        ngVersion: "18.0.0"
        standalone: "true"
        providedInRoot: "true"
        serviceSuffix: "Service"
        stringEnums: "true"
```

```
Backend Implementation (Quarkus REST/Panache)
                        |
                        ▼
┌──────────────────────────────────────────────────────────────┐
│ 1. Backend Spec Extraction (ContractSyncTool)                │
│    • Injects SmallRye flags during compilation               │
│    • Multi-module reactor safe: -pl <app-module> -am         │
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

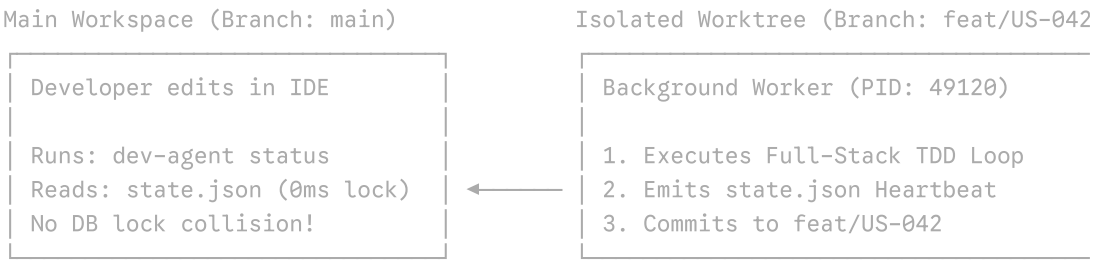


Safety & Blast Radius Governance

1. **Isolated Output Directory**: Generated code writes strictly to `.../core/api/generated/`. It never overwrites the parent directory, preserving manually authored HTTP interceptors, error bridges, or index files.
2. **Diff Noise Suppression**: If a codegen run updates file timestamps without changing schema definitions, the tool executes `git checkout --` on unmodified content, preventing Git diff pollution.

6. Isolation, Multi-Tenant State & Process Topology

To support uninterrupted developer workflows, the Dev-Agent decouples execution from the user's active workspace and prevents multi-process database deadlocks.



Git Worktree Lifecycle Engine (`GitWorktreeService`)

Worktree Allocation: Detached runs create an isolated working directory under `.git/agent-worktrees/<task-id>/` checked out to branch `feat/<task-id>`.

Automatic Working-Tree Hydration: If the developer has uncommitted changes in their IDE, the agent creates an ephemeral stash (`git stash create`) and applies it inside the newly allocated worktree, capturing in-progress thoughts without altering the primary workspace index.

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Non-Locking Ephemeral State Sockets (`state.json`)

To avoid H2 MVStore database lock crashes (`Database may be already in use`) when running status queries while a background worker is compiling:

1. Active background workers write an atomic JSON heartbeat file into their worktree: `.git/agent-worktrees/<task-id>/state.json` containing `taskId` , `status` , `phase` , `tokens` , `cost` , and `pid` .
2. CLI read commands (`dev-agent status` , `dev-agent logs`) read these JSON heartbeats directly without opening JDBC database connections.
3. Central H2 database writes (`~/.config/dev-agent/data/agent.mv.db`) occur strictly at major phase boundaries and task completion, fully namespaced by `project_id` .

Concurrency & Host Resource Guardrails (`LocalWorkerLimiter`)

Worker Limit: Caps simultaneous background worker JVMs at **2** (configurable via `agent.max-concurrent-workers`). Additional detached runs enter a `QUEUED` state.

Surefire Heap Slicing: Inner Maven test forks are bounded with `-Xmx512m -XX:+TieredCompilation -XX:TieredStopAtLevel=1` , preventing workstation memory exhaustion.

Context-Aware Secret Redaction (`SecretRedactionFilter`)

To prevent corrupting binary base64 fixtures, Content-Addressable Storage (CAS) digests, or test assertions (e.g., `assertEquals("Basic dXNlcjpwYXNz", header)`), secret scrubbing is strictly quarantined:

Never applied to raw Java/TypeScript source code or test files.

Strictly applied to tool outputs before writing to H2 `StepAuditEntity` , HTTP Authorization headers sent to remote APIs, and exported Markdown runbook metadata.

7. Declarative Configuration Cascade & User Overrides

Configuration resolves across five tiers, allowing zero-boilerplate defaults for standard repositories while supporting deep overrides for complex monorepos:

```
[1. Picocli CLI Flags]       (--offline, --max-retries=5, --yes, --gui, --dir)
          |
          ▼
[2. Environment Variables]   (GEMINI_API_KEY, AGENT_MAX_CONCURRENT_WORKERS)
          |
          ▼
[3. Project Configuration]   (.agents/dev-agent.yaml & .agents/config.json)
          |
          ▼
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

▼

[5. Classpath Defaults] (Convention over Configuration)

Complete Schema Specification (.agents/dev-agent.yaml)

YAML

```yaml
# ========================================================================
# Quarkus Dev-Agent Declarative Configuration (.agents/dev-agent.yaml)
# Pre-populated via auto-detection on `dev-agent init`.
# ========================================================================

agent:
  # 1. Project & Workspace Roots
  workspace:
    # backend-root: "."
    # frontend-root: "frontend"
    # active-backend-module: "repo-app"

  # 2. Build Tooling & Test Slicing
  build:
    backend:
      # executable: "./mvnw"
      # extra-args: ["-Pfast"]
      dev-services:
        enabled: false # Enforces in-memory/mock beans for fast inner TDD
      target-dir: "target-agent"
    frontend:
      # executable: "npm"
      # custom-test-command: "npm test -- --watch=false <specPath>"
      # custom-type-check-command: "npx tsc --noEmit"

  # 3. Contract Synchronization (OpenAPI -> TypeScript)
  contract:
    enabled: true
    spec:
      strategy: quarkus # 'quarkus', 'plugin', 'live-url', 'custom'
      output-file: "target-agent/openapi/openapi.yaml"
    client:
      tool: openapi-generator # 'openapi-generator', 'orval', 'npm-script', 'cus
      target-dir: "frontend/src/app/core/api/generated"
      generator: typescript-angular
      additional-properties:
        ngVersion: "18.0.0"
        standalone: "true"
        providedInRoot: "true"
        serviceSuffix: "Service"
        stringEnums: "true"

  # 4. Cognitive Model Routing & Dynamic Prompts
  models:
    planner:
      provider: "gemini"
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
    model-name: "gemini-1.5-flash"
    temperature: 0.0
  coder:
    provider: "anthropic"
    model-name: "claude-3-5-sonnet"
    temperature: 0.1
  fallback:
    provider: "ollama"
    endpoint: "http://localhost:11434"
    model-name: "qwen2.5-coder:7b"

# 5. Knowledge Ingestion & ADR Lexical Retrieval
knowledge:
  rules-paths:
    - ".agents/rules/**/*.md"
    - "docs/adr/*.md"
  retrieval:
    engine: "bm25" # Pure Java native-safe search
    max-adrs-per-prompt: 4
    min-relevance-score: 0.65

# 6. Git Isolation & Worktree Placement
isolation:
  use-worktrees: true
  branch-template: "feat/{task-id}-{story-slug}"
  auto-stash-uncommitted: true
  auto-prune-on-complete: true

# 7. Remote Delivery & Quality Gates
delivery:
  github:
    target-branch: "main"
    draft: true
    labels: ["dev-agent", "automated-pr"]
  ci:
    enabled: true
    timeout: "20m"
    poll-interval: "20s"
    job-filter-regex: "^(build|unit-tests|lint).*"
  sonar:
    enabled: false
    project-key: "my-org_my-project"
  coderabbit:
    enabled: true
    max-bot-turns: 2 # Prevents infinite bot-to-bot arguments

# 8. Financial Ceilings & Loop Guardrails
budget:
  max-task-cost-usd: 2.00
  max-task-tokens: 250000
  max-self-healing-retries: 3
  on-limit-exceeded: "rollback"

# 9. Local Web Review GUI (--gui)
gui:
  port: 9999
  bind-address: "127.0.0.1"
```

Gemini Canvas ausprobieren

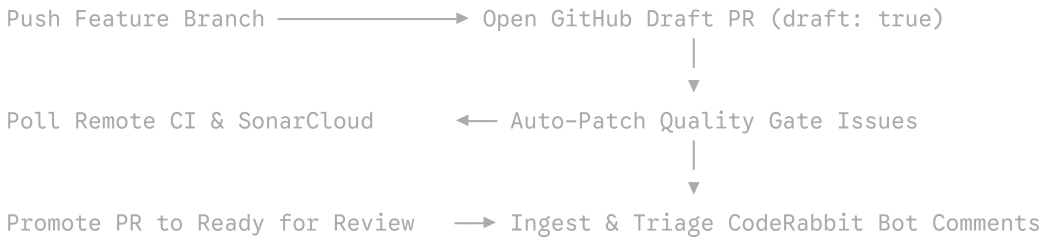
Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

```
enabled: true
url: "http://localhost:6006"
```

8. Remote Quality Gates & Automated Remediation

The remote delivery engine bridges verified local worktrees with GitHub Actions, SonarCloud, and CodeRabbit review bots.



1. GitHub Draft PR Lifecycle & GraphQL Promotion

Pushes the isolated feature branch to `origin/feature/<task-id>`.

Opens a Pull Request in **Draft** state (`draft: true`) via declarative REST clients.

When all remote CI check-runs pass and SonarCloud reports `OK`, promotes the PR to **"Ready for Review"** using a GraphQL mutation:

GraphQL

```
mutation MarkPrReady($pullRequestId: ID!) {
  markPullRequestReadyForReview(input: {pullRequestId: $pullRequestId}) {
    pullRequest { isDraft }
  }
}
```

2. Autonomous CI & SonarCloud Remediation

Remote CI Failure Loop: Downloads failing GitHub Actions check-run logs, passes them through `LogSanitizerService`, synthesizes fixes, applies patches, commits, and pushes up to a maximum of 3 remediation turns.

SonarCloud Auto-Patching: If SonarCloud returns `ERROR`, queries `/api/issues/search`, extracts rule violations (e.g., `java:S1144`), applies local fixes, and pushes commits matching `fix(sonar): resolve <rule-id>`.

Gemini Canvas ausprobieren

3. CodeRabbit Review Triage & GraphQL Thread Resolution

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

1. `ACCEPT_AND_FIX` : Applies code modification, runs local tests, pushes a commit, and marks the thread resolved.
2. `REBUT_WITH_ADR` : Formulates a technical rebuttal citing the project's specific ADR (e.g., defending a zero-copy buffer pattern required by performance invariants).
3. `DISMISS` : Dismisses stylistic nitpicks or invalid recommendations with an explanation.

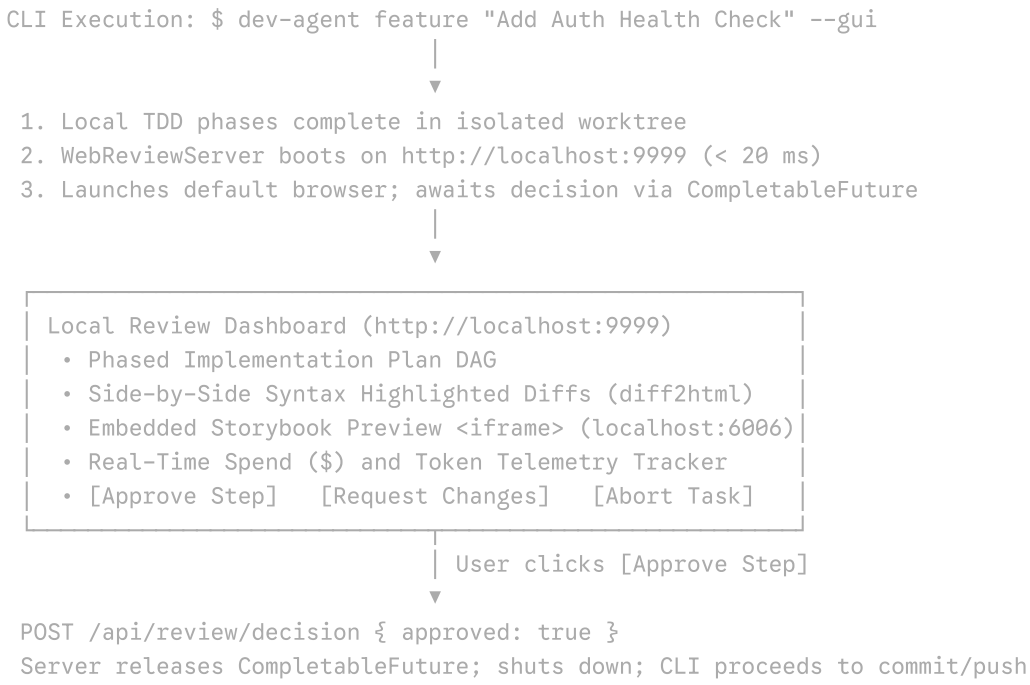
Outdated Diff Resynchronization: Resolves comment threads via **GraphQL Node IDs** (`resolveReviewThread(input: {threadId: $threadId})`), preventing HTTP 422 errors caused by shifting line numbers after fix commits are pushed.

Bot Ping-Pong Breaker: Caps bot-to-bot conversational replies at **2 turns**.

Human Developer Override: If any reply in a thread originates from a non-bot user, automated triage immediately halts and defers unconditionally to the human developer.

9. Local Web Review GUI (`--gui`) & Storybook Preview

To simplify reviewing complex full-stack features, running with `--gui` starts an embedded Vert.x HTTP server on `http://localhost:9999` in under 20 ms.



Zero-CDN Native Assets: CSS and JavaScript assets (`diff2html.min.js` , dark-first Carbon/Cyan themes) are bundled directly inside the native executable under `src/main/resources/web-ui/` .

Visual Component Verification: Embeds an `<iframe>` pointing to the local Storybook development server (`http://localhost:6006`) so engineers can

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

10. Verification, ArchUnit Governance & Implementation Roadmap

ArchUnit Native Compliance Rules

To prevent GraalVM native binary crashes caused by missing reflection registrations or improper CDI scopes:

Java

```java
@AnalyzeClasses(packages = "org.quarkus.agent")
public class AgentArchitectureTest {

    @ArchTest
    public static final ArchRule toolsMustBeApplicationScoped =
        methods()
            .that().areAnnotatedWith(Tool.class)
            .should().beDeclaredInClassesThat().areAnnotatedWith(ApplicationScop

    @ArchTest
    public static final ArchRule recordsAndEntitiesMustRegisterReflection =
        classes()
            .that().resideInAnyPackage("..model..", "..entity..", "..remote..",
            .and().areNotInterfaces()
            .should().beAnnotatedWith(RegisterForReflection.class);
}
```

Evolutionary Walking Skeleton Roadmap

Cycle	Primary Milestone	Strategic Deliverables	Validation S
Cycle 1	**Standalone Full-Stack Walking Skeleton**	Standalone repo, Maven & npm build SPIs, workspace auto-detection, canonical path jail, dual log sanitizer, transactional Git rollback.	Days 1–5
Cycle 2	**Multi-Tenant State & Memory Hardening**	Central H2 database (`~/.config/dev-agent/`), project namespacing, Markdown projection sync, 3-tier memory store, context-aware secret scrubbing.	Week 2

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

Cycle	Primary Milestone	Strategic Deliverables	Validation S
		BM25 search, Caffeine gatekeeper cache (`@CacheResult`), normalized error loop circuit breaker.	
Cycle 4	**Async Isolation & Detached Workers**	Dedicated Git worktrees, build target slicing (`target-agent/`), non-locking `state.json` heartbeats, worker concurrency gate (max 2), desktop alerts.	Week 4
Cycle 5	**Contract Sync & Remote Quality Gates**	OpenAPI → TypeScript generator, Draft PR lifecycle, remote CI self-healing, SonarCloud auto-patcher, CodeRabbit triage state machine.	Week 5
Cycle 6	**N R**	can I use `.git/dev-agent-worktrees/<task-id>/` to keep directory names consistent?	
		budget stops ($2.00 limit), embedded Vert.x Web Diff & Storybook	

Yes, absolutely. You can use `.git/dev-agent-worktrees/<task-id>/` , and it is actually an improvement for namespace and branding consistency across your toolchain.

Why `.git/dev-agent-worktrees/` Works Well

Clear Namespace Alignment: The tool is named `dev-agent` (used in `tools/dev-agent` , `.agents/dev-agent.yaml` , and `~/.config/dev-agent/`). Renaming the base directory from `.git/agent-worktrees/` to `.git/dev-agent-worktrees/` keeps all tool-generated artifacts explicitly prefixed with `dev-agent` , avoiding ambiguity if other agents, scripts, or developers create their own worktrees.

No Git Metadata Collisions: Git internally reserves `.git/worktrees/<name>/` to store administrative metadata (such as individual `HEAD` refs and index files) for linked worktrees. Using `.git/dev-agent-worktrees/` safely isolates the actual checked-out filesystem trees from Git's internal bookkeeping directory.

Gemini Canvas ausprobieren

Automatic Git & IDE Hygiene: Because it resides inside `.git/` , the directory is

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

What Needs to Be Updated

To adopt `.git/dev-agent-worktrees/<task-id>/`, update the corresponding paths in your implementation:

1. `GitWorktreeService.java`

Update the base path definition where worktrees are allocated and released:

Java

```java
@ApplicationScoped
public class GitWorktreeService {

    @Inject
    GitCliService gitCli;

    private final Path repoRoot = Path.of(System.getProperty("user.dir")).toAbsoluteP
    // Updated directory name:
    private final Path worktreesBaseDir = repoRoot.resolve(".git").resolve("dev-agent

    public Path allocateWorktree(String taskId, String branchName) throws IOException
        Files.createDirectories(worktreesBaseDir);
        Path targetPath = worktreesBaseDir.resolve(taskId);
        // ...
        gitCli.runGitDirect("worktree", "add", targetPath.toString(), "-b", branchNam
        return targetPath;
    }

    public void releaseWorktree(String taskId) {
        Path targetPath = worktreesBaseDir.resolve(taskId);
        if (Files.exists(targetPath)) {
            gitCli.runGitDirect("worktree", "remove", "--force", targetPath.toString(
            gitCli.runGitDirect("worktree", "prune");
        }
    }
}
```

2. Startup Pruner (`WorktreeOrphanPruner.java`)

Ensure the automatic cleanup routine scans the updated location for abandoned worktrees:

Java

```java
Path worktreesBaseDir = repoRoot.resolve(".git").resolve("dev-agent-worktrees");
if (Files.exists(worktreesBaseDir)) {
    try (Stream<Path> stream = Files.list(worktreesBaseDir)) {
        // Evaluate task IDs against H2 status and prune dead PIDs
    }
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps

3. Log Tailing & Supervision Commands

Update `dev-agent logs --follow <task-id>` to locate the background worker logs under:

```
.git/dev-agent-worktrees/<task-id>/agent.log
```

4. Configuration Default (`.agents/dev-agent.yaml`)

If you expose the isolation base directory in your YAML configuration cascade, set the default to match:

YAML

```yaml
agent:
  isolation:
    use-worktrees: true
    base-dir: ".git/dev-agent-worktrees"
    branch-template: "feat/{task-id}-{story-slug}"
```

Gemini Canvas ausprobieren

Da Gemini Fehler machen kann, auch bei Informationen über Personen, solltest du die Antworten überprüfen. Dieser Inhalt wurde von einer anderen Person erstellt. Er könnte falsch oder unsicher sein.

Datenschutzerklärung von Google Google-Nutzungsbedingungen Datenschutz & Gemini-Apps